

MI SYS API

Version 3.9

©2022 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
3.0	Initial release	12/05/2020
3.1	- Add new module ID E_MI_MODULE_ID_WBC	01/06/2021
3.2	Add E_MI_SYS_PIXEL_FRAME_YUV422_PLANAR and E_MI_SYS_PIXEL_FRAME_YUV420_PLANAR format in MI_SYS_PixelFormat_e	04/21/2021
3.3	- Add new modules MI_SYS_ChnInputPortSetUserPicture MI_SYS_EnableUserPicture MI_SYS_DisableUserPicture MI_SYS_UserPictureInfo_t`	04/27/2021
3.4	- Delete MI_SYS internal structure hierarchy introduction and Pass introduction - Delete modules MI_SYS_SetGlobalFlag MI_SYS_GlobalFlagType_e MI_SYS_GlobalFlagParam_t - Added insmod parameter description - Modify "Idr handle" to "handle" - Modify modules MI_SYS_BindChnPort2 MI_SYS_SetChnOutputPortDepth MI_SYS_SetChnMMAConf MI_SYS_BindType_e	06/02/2021
3.5	Modify MI_SYS_MMA_Free	06/29/2021
3.6	- Add XVR flow diagram - Add FRC user guide - Add New Frame tile mode	07/22/2021
	Added PROCFS INTRODUCTION	08/25/2021
3.7	- Add bCrcCheck in MI_SYS_BufConf_t and MI_SYS_BufInfo_t - Add u16BufCompressAlignment and u16BufExtraSize in MI_SYS_FrameBufExtraConfig_t	11/05/2021
3.8	Add new pixel format E_MI_SYS_PIXEL_FRAME_RGB888_PLANAR	11/26/2021
3.9	- Add new moudles MI_SYS_SetChnOutputPortUserFrc	11/30/2021
	- Modify 1.6 Insmod Parameter List - Modify 5.20 mma_heap_name0 - Modify 2.5.14 MI_SYS_PrivateDevChnHeapFree	12/08/2021

Revision No.	Description	Date
3.0	Initial release	12/05/2020
3.10	- Add new private pool config MI_SYS_PerDevPrivRingPoolConf_t - Modify memory alignment requirements MI_SYS_Mmap MI_SYS_FlushInvCache - Modify 2.4.2 MI_SYS_BindChnPort2 - Modify 3.2.4 MI_SYS_FrameTileMode_e - Modify 3.2.23 MI_SYS_InsidePrivatePoolType_e - Modify 3.2.29 MI_SYS_GlobalPrivPoolConfig_t	03/30/2022

TABLE OF CONTENTS

REVISION HISTORY	i
TABLE OF CONTENTS	iii
1. Overview	1
1.1. Module Description	1
1.2. Document format constraints	1
1.3. Keyword Description	2
1.4. Flow frame	3
1.4.1 Relationship between Dev/Chn/Port	3
1.4.2 A typical NVR data stream	5
1.4.3 A typical IPC data stream	5
1.4.4 A typical DVR data stream	6
1.4.5 A typical XVR data stream	7
1.5. Bind and Frame Rate Control Scenarios	7
1.5.1 One-to-One Binding	8
1.5.2 One-to-Multiple Binding	9
1.5.3 Set UserDepth While Binding	9
1.5.4 Supplement to VDEC Frame Rate Control in NVR Scenarios	10
1.6. Insmo d Parameter List	11
2. API Reference	12
2.1. API Format Description	12
2.2. Feature Module API List	12
2.3. System functional class API	14
2.3.1 MI_SYS_Init	14
2.3.2 MI_SYS_Exit	16
2.3.3 MI_SYS_GetVersion	17
2.3.4 MI_SYS_GetCurPts	17
2.3.5 MI_SYS_InitPtsBase	18
2.3.6 MI_SYS_SyncPts	19
2.3.7 MI_SYS_SetReg	19
2.3.8 MI_SYS_GetReg	20
2.3.9 MI_SYS_ReadUuid	21
2.3.10 MI_SYS_EnableChnOutputPortLowLatency	22
2.4. Data flow class API	23
2.4.1 MI_SYS_BindChnPort	23
2.4.2 MI_SYS_BindChnPort2	24
2.4.3 MI_SYS_UnBind_Ch nPort	27
2.4.4 MI_SYS_GetBindbyDest	28
2.4.5 MI_SYS_Ch nInputPortGetBuf	29
2.4.6 MI_SYS_Ch nInputPortPutBuf	31
2.4.7 MI_SYS_Ch nOutputPortGetBuf	31
2.4.8 MI_SYS_Ch nOutputPortPutBuf	35
2.4.9 MI_SYS_Ch nInputPortGetBufPa	35
2.4.10 MI_SYS_Ch nInputPortPutBufPa	38

2.4.11	MI_SYS_ChOutputPortGetBufPa.....	38
2.4.12	MI_SYS_ChOutputPortPutBufPa.....	42
2.4.13	MI_SYS_SetChOutputPortDepth.....	43
2.4.14	MI_SYS_ChPortInjectBuf.....	44
2.4.15	MI_SYS_GetFd.....	45
2.4.16	MI_SYS_CloseFd.....	45
2.4.17	MI_SYS_DupBuf.....	46
2.4.18	MI_SYS_ChInputPortSetUserPicture.....	48
2.4.19	MI_SYS_EnableUserPicture.....	50
2.4.20	MI_SYS_DisableUserPicture.....	51
2.4.21	MI_SYS_SetChOutputPortUserFrc.....	52
2.5.	Memory Management Class API.....	52
2.5.1	MI_SYS_SetChnMMAConf.....	52
2.5.2	MI_SYS_GetChnMMAConf.....	53
2.5.3	MI_SYS_MMA_Alloc.....	54
2.5.4	MI_SYS_MMA_Free.....	56
2.5.5	MI_SYS_Mmap.....	56
2.5.6	MI_SYS_FlushInvCache.....	57
2.5.7	MI_SYS_Munmap.....	58
2.5.8	MI_SYS_MemsetPa.....	59
2.5.9	MI_SYS_MemcpyPa.....	60
2.5.10	MI_SYS_BufFillPa.....	61
2.5.11	MI_SYS_BufBlitPa.....	62
2.5.12	MI_SYS_ConfigPrivateMMAPool.....	64
2.5.13	MI_SYS_PrivateDevChnHeapAlloc.....	67
2.5.14	MI_SYS_PrivateDevChnHeapFree.....	68
2.5.15	MI_SYS_Va2Pa.....	69
3.	Data type.....	71
3.1.	Description of data structure Description Format.....	71
3.2.	List of data structures.....	71
3.2.1	MI_ModuleId_e.....	72
3.2.2	MI_SYS_PixelFormat_e.....	75
3.2.3	MI_SYS_CompressMode_e.....	78
3.2.4	MI_SYS_FrameTileMode_e.....	79
3.2.5	MI_SYS_FieldType_e.....	80
3.2.6	MI_SYS_BufDataType_e.....	80
3.2.7	MI_SYS_FrameIspInfoType_e.....	81
3.2.8	MI_SYS_ChPort_t.....	82
3.2.9	MI_SYS_MetaData_t.....	82
3.2.10	MI_SYS_RawData_t.....	83
3.2.11	MI_SYS_WindowRect_t.....	84
3.2.12	MI_SYS_FrameData_t.....	84
3.2.13	MI_SYS_BufInfo_t.....	86
3.2.14	MI_SYS_FrameBufExtraConfig_t.....	87

3.2.15	MI_SYS_BufFrameConfig_t.....	88
3.2.16	MI_SYS_BufRawConfig_t.....	89
3.2.17	MI_SYS_MetaDataConfig_t.....	89
3.2.18	MI_SYS_BufConf_t.....	90
3.2.19	MI_SYS_Version_t.....	91
3.2.20	MI_VB_PoolListConf_t.....	91
3.2.21	MI_SYS_BindType_e.....	92
3.2.22	MI_SYS_FrameData_PhySignalType.....	93
3.2.23	MI_SYS_InsidePrivatePoolType_e.....	93
3.2.24	MI_SYS_PerChnPrivHeapConf_t.....	94
3.2.25	MI_SYS_PerDevPrivHeapConf_t.....	95
3.2.26	MI_SYS_PerVpe2VencRingPoolConf_t.....	95
3.2.27	MI_SYS_PerChnPortOutputPool_t.....	96
3.2.28	MI_SYS_PerDevPrivRingPoolConf_t.....	97
3.2.29	MI_SYS_GlobalPrivPoolConfig_t.....	97
3.2.30	MI_SYS_FrameIspInfo_t.....	98
3.2.31	MI_SYS_FbcData_t.....	99
3.2.32	MI_SYS_BufFrameMultiPlaneConfig_t.....	100
3.2.33	MI_SYS_FrameDataSubPlane_t.....	101
3.2.34	MI_SYS_BufFrameMetaConfig_t.....	102
3.2.35	MI_SYS_FrameDataMultiPlane_t.....	103
3.2.36	MI_SYS_UserPictureInfo_t.....	103
4.	Error Code.....	105
4.1.	The composition of the error code.....	105
4.2.	The error code list for MI_SYS API.....	105
5.	PROCFS INTRODUCTION.....	107
5.1.	common.....	107
5.1.1	dump_mmap.....	107
5.1.2	dump_config.....	108
5.2.	mi_log_info.....	112
5.2.1	cat.....	112
5.2.2	echo.....	114
5.3.	mi_global_info.....	115
5.3.1	cat.....	115
5.4.	mi_bufqueue_status.....	117
5.4.1	cat.....	117
5.5.	mi_dump_buffer_delay_worker.....	118
5.5.1	cat.....	118
5.5.2	echo.....	119
5.6.	mi_infer_graph.py.....	119
5.6.1	cat.....	119
5.7.	mi_graph_contrl.....	120
5.7.1	cat.....	120
5.7.2	echo.....	120

5.8.	debug_level.....	121
5.8.1	cat/echo.....	121
5.9.	debug_file.....	122
5.9.1	echo.....	122
5.10.	debug_func.....	122
5.10.1	echo.....	122
5.11.	module_version_file.....	123
5.11.1	cat.....	123
5.12.	show threads.....	124
5.12.1	echo.....	124
5.13.	debug_frc.....	124
5.13.1	echo.....	124
5.14.	set_outputport_depth.....	125
5.14.1	echo.....	125
5.15.	set_thread_priority.....	125
5.15.1	echo.....	125
5.16.	mmu debug.....	126
5.16.1	echo.....	126
5.16.2	insmod.....	127
5.17.	set_inputport_pattern.....	127
5.17.1	echo.....	127
5.18.	mi_sys0.....	128
5.18.1	cat.....	128
5.19.	miu_protect.....	129
5.19.1	cat.....	129
5.20.	mma_heap_name0.....	130
5.20.1	cat.....	130
5.20.2	echo.....	135
5.21.	imi_mma_heap.....	135
5.21.1	cat.....	135
5.22.	meta.....	137
5.22.1	cat.....	137
5.22.2	echo.....	138
5.23.	mi_modulename devid.....	138
5.23.1	cat.....	138
5.23.2	echo.....	142
5.24.	mem_stat.....	143
5.24.1	cat.....	143

1. Overview

1.1. Module Description

MI_SYS is the foundation module for the entire MI system, and it provides the basis for the operation of other MI modules.

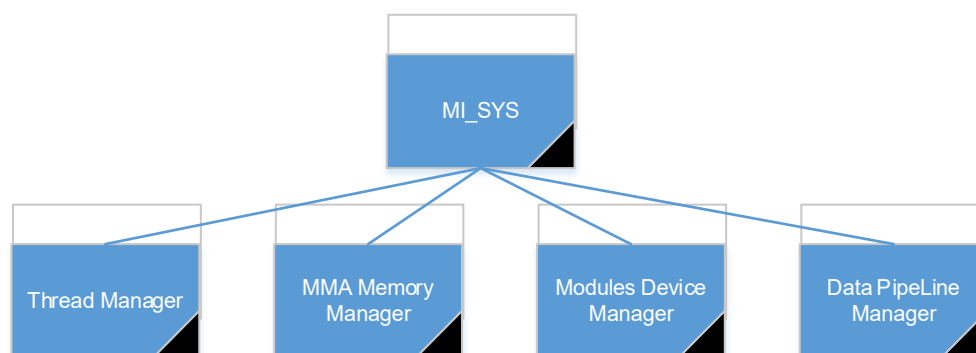


Figure 1-1 : MI_SYS System Framework

As shown in Figure 1 -1, the main functions of the MI_SYS Overview are as follows:

- Realize MI system initialization, MMA memory buffer pool management.
- Provides the registration device nodes of each module, the general interface established by the proc system.
- Provide s/he interface for each module to establish a binding relationship and manage the flow of data between the modules.
- Provide s/he interface for each module to request MMA continuous physical memory, manage memory allocation, map virtual addresses, and reclaim memory.
- Provides the interface of each module to establish the worker thread, manages the creation, operation, destruction of each module thread.

The ko file corresponding to MI_SYS is mi_sys.ko, the library files are libmi_sys.so and libmi_sys.a, and the header files are mi_sys.h and mi_sys_datatype.h. Please refer to the API interface description.

1.2. Document format constraints

Body: for writing the body content of the document, where the writing of the snippet needs to be written in an equal-width font.

Body and bold: For writing important content in the body of a document.

Italics: for writing in the Tips section of the document.

Italics and bold: For writing important content in the Tips section of the document.

1.3. Keyword Description

ID: Identity document, which means unique encoding.

MI: SStar SDK Middle Interface, in this article, a similar structure of "MI_SYS" represents the MI SYS module, a simple "MI" refers to the entire SDK.

Hex: Hexadecimal.

Kernel Mode: refers to code that works in kernel environments and has control over the hardware that operates directly, such as functions and threads in ko.

User Mode: refers to working in a User environment, such as customer applications, system calls, and so on.

APP: Application, which refers primarily to an application that calls the MI API.

API: Application Programming Interface.

NVR: Network Video Recorder.

DVR: Digital Video Recorder.

XVR: A Video Recorder with any one or more functions combinations. Currently Sstar XVR consists of DVR and NVR.

IPC: IP Camera.

HW: Hardware.

Dev: Device, represents for MI module device in this article, see section 1.4.1 for detail.

Chn: Channel, represents a channel of MI module device, see section 1.4.1 for detail.

Port: represents a port in the MI module device channel, see section 1.4.1 for detail.

Soc: System on Chip.

SocId: represents for specific Soc serial number, some platform use PCIE for multi-chip cascading, so SocId needs to be used to distinguish. Some interfaces support cross-Soc settings in MI_SYS, which have SocId parameters. As shown in Figure 1-3, bridging Soc1 and Soc2 by PCIE. Take Soc1 as Master to run Linux environment, MI SDK and customer service program. Take Soc as Slave to run Linux environment and MI SDK. You can choose to operate on Soc1 or Soc2 by calling the API carrying the SocId parameter.

Additional notes:

- Without separate instructions, the MI_SYS in this article have the same meaning as SYS, MI_DISP, and DISP, and the remaining modules have similar names.
- All module names that appear in this article can be found in [MI_ModuleId_e](#).

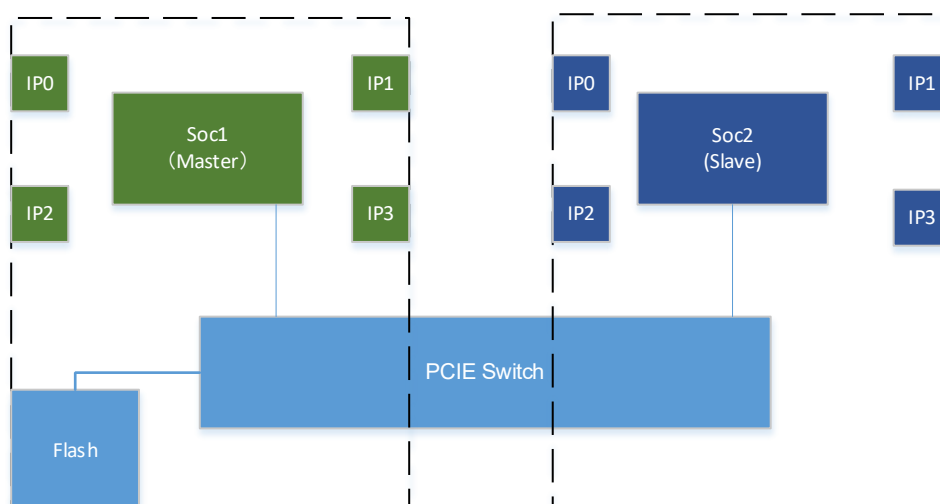


Figure 1-2: PCIe cascaded Soc model

1.4. Flow frame

1.4.1 Relationship between Dev/Chn/Port

A typical MI module will have a Dev/Chn/Port three-stage structure as shown in the figure below.

Dev

An MI module will have one or more Devs, and in general different Devs indicate that the module device needs to invoke different HW resources or work in different working patterns. For example, VENC needs to invoke different HW resources when encoding H264/H265 and Jpeg, and Dev has to separate.

Chn

A Dev will have one or more Chns, which generally mean that the channel is different, and that the channel, although it shares HW resources with other Chns under Dev, is different from the data source or working pattern. For example, the source of the code flow is different, Chn is generally not the same.

Port

A Chn will have one or more ports, including Input Port (for data inflow) and Output Port (for data outflow). Different Ports indicate that the channel shares HW resources and data sources with other Ports under Dev and Chn, but the parameters that need to be set are different, such as different resolutions.

In general, port is the smallest independent unit for the customer to operate the MI module because it identifies all the information: **HW resources, data sources and parameter properties**.

Note: InputPort and OutputPort are not necessary in a chn, it depends on the behaviours of module.

Figure 1 -3 : Three-level structure of the MI module lists the different scenarios of Port arrangement under a single Chn in MI_SYS:

1. InputPort only, which is required for data inflow, but no need for output module. Such as Disp, which display the results on the Panel directly.

2. Output only, which doesnot need for data input, or input data without modules in MI_SYS. Such as VDEC, in which the data can be sent to Vdec RingPool directly by calling the VDEC interface.
3. One InputPort and one OutputPort, which are used by modules for single data input and output. Such as LDC, data input first, and then hardware process to output result.
4. One InputPort and more than one OutputPort, which are used for single data input and diffrent output process to output different data. Such as SCL, share a data source but output various results according diffrent OutputPort parameters.

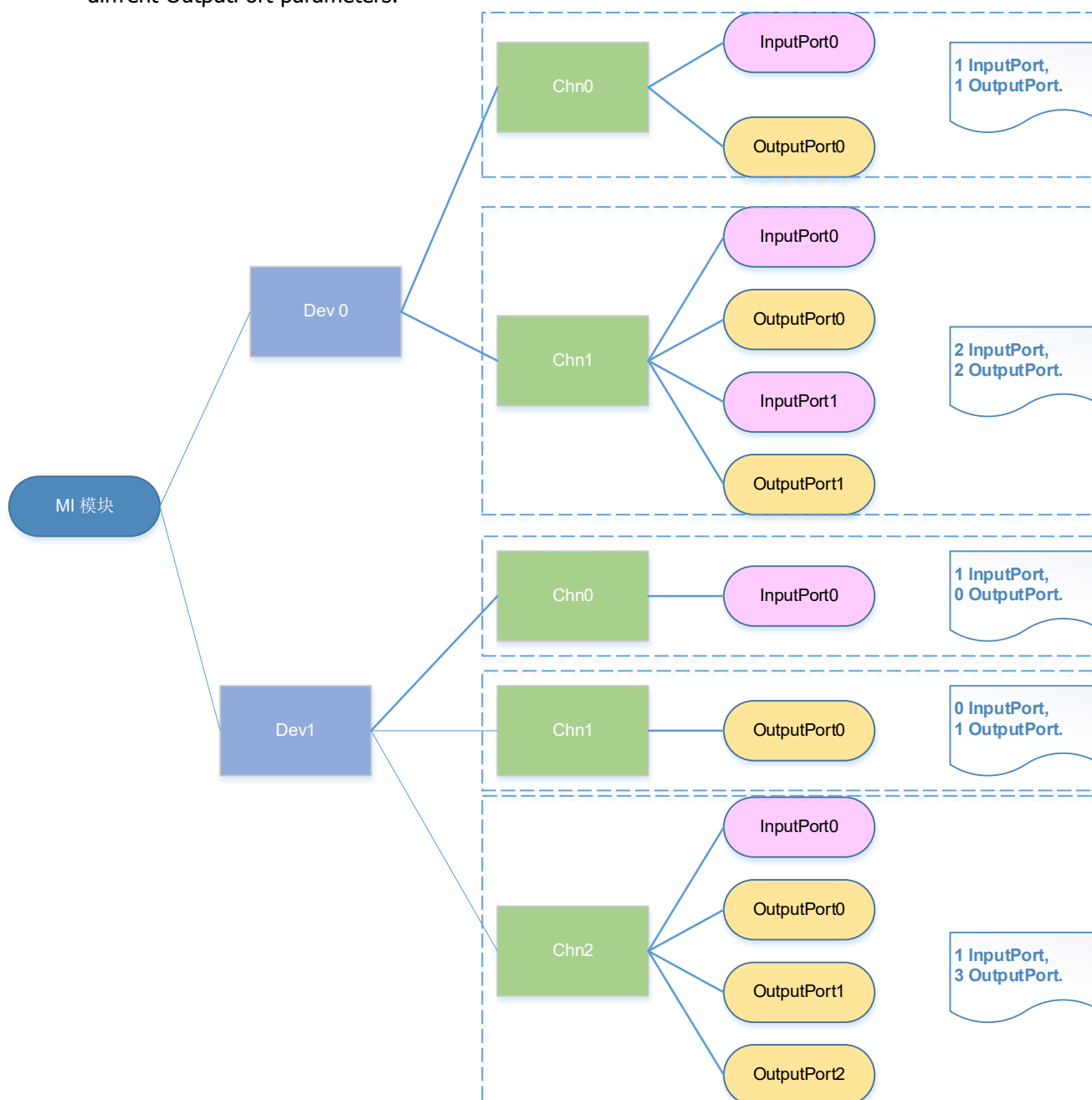


Figure 1-3 : Three-level structure of the MI module

Tips:

The boundaries between Chn and Port are not always clear, if there is a difference between the interpretation of a module's API documentation and the explanation above, call the module to its API documentation.

1.4.2 A typical NVR data stream

The following figure is a typical NVR data flow model. The flow process is as follows:

1. Establishing a binding relationship with Vdec -Disp;
2. User writes a stream to RingPool in Vdec;
3. Vdec decoding, writing decoded data to Vdec OutputPort requested memory, sent to the next level;
4. Disp receive the data and display it.

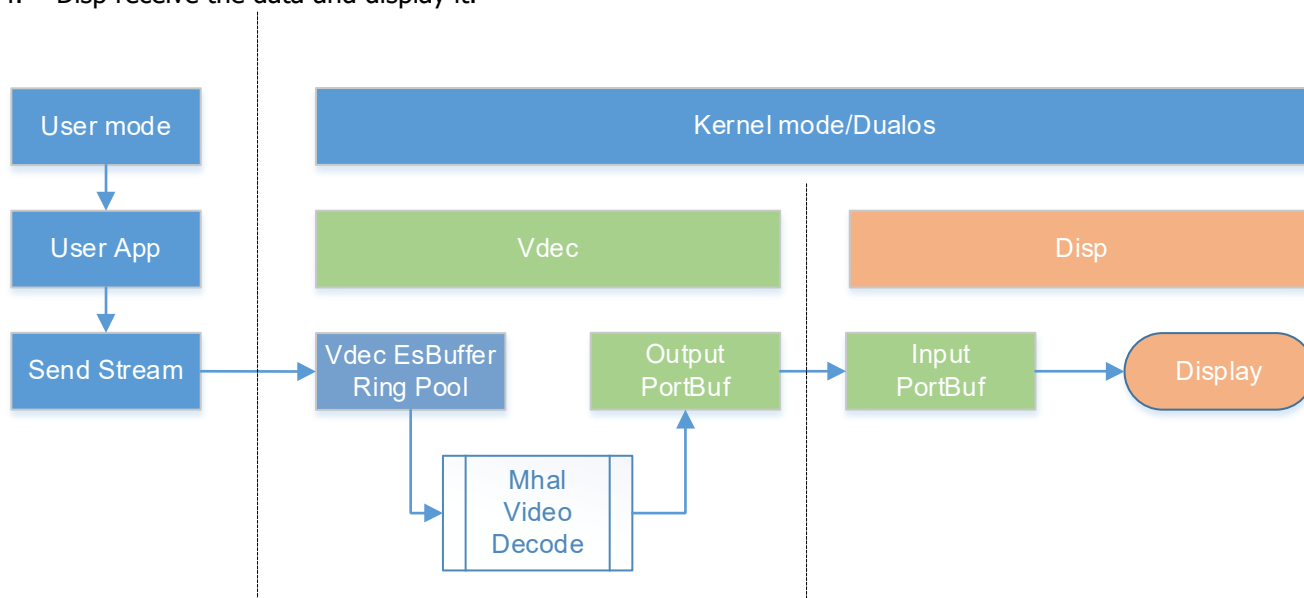


Figure 1-4: A typical NVR data stream model

1.4.3 A typical IPC data stream

The following figure is a typical IPC data flow model with the following flow procedures:

1. Establishing a binding relationship with Vif-Isp-Scl-Venc;
2. Sensor feeds the data into vif processing;
3. Vif sends the processed data to the memory requested by Output Port and sends it to the next level;
4. Isp receives and processes the data, and writes the result into the memory requested by the output port and send it to the next level;
5. Scl receives and processes the data, and writes the result into the memory requested by the output port and send it to the next level;
6. Venc receives the data, feeds the encoder for coding processing, and writes the encoded data to the RingPool memory area;
7. The user calls Venc's interface to retrieve the stream and feeds it into the user's business layer app.

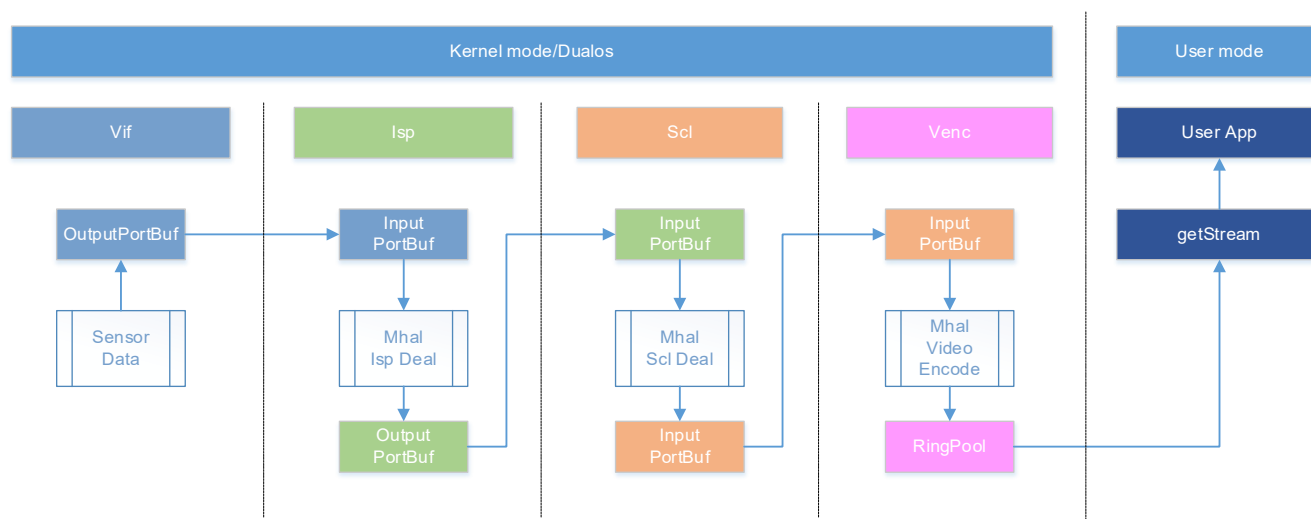


Figure 1-5: A typical IPC data stream model

1.4.4 A typical DVR data stream

The following figure is a typical DVR data stream model, which is divided into two situations according to the input data stream type and application scenarios:

- The input source is a playback scene of the H264/H265 encoded data type read from the disk, and the process is as follows:
 1. Establish the binding relationship of Vdec->Disp;
 2. User writes a stream to RingPool in Vdec;
 3. Vdec decoding, writing decoded data to Vdec OutputPort requested memory, sent to the next level
 4. Disp receive the data and display it.
- The input source is sensor, and the process is as follows:
 1. Establish the binding relationship of Vif->Isp->Scl->Disp;
 2. Sensor feeds the data into vif processing;
 3. Vif writes the processed data into the memory requested by Output Port and sends it to the next level;
 4. Isp receives data for processing, writing the result into the memory requested by Output Port, and sends it to the next level;
 5. SCL receives data for processing, writes the result into the memory requested by Output Port, and sends it to the next level;
 6. Venc receives the data, feeds the encoder for coding processing, and writes the encoded data to the RingPool memory area;
 7. Disp previews the received data;
 8. The user calls Venc's interface to retrieve the stream and feeds it into the user's business layer app.

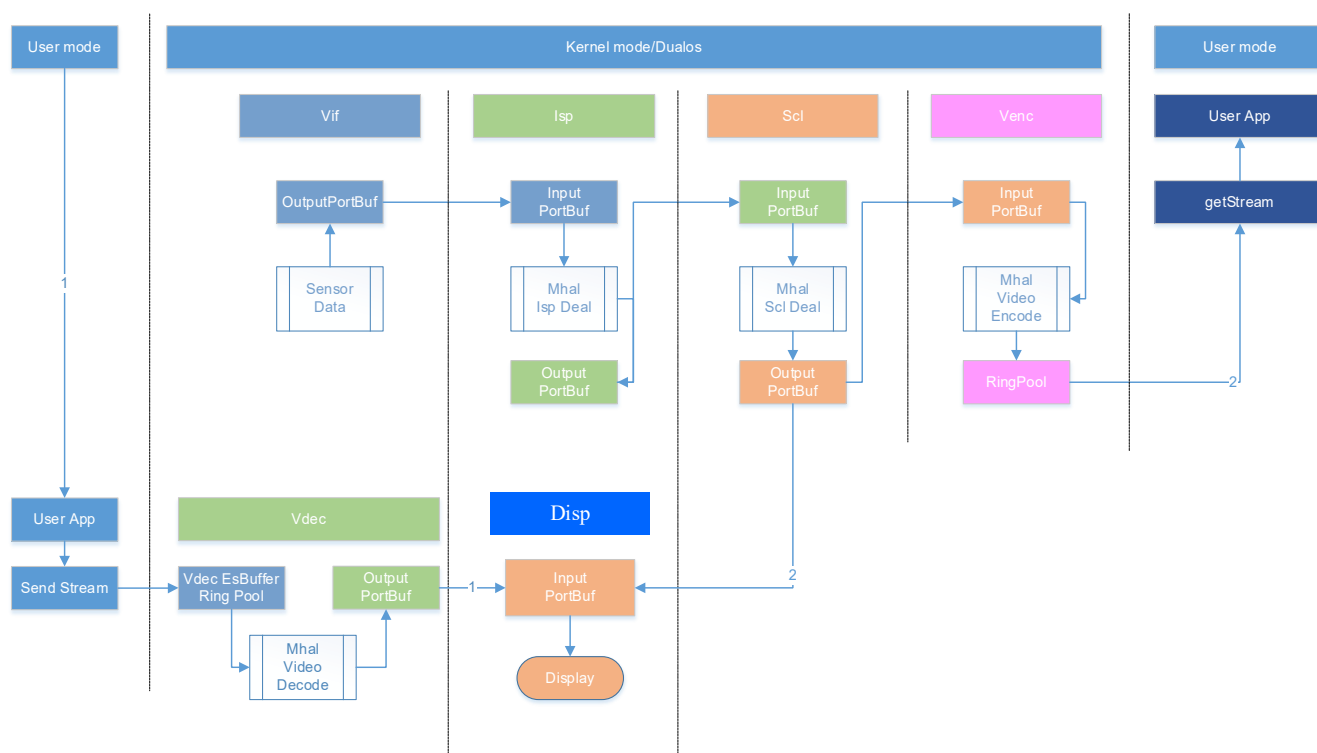


Figure 1-6: A typical DVR data stream model

1.4.5 A typical XVR data stream

XVR has the functions of NVR and DVR, which can switch between the functions of the two devices, and so does the data flow.

1.5. Bind and Frame Rate Control Scenarios

The common data stream scenarios are listed below to illustrate the working mechanism of the frame rate. The members in the figure are defined as follows:

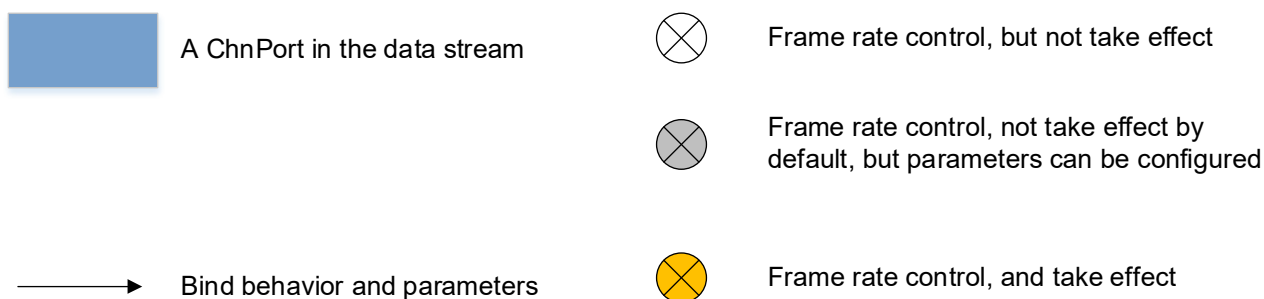


Figure 1-7: Bind scenarios member definition

Tips:

To ensure the ease of use of the interface, MI_SYS will automatically calibrate the frame rate of the previous output.

If there is a difference between the `u32SrcFrmrate` set by user and the internal calculation value of `MI_SYS`, the calculated value will replace `u32SrcFrmrate` for frame rate control. But the calculation takes time, when the frame rate changes greatly, there will be 1-2 seconds of jitter.

In summary, when using the `MI_SYS_BindXXX` interface, please ensure that the two frame rates are set accurately. When the frame rate changes, please call `MI_SYS_BindXXX` actively.

The following assumes that the `u32SrcFrmrate` set by the user is the accurate previous frame rate, and `u32DstFrmrate` is the required next frame rate.

1.5.1 One-to-One Binding

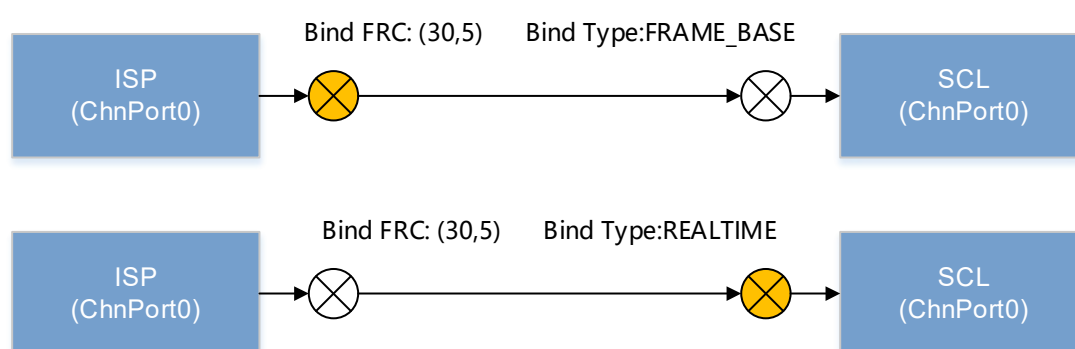


Figure 1-8: Bind scene

Corresponding to the output of the previous Chnport One-to-one binding of the next Chnport input scene, the frame rate control is determined by Bind Type.

If it is in `FRAME_BASE` mode, the previous output frame rate control is effective, and the output frame rate is controlled to the target value, and the next input frame rate control is not effective. If it is `REALTIME/HW_RING` mode, the previous output frame rate control is not effective, then the next input frame rate is directly controlled to the target value.

Take the figure above as an example:

Case1: set Bind FRC: (30, 5), Bind Type:FRAME_BASE, the output frame rate of the previous ISP is controlled to 5FPS.

Case2: set Bind FRC: (30,5) Bind Type:REALTIME, the output frame rate of the previous ISP is maintained at 30FPS, and the input frame rate of the back-end SCL is controlled to 5FPS.

Tips:

Regardless of the scene in `REALTIME/HW_RING` mode, the previous output frame rate control is not effective, and only the next input frame rate can be controlled, which is determined by the hardware.

1.5.2 One-to-Multiple Binding

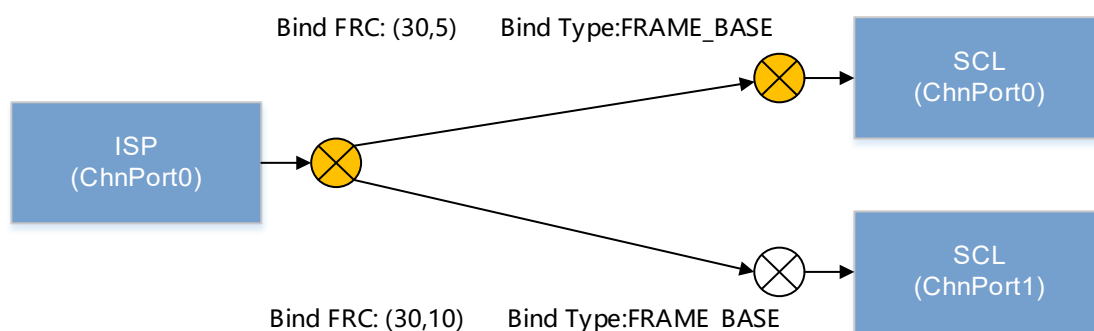


Figure 1-9: Bind scene

Corresponding to the output of the previous Chnport One-to-Multiple binding of the next Chnport input scene, the frame rate control is determined by Bind Type and Bind FRC.

If it is in FRAME_BASE mode, the front-level ISP binds multiple back-levels at the same time, first control the frame rate to meet the frame rate requirements of all the back-level bind modules, and then perform frame rate control separately at the back-level output.

If it is in REALTIME/HW_RING mode, the frame rate control of the previous output will not take effect, and the input of the subsequent stage will control the frame rate separately.

Take the figure above as an example:

In this scenario, the two bind frame rates for ISP ChnPort0 are (30, 5) (30, 10), so the Output frame rate of ISP ChnPort0 is 10FPS.

The input of SCL ChnPort0 needs to control the frame rate from 10FPS to 5FPS. SCL ChnPort1 does not require frame rate control.

1.5.3 Set UserDepth While Binding

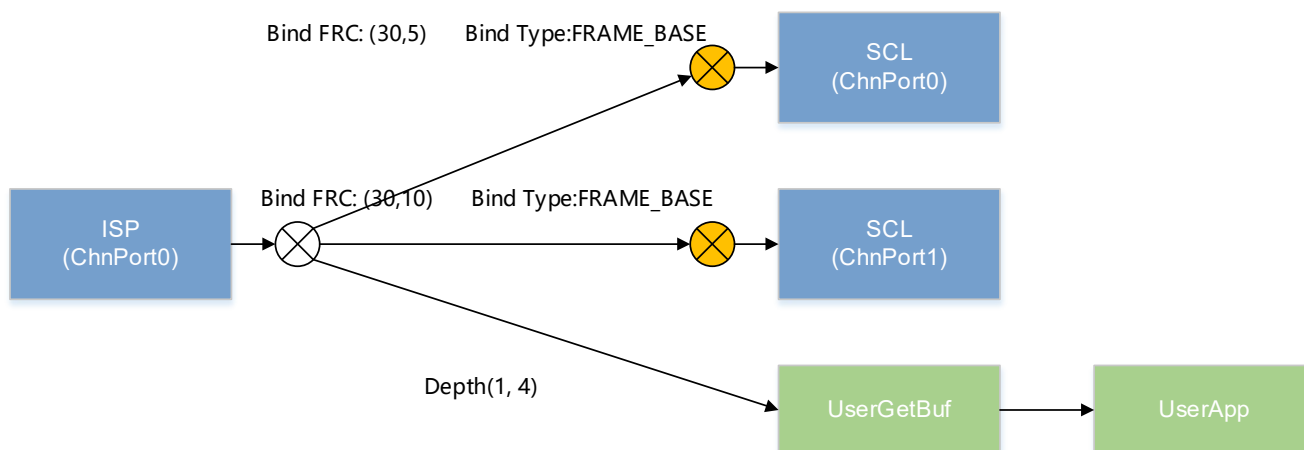


Figure 1-10: Set UserDepth While Binding

The scene of UserDepth is also set in the front output when Bind the rear stage, and the frame rate control is fixed and unique.

When user needs to use the output data, UserDepth set by MI interface will be called, and the previous Output will give user the latest data, so the previous output does not control the frame rate.

If there is a binding at the rear stage, do frame rate control at the input port. If not, the frame rate control will not take effect. At this time, the data output is determined by the speed of data generation and consumption. Explain in detail when explaining the concept of Depth.

Take the figure above as an example:

In this scenario, the two bind frame rates for ISP ChnPort0 are (30, 5), (30, 10), but the Output Depth with ISP ChnPort0 set is (1, 4), and the Output frame rate of ISP ChnPort0 is 30FPS. The input port of SCL ChnPort0 controls the frame rate from 30FPS to 5FPS. SCL ChnPort1 controls the frame rate from 30FPS to 10FPS.

1.5.4 Supplement to VDEC Frame Rate Control in NVR Scenarios

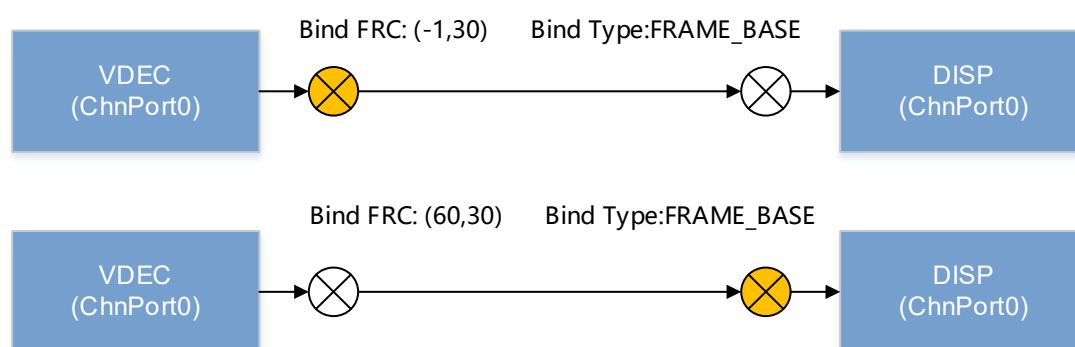


Figure 1-11: Supplement to VDEC Frame Rate Control in NVR Scenarios

Corresponding to Vdec Chnport's Output one-to-one bind post-stage Chnport Input scene, the frame rate control is determined by BindFRC.

NVR requires user to input the code stream to Vdec, and it is difficult to ensure a balanced transmission speed. This is different from IPC, whose data source is strictly controlled by interrupts. If the stability of the incoming frame rate cannot be ensured, you can use the function of automatically calculating the sending frame rate provided by MI, which is automatically enabled when u32SrcFrmrate is set to -1, and MI will decide whether to allocate Buffer to Vdec according to u32DstFrmrate.

NVR provides a double-speed playback function. When the Timing of the subsequent DISP cannot keep up, it needs to actively drop frames before inputting to the DISP. If VDEC decoding speed is faster than DISP display Timing, you can use the proportional drop frame function provided by MI to set the ratio of u32SrcFrmrate and u32DstFrmrate.

Take the figure above as an example:

Case1: set Bind FRC: (-1, 30) Bind Type: FRAME_BASE. Suppose the output FrameRate of the previous Vdec is 30, it takes $1000\text{ms}/30 = 33.3\text{ms}$ for the Vdec Output to get the buffer allocated by MI.

Case2: set Bind FRC: (60, 30) Bind Type: FRAME_BASE, the ratio is $60/30 = 2/1$. Suppose the output FrameRate of the previous Vdec is 60, only one of the two images decoded by the Vdec Output will be sent to Disp Input.

Tips:

- NVR products can query the current ChnPort legacy frames through the API interface of MI_VDEC to dynamically select two strategies to achieve better results
- The function of automatically calibrating the real frame rate of previous Output when Vdec Bind is in the next stage is no longer effective.

1.6. Insmod Parameter List

The parameters that can be carried after insmod mi_sys.ko are as follows.

Table 1-1 Parameter list

Parameter	Description	Value
bEnableMmu	Whether to enable MMU	1: enable mmu 0:disable mmu The default value of bEnableMmu is 1. Ex: bEnableMmu=1, enable mmu bEnableMmu=0, disable mmu
logBufSize	Memory length of cache log	Default: 4KByte; Unit: Byte Ex: logBufSize=4096, set 4096Byte, which is 4KByte
entrySize	Need to be configured after enabled mmu	Tiramisu supports 128/256/512, unit: Kbyte The default value of entrySize is 0, it will use another size setting by system. The setting may be not same on different IC. Ex: entrySize=128, set 128KByte
cmdQBufSize	cmdq memory size	Determined by product specifications, unit: Kbyte cmdQBufSize is set to be 1024 by default on linux, and the default value may be not same on RTK running on different IC. Ex: cmdQBufSize=256, set 256KByte

2. API Reference

2.1. API Format Description

This manual uses information about the Description APIs for the 8 reference domains, which are represented by Table 2-1.

Table 2-2 : API Format Description

Label	Function
Function	Briefly Description the main features of the API.
Syntax	List the header files that should be included in the call API and the prototype declaration of the API.

Parameters	List the parameters, parameter descriptions, and parameter properties of the API.
Return value	List all possible return values of the API and what it means.
Dependency	Lists the header files that the API contains and the library files that the API will link to when the API compiles.
Attention	List the things you should be aware of when using the API.
Example	List instances that use the API.
Related topics	The interface associated with the call context.

2.2. Feature Module API List

As mentioned earlier, we can roughly divide the MI_SYS's API into three broad categories: system functional class, data flow class, memory management class.

Tips:

Only some interfaces of MI_SYS support cross-Soc settings, which all carry the u16SocId parameter. The interfaces that do not carry this parameter do not support cross-Soc settings by default.

Table 2-3 : API List

API Name	Function
System functional class	
MI_SYS_Init	Initialize the MI_SYS system
MI_SYS_Exit	Destructing MI_SYS System
MI_SYS_GetVersion	Get the system version number of MI
MI_SYS_GetCurPts	Get the current timestamp of the MI system
MI_SYS_InitPtsBase	Initializing MI System Baseline Timestamp
MI_SYS_SyncPts	Synchronized MI system timestamp
MI_SYS_SetReg	Set the value of the register, debug with
MI_SYS_GetReg	Get the value of the register, debug with
MI_SYS_ReadUuid	Get Chip's Unique ID
MI_SYS_EnableChnOutputPortLowLatency	Enable or disable output port low latency
Data stream class	
MI_SYS_BindChnPort	Binding of the data source Output port to the recipient Input port
MI_SYS_BindChnPort2	Binding from the output port of the data source to the recipient Input port, requiring a working mode to be specified
MI_SYS_UnBind_ChnPort	The de-binding of the data source Output port to the recipient Input port
MI_SYS_GetBindbyDest	Query the corresponding source Output port for the data recipient Input port
MI_SYS_ChnInputPortGetBuf	Get the buf of channel input Port

API Name	Function
MI_SYS_ChnlInputPortPutBuf	Add the buf of the channel input Port to the pending queue
MI_SYS_ChnlOutputPortGetBuf	Get the buf of the channel output Port
MI_SYS_ChnlOutputPortPutBuf	Release channel outputPort's buf
MI_SYS_ChnlInputPortGetBufPa	The buf object corresponding to the allocation channel input port, which only returns MIU physical address
MI_SYS_ChnlInputPortPutBufPa	Add the buf object corresponding to the channel input port to the pending queue, which needs to be used in pair with the function MI_SYS_ChnlInputPortGetBufPa
MI_SYS_ChnlOutputPortGetBufPa	The buf object corresponding to the allocation channel output port, which only returns MIU physical address
MI_SYS_ChnlOutputPortPutBufPa	Release channel output port corresponding to the buf object, which needs to be used in pair with the function MI_SYS_ChnlOutputPortGetBufPa
MI_SYS_SetChnlOutputPortDepth	Set the depth of the channel OutputPort
MI_SYS_ChnlPortInjectBuf	Inject output port Buf data into module channel input port
MI_SYS_GetFd	Get the file Description character of the current output Port wait event
MI_SYS_CloseFd	File Description character to close the current output Port
MI_SYS_DupBuf	Duplicate buf object
MI_SYS_ChnlInputPortSetUserPicture	According to the set frame rate, the buf will be repeatedly input to the current input port and add to the waiting queue
MI_SYS_EnableUserPicture	Enable user picture
MI_SYS_DisableUserPicture	Disable user picture, release buf
MI_SYS_SetChnlOutputPortUserFrc	Set the frame rate when user get buffer from output port, which will save bandwidth
Memory management classes	
MI_SYS_SetChnlMMAConf	Set the MMA pool name for the default allocation of memory for the module device channel Output port
MI_SYS_GetChnlMMAConf	Get the MMA pool name of the module device channel Output port's default allocated memory
MI_SYS_MMA_Alloc	Application requests physical continuous memory from MMA memory management pool
MI_SYS_MMA_Free	Memory allocated to the MMA memory management pool in the user state
MI_SYS_Mmap	Mapping physical memory to CPU virtual addresses
MI_SYS_Munmap	Un-mapping physical memory to virtual addresses
MI_SYS_FlushInvCache	Flush cache CPU virtual address

API Name	Function
MI_SYS_ConfigPrivateMMAPool	Configure private MMA Heap for mold fast
MI_SYS_PrivateDevChnHeapAlloc	Request memory from module channel private MMA Pool
MI_SYS_PrivateDevChnHeapFree	Free memory from module channel private MMA pool
MI_SYS_Va2Pa	Turn CPU virtual address pointer to Physical address

2.3. System functional class API

2.3.1 MI_SYS_Init

➤ Function

MI_SYS initialization, the MI_SYS module provides basic support for other MI modules in the system and needs to be initialized earlier than other MI modules in the system, otherwise other stream types within the module will fail when initialization.

➤ Syntax

```
MI_S32 MI_SYS_Init(MI_U16 u16SocId);
```

➤ Parameters

Parameter Name	Description	Input/Output
u16SocId	Specify the target Soc for this operation	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

- MI_SYS_Init need to be called earlier than other MI modules in the Init function.
- The MI_SYS_Init can be repeatedly called in the same process or in multiple processes, but must be used in pairs with the [MI_SYS_Exit](#) or an error will be reported.
- The system needs to configure the configuration parameters of the MMA memory heap within the kernel boot parameters.

➤ Sample

MI_SYS init Sample

```
#define ST_DEFAULT_SOC_ID 0
MI_S32 ST_Sys_Init(void)
{
    MI_SYS_Version_t stVersion;
    MI_U64 u64Pts = 0;
```

```

    MI_U16 u16SocId = ST_DEFAULT_SOC_ID;
    MI_U64 u64Uuid;
    STCHECKRESULT(MI_SYS_Init(u16SocId));

    memset(&stVersion, 0x0, sizeof(MI_SYS_Version_t));
    STCHECKRESULT(MI_SYS_GetVersion(u16SocId , &stVersion));
    ST_INFO("u8Version:%s\n", stVersion.u8Version);

    STCHECKRESULT(MI_SYS_ReadUuid(u16SocId, &u64Uuid));
    INFO("u64Uuid:%llx\n", u64Uuid);

    STCHECKRESULT(MI_SYS_GetCurPts(u16SocId, &u64Pts));
    ST_INFO("u64Pts:0x%llx\n", u64Pts);

    u64Pts = 0xF1237890F1237890;
    STCHECKRESULT(MI_SYS_InitPtsBase(u16SocId, u64Pts));

    u64Pts = 0xE1237890E1237890;
    STCHECKRESULT(MI_SYS_SyncPts(u16SocId, u64Pts));

    return MI_SUCCESS;
}

MI_S32 ST_Sys_Exit(void)
{
    MI_U16 u16SocId = ST_DEFAULT_SOC_ID;
    STCHECKRESULT(MI_SYS_Exit(u16SocId));

    return MI_SUCCESS;
}

```

Tips:

This example is intended for: MI_SYS_Init / MI_SYS_Exit / MI_SYS_GetVersion / MI_SYS_GetCurPts / MI_SYS_InitPtsBase / MI_SYS_SyncPts / MI_SYS_ReadUuid .

➤ Related topics

[MI_SYS_Exit](#)

2.3.2 MI_SYS_Exit

➤ Function

MI_SYS initialization, before calling MI_SYS_Exit, you need to make sure that all other modules in the system have been de-initialized and that all VBPOOL has been Destroyed

or the MI_SYS_Exit will return to failure.

➤ Syntax

MI_S32 MI_SYS_Exit (MI_U16 u16SocId);

➤ Parameters

Parameter Name	Description	Input/Output
u16SocId	Specify the target Soc for this operation	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

- MI_SYS_Exit Before calling, make sure that all other modules in the system have been deinitialized.
- MI_SYS_Exit Before calling, you need to make sure that all created VBPOOL in the system has been successfully destroyed.

➤ Sample

Refer to the `Sample` in [MI_SYS_init](#).

➤ Related topics

[MI_SYS_Init](#)

2.3.3 MI_SYS_GetVersion

➤ Function

Get the system version number of MI.

➤ Syntax

MI_S32 MI_SYS_GetVersion (MI_U16 u16SocId, [MI_SYS_Version_t](#) *pstVersion);

➤ Parameters

Parameter Name	Description	Input/Output
u16SocId	Specify the target Soc for this operation	Input
pstVersion	System version number returns data structure pointer	Output

➤ Return

- 0 Successful.

- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

N/A

➤ Sample

Refer to the `Sample` in [MI_SYS_init](#).

➤ Related topics

N/A

2.3.4 MI_SYS_GetCurPts

➤ Function

Get the current timestamp of the MI system.

➤ Syntax

MI_S32 MI_SYS_GetCurPts (MI_U16 u16SocId, MI_U64 *pu64Pts);

➤ Parameters

Parameter Name	Description	Input/Output
u16SocId	Specify the target Soc for this operation	Input
pu64Pts	The system's current timestamp returns address	Output

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

N/A

➤ Sample

Refer to the `Sample` in [MI_SYS_init](#).

➤ Related topics

N/A

2.3.5 MI_SYS_InitPtsBase

➤ Function

Initializing MI System Baseline Timestamp.

➤ Syntax

MI_S32 MI_SYS_InitPtsBase (MI_U16 u16SocId, MI_U64 u64PtsBase);

➤ Parameters

Parameter Name	Description	Input/Output
u16SocId	Specify the target Soc for this operation	Input
u64PtsBase	Set system timestamp baseline	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

N/A

➤ Sample

Refer to the `Sample` in [MI_SYS_init](#).

➤ Related topics

N/A

2.3.6 MI_SYS_SyncPts

➤ Function

Synchronized MI system timestamp.

➤ Syntax

MI_S32 MI_SYS_SyncPts (MI_U16 u16SocId, MI_U64 u64Pts);

➤ Parameters

Parameter Name	Description	Input/Output
u16SocId	Specify the target Soc for this operation	Input
u64Pts	Fine-tuned system timestamp baseline	Input

- Return
 - 0 Successful.
 - !0 Failed, please refer to the [Error Code](#).

- Dependency
 - Head file: mi_sys_datatype.h, mi_sys.h
 - Library files: libmi_sys.a / libmi_sys.so

※ Note

N/A

- Sample

Refer to the `Sample` in [MI_SYS_init](#).

- Related topics

N/A

2.3.7 MI_SYS_SetReg

- Function

Set the value of the register.

- Syntax

MI_S32 MI_SYS_SetReg (MI_U16 u16SocId, MI_U32 u32RegAddr, MI_U16 u16Value, MI_U16 u16Mask);

- Parameters

Parameter Name	Description	Input/Output
u16SocId	Specify the target Soc for this operation	Input
u32RegAddr	Register Bus Address	Input
u16Value	16bit register value to be written	Input
u16Mask	This time the Mask ingests the bar for the register value	Input

- Return
 - 0 Successful.
 - !0 Failed, please refer to the [Error Code](#).

- Dependency
 - Head file: mi_sys_datatype.h, mi_sys.h

- Library files: libmi_sys.a / libmi_sys.so

※ Note

N/A

➤ Sample

N/A

➤ Related topics

N/A

2.3.8 MI_SYS_GetReg

➤ Function

Get the value of the register, debug with.

➤ Syntax

MI_S32 MI_SYS_GetReg (MI_U16 u16SocId, MI_U32 u32RegAddr, MI_U16 *pu16Value);

➤ Parameters

Parameter Name	Description	Input/Output
u16SocId	Specify the target Soc for this operation	Input
u32RegAddr	Register Bus Address	Input
pu16Value	To read back to 16bit register value return address	Output

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

N/A

➤ Sample

N/A

➤ Related topics

N/A

2.3.9 MI_SYS_ReadUuid

➤ Function

Get Chip's Unique ID.

➤ Syntax

```
MI_S32 MI_SYS_ReadUuid (MI_U16 u16SocId, MI_U64 *u64Uuid);
```

➤ Parameters

Parameter Name	Description	Input/Output
u16SocId	Specify the target Soc for this operation	Input
u64Uuid	Get a pointer to the chip unique ID value	Output

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

N/A

➤ Sample

Refer to the `Sample` in [MI_SYS_init](#).

2.3.10 MI_SYS_EnableChnOutputPortLowLatency

➤ Function

Enable or disable output port low latency.

➤ Syntax

```
MI_S32 MI_SYS_EnableChnOutputPortLowLatency(MI_U16 u16SocId, MI_SYS_ChnPort_t
*pstChnPort, MI_BOOL bEnable , MI_U32 u32Param);
```

➤ Parameters

Parameter Name	Parameter meaning	Input/Output
u16SocId	Specify the target Soc for this operation	Input
pstChnPort	A pointer to the output port of the module channel	Input
bEnable	TRUE: enable; FALSE: disable, in default.	Input

Parameter Name	Parameter meaning	Input/Output
u32Param	Low Latency parameter, which is determined by specific modules. It is used to configure the Line Count. The buffer of frame is sent to user or backward once the count of lines is written, instead of waiting for the whole frame done.	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

When bEnable is true, u32Param must be greater than 0; otherwise the setting is invalid.

➤ Sample

MI_SYS_EnableChnOutputPortLowLatency Sample

```
MI_SYS_ChnPort_t stChnPort;
MI_U16 u16SocId = ST_DEFAULT_SOC_ID;
stChnPort.eModId = E_MI_MODULE_ID_ISP;
stChnPort.u32DevId = 0;
stChnPort.u32ChnId = 0;
stChnPort.u32PortId = 0;
MI_SYS_EnableChnOutputPortLowLatency(u16SocId, &stChnPort, TRUE, 100);

MI_SYS_EnableChnOutputPortLowLatency(u16SocId, &stChnPort, FALSE, 0);
```

2.4. Data flow class API

2.4.1 MI_SYS_BindChnPort

➤ Function

Binding of the data source Output port to the data receiver Input port.

➤ Syntax

```
MI_S32 MI_SYS_BindChnPort(MI_U16 u16SocId, MI_SYS_ChnPort_t *pstSrcChnPort,
MI_SYS_ChnPort_t *pstDstChnPort, MI_U32 u32SrcFrmrate, MI_U32 u32DstFrmrate);
```

➤ Parameters

Parameter Name	Parameter meaning	Input/Output
u16SocId	Specify the target Soc for this operation	Input

pstSrcChnPort	Source port configuration information data structure pointer	Input
pstDstChnPort	Target port configuration information data structure pointer	Input
u32SrcFrmrate	Frame rate of source port configuration	Input
u32DstFrmrate	Frame rate of target port configuration	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

- The source port must be a channel Output port.
- The destination port must be a channel Input port.
- The source and destination ports must not have been bound before.
- MI_SYS_BindChnPort can be called multiple times when pstSrcChnPort, pstDstChnPort and eBindType are the same to reset the u32SrcFrmrate and u32DstFrmrate that need to be modified.
- This interface only supports binding modules in E_MI_SYS_BIND_TYPE_FRAME_BASE mode, not recommended in version 2.0 or above, please use MI_SYS_BindChnPort2 instead.

➤ Sample

MI_SYS_BindChnPort Sample

```
MI_SYS_ChnPort_t stSrcChnPort;
MI_SYS_ChnPort_t stDstChnPort;
MI_U32 u32SrcFrmrate;
MI_U32 u32DstFrmrate;

MI_U16 u16SocId = ST_DEFAULT_SOC_ID;
stSrcChnPort.eModId = E_MI_MODULE_ID_ISP;
stSrcChnPort.u32DevId = 0;
stSrcChnPort.u32ChnId = 0;
stSrcChnPort.u32PortId = 0;
stDstChnPort.eModId = E_MI_MODULE_ID_VENC;
stDstChnPort.u32DevId = 0;
stDstChnPort.u32ChnId = 0;
stDstChnPort.u32PortId = 0;
u32SrcFrmrate = 30;
u32DstFrmrate = 30;
MI_SYS_BindChnPort(u16SocId, &stSrcChnPort, &stDstChnPort, u32SrcFrmrate, u32DstFrmrate);
```

➤ Related topics

[MI_SYS_BindChnPort2](#), [MI_SYS_UnBind_ChnPort](#).
[Bind and Frame Rate Control Scenarios](#)

2.4.2 MI_SYS_BindChnPort2

➤ Function

Binding of the data source Output port to the data receiver Input port requires additional operating mode.

➤ Syntax

```
MI_S32 MI_SYS_BindChnPort2(MI_U16 u16SocId, MI\_SYS\_ChnPort\_t *pstSrcChnPort,
MI\_SYS\_ChnPort\_t *pstDstChnPort, MI_U32 u32SrcFrmrate, MI_U32 u32DstFrmrate,
MI\_SYS\_BindType\_e eBindType, MI_U32 u32BindParam);
```

➤ Parameters

Parameter Name	Parameter meaning	Input/Output
u16SocId	Specify the target Soc for this operation	Input
pstSrcChnPort	The source port configures the information data structure pointer.	Input
pstDstChnPort	The destination port configures the information data structure pointer.	Input
u32SrcFrmrate	Frame rate of source port configuration	Input
u32DstFrmrate	Frame rate of target port configuration	Input
eBindType	The operating mode of the source port connected to the destination port, refer to the MI_SYS_BindType_e	Input
u32BindParam	Additional parameters to be brought in by different working modes.	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

- The source port must be a channel Output port.
- The destination port must be a channel Input port.

- The same source port can be bound to multiple destination ports.
- The same destination port can only be bound to one source port and it must not have been bound before.
- MI_SYS_BindChnPort2 can be called multiple times when pstSrcChnPort, pstDstChnPort and eBindType are the same to reset u32SrcFrmrate and u32DstFrmrate that need to be modified.
- Older versions of MI SYS do not provide this interface if not found, do not need to be set.
- The usage scenarios of eBindType and u32BindParam are as follows.

eBindType	Applicable Scenario
E_MI_SYS_BIND_TYPE_SW_LOW_LATENCY	u32BindParam represents low latency value, unit ms
E_MI_SYS_BIND_TYPE_HW_RING	In the chip before Maruko, u32BindParam represents ring buffer depth, currently only vpe and venc (h264/h265) support this mode, only one way. In Maruko and later chips, u32BindParam is not used yet.
E_MI_SYS_BIND_TYPE_REALTIME	u32 Bind Param unused. Used scene: jpe imi, only support one channel; When vif->isp, support one channel; when isp->scl, support multiple channels
E_MI_SYS_BIND_TYPE_FRAME_BASE	u32BindParam unused, default is to walk this frame mode

➤ Sample

MI_SYS_BindChnPort2 Sample

```

MI_SYS_ChnPort_t stSrcChnPort;
MI_SYS_ChnPort_t stDstChnPort;
MI_U32 u32SrcFrmrate;
MI_U32 u32DstFrmrate;
MI_SYS_BindType_e eBindType;
MI_U32 u32BindParam;

MI_U16 u16SocId = ST_DEFAULT_SOC_ID;
// a. When the type of connection between ISP and venc is E_MI_SYS_BIND_TYPE_FRAME_BASE, the code
is as follows:
stSrcChnPort.eModId = E_MI_MODULE_ID_ISP;
stSrcChnPort.u32DevId = 0;
stSrcChnPort.u32ChnId = 0;
stSrcChnPort.u32PortId = 0;
stDstChnPort.eModId = E_MI_MODULE_ID_VENC;
stDstChnPort.u32DevId = 0;
stDstChnPort.u32ChnId = 0;
stDstChnPort.u32PortId = 0;
u32SrcFrmrate = 30;

```

```

u32DstFrmrate = 30;
eBindType = E_MI_SYS_BIND_TYPE_FRAME_BASE;
u32BindParam = 0;
    STCHECKRESULT(MI_SYS_BindChnPort2(u16SocId,
&stSrcChnPort, &stDstChnPort, u32SrcFrmrate, u32DstFrmrate, eBindType, u32BindParam));

    // b. When the type of connection between ISP and venc is E_MI_SYS_BIND_TYPE_REALTIME, the code
is as follows:
    stSrcChnPort.eModId = E_MI_MODULE_ID_ISP;
    stSrcChnPort.u32DevId = 0;
    stSrcChnPort.u32ChnId = 0;
    stSrcChnPort.u32PortId = 0;
    stDstChnPort.eModId = E_MI_MODULE_ID_VENC;
    stDstChnPort.u32DevId = 1;
    stDstChnPort.u32ChnId = 0;
    stDstChnPort.u32PortId = 0;
    u32SrcFrmrate = 30;
    u32DstFrmrate = 30;
    eBindType = E_MI_SYS_BIND_TYPE_REALTIME;
    u32BindParam = 0;
    STCHECKRESULT(MI_SYS_BindChnPort2(u16SocId, &stSrcChnPort, &stDstChnPort, u32SrcFrmrate,
u32DstFrmrate, eBindType, u32BindParam));

    // c. When the type of connection between ISP and venc is E_MI_SYS_BIND_TYPE_HW_RING, the code
is as follows:
    tSrcChnPort.eModId = E_MI_MODULE_ID_ISP;
    stSrcChnPort.u32DevId = 0;
    stSrcChnPort.u32ChnId = 0;
    stSrcChnPort.u32PortId = 0;
    stDstChnPort.eModId = E_MI_MODULE_ID_VENC;
    stDstChnPort.u32DevId = 0;
    stDstChnPort.u32ChnId = 0;
    stDstChnPort.u32PortId = 0;
    u32SrcFrmrate = 30;
    u32DstFrmrate = 30;
    eBindType = E_MI_SYS_BIND_TYPE_HW_RING;
    u32BindParam = 1080; // Suppose ISP output resolution is 1920*1080, and ring buffer depth is 1080
    STCHECKRESULT(MI_SYS_BindChnPort2(u16SocId, &stSrcChnPort, &stDstChnPort, u32SrcFrmrate,
u32DstFrmrate, eBindType, u32BindParam));

```

➤ Related topics

[MI_SYS_BindChnPort2](#)

[MI_SYS_UnBind_ChnPort.](#)

[Bind and Frame Rate Control Scenarios](#)

2.4.3 MI_SYS_UnBind_ChnPort

➤ Function

De-binding between the data source Output port to the data receiver Input port.

➤ Syntax

```
MI_S32 MI_SYS_UnBindChnPort(MI_U16 u16SocId,  
MI\_SYS\_ChnPort\_t MI\_SYS\_ChnPort\_t *pstSrcChnPort,  
MI\_SYS\_ChnPort\_t MI\_SYS\_ChnPort\_t *pstDstChnPort);
```

➤ Parameters

Parameter Name	Description	Input/Output
u16SocId	Specify the target Soc for this operation	Input
pstSrcChnPort	The source port configures the information data structure pointer.	Input
pstDstChnPort	The destination port configures the information data structure pointer.	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

- The source port must be a channel Output port.
- The destination port must be a channel Input port.
- The source and destination ports must have been bound before

➤ Sample

N/A

➤ Related topics

[MI_SYS_Bind_ChnPort](#)
[MI_SYS_BindChnPort2](#).

2.4.4 MI_SYS_GetBindbyDest

➤ Function

Query the corresponding source Output port for the data recipient Input port.

➤ Syntax

```
MI_S32 MI_SYS_GetBindbyDest (MI_U16 u16SocId,
MI_SYS_ChNPort_t MI_SYS_ChNPort_t *pstDstChNPort,
MI_SYS_ChNPort_t MI_SYS_ChNPort_t *pstSrcChNPort, MI_SYS_BindType_e
*peBindType);
```

➤ Parameters

Parameter Name	Description	Input/Output
u16SocId	Specify the target Soc for this operation	Input
pstDstChNPort	The destination port configures the information data structure pointer.	Input
pstSrcChNPort	The source port configures the information data structure pointer.	Output
peBindType	Binding mode	Output

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

- The destination port must be a channel Input port.
- The target port must have been bound before

➤ Sample

N/A

➤ Related topics

N/A

2.4.5 MI_SYS_ChNInputPortGetBuf

➤ Function

The buf object corresponding to the allocation channel input port.

➤ Syntax

```
MI_S32 MI_SYS_ChNInputPortGetBuf (MI_SYS_ChNPort_t MI_SYS_ChNPort_t
*pstChNPort, MI_SYS_BufConf_t *pstBufConf, MI_SYS_BufInfo_t MI_SYS_BufInfo_t
*pstBufInfo, MI_SYS_BUF_HANDLE *phHandle , MI_S32 s32TimeOutMs);
```

➤ Parameters

Parameter Name	Description	Input/Output
pstChnPort	A pointer to the input port of the module channel	Input
pstBufConf	Memory configuration information to be allocated	Input
pstPortBuf	Return buf pointer	Output
phHandle	Get the handle of input Port Buf	Output
s32TimeOutMs	Milliseconds waiting for timeout, >=0 is the specific time; <0 is 20ms, which is the default value	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

N/A

➤ Sample

MI_SYS_ChnInputPortGetBuf Call Sample

```

MI_SYS_ChnPort_t stIspChnInput;
MI_SYS_BUF_HANDLE hHandle = 0;
MI_SYS_BufConf_t stBufConf;
MI_SYS_BufInfo_t stBufInfo;
struct timeval stTv;
MI_U16 u16Width = 1920, u16Height = 1080;
FILE *fp = NULL;

memset(&stIspChnInput, 0x0, sizeof(MI_SYS_ChnPort_t));
memset(&stBufConf, 0x0, sizeof(MI_SYS_BufConf_t));
memset(&stBufInfo, 0x0, sizeof(MI_SYS_BufInfo_t));

stIspChnInput.eModId = E_MI_MODULE_ID_Isp;
stIspChnInput.u32DevId = 0;
stIspChnInput.u32ChnId = 0;
stIspChnInput.u32PortId = 0;

fp = fopen("/mnt/ispport0_1920x1080_pixel0_737.raw", "rb");
if(fp == NULL)
{
    printf("file %s open fail\n", "/mnt/vpeport0_1920x1080_pixel0_737.raw");
    return 0;
}

```

```
while(1)
{
    stBufConf.eBufType = E_MI_SYS_BUFDATA_FRAME;
    gettimeofday(&stTv, NULL);
    stBufConf.u64TargetPts = stTv.tv_sec*1000000 + stTv.tv_usec;
    stBufConf.stFrameCfg.eFormat = E_MI_SYS_PIXEL_FRAME_YUV422_YUYV;
    stBufConf.stFrameCfg.eFrameScanMode = E_MI_SYS_FRAME_SCAN_MODE_PROGRESSIVE;
    stBufConf.stFrameCfg.u16Width = u16Width;
    stBufConf.stFrameCfg.u16Height = u16Height;

    if(MI_SUCCESS == MI_SYS_ChnInputPortGetBuf(&stIspChnInput,&stBufConf,&stBufInfo,&hHandle,0))
    {
        if(fread(stBufInfo.stFrameData.pVirAddr[0], u16Width*u16Height*2, 1, fp) <= 0)
        {
            fseek(fp, 0, SEEK_SET);
        }

        MI_SYS_ChnInputPortPutBuf(hHandle,&stBufInfo, FALSE);
    }
}
```

➤ Related topics

[MI_SYS_ChnInputPortPutBuf](#)

[MI_SYS_ChnPortInjectBuf](#)

2.4.6 MI_SYS_ChnInputPortPutBuf

➤ Function

Add the buf object corresponding to the channel input port to the pending queue.

➤ Syntax

```
MI_S32 MI_SYS_ChnInputPortPutBuf (MI_SYS_BUF_HANDLE hHandle
, MI\_SYS\_BufInfo\_t MI_SYS_BufInfo_t *pstBufInfo, MI_BOOL bDropBuf);
```

➤ Parameters

Parameter Name	Description	Input/Output
hHandle	Current buf's Handle	Input
pstBufInfo	Buf pointer to be submitted	Input
bDropBuf	Whether to discard buf directly without submitting to the pending queue	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

N/A

➤ Sample

Refer to the `sample` in [MI_SYS_ChnlInputPortGetBuf](#).

➤ Related topics

[MI_SYS_ChnlInputPortGetBuf](#)

[MI_SYS_ChnlInputPortPutBuf](#)

[MI_SYS_ChnlPortInjectBuf](#)

2.4.7 MI_SYS_ChnlOutputPortGetBuf

➤ Function

The buf object corresponding to the allocation channel input port.

➤ Syntax

```
MI_S32 MI_SYS_ChnlOutputPortGetBuf (MI_SYS_ChnlPort_t MI_SYS_ChnlPort_t
*pstChnlPort, MI_SYS_BufInfo_t MI_SYS_BufInfo_t *pstBufInfo ,
MI_SYS_BUF_HANDLE *phHandle);
```

➤ Parameters

Parameter Name	Description	Input/Output
pstChnlPort	A pointer to the input port of the module channel	Input
pstBufInfo	Return buf pointer	Output
phHandle	Get the handle of outputPort Buf	Output

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

N/A

➤ Sample

MI_SYS_ChnOutputPortGetBuf Sample

```
MI_SYS_ChnPort_t stChnPort;
MI_SYS_BufInfo_t stBufInfo;
MI_SYS_BUF_HANDLE stBufHandle;
MI_S32 s32Ret = MI_SUCCESS;
MI_S32 s32Fd = 0;
fd_set read_fds;
struct timeval TimeoutVal;
char szFileName[128];
int fd = 0;
MI_U32 u32GetFramesCount = 0;
MI_BOOL _bWriteFile = TRUE;

MI_U16 u16SocId = ST_DEFAULT_SOC_ID;

stChnPort.eModId = E_MI_MODULE_ID_SCL;
stChnPort.u32DevId = 0;
stChnPort.u32ChnId = SCL_CHN_FOR_VDF;
stChnPort.u32PortId = 0;

s32Ret = MI_SYS_GetFd(&stChnPort, &s32Fd);
if(MI_SUCCESS != s32Ret)
{
    ST_ERR("MI_SYS_GetFd 0, error, %X\n", s32Ret);
    return NULL;
}
s32Ret = MI_SYS_SetChnOutputPortDepth(u16SocId,&stChnPort, 2, 3);
if (MI_SUCCESS != s32Ret)
{
    ST_ERR("MI_SYS_SetChnOutputPortDepth err:%x, chn:%d,port:%d\n", s32Ret,
        stChnPort.u32ChnId, stChnPort.u32PortId);
    return NULL;
}

s32Ret = MI_SYS_SetChnOutputPortUserFrc(&stChnPort, 15);
if (MI_SUCCESS != s32Ret)
{
    ST_ERR("MI_SYS_SetChnOutputPortUserFrc err:%x, chn:%d,port:%d\n", s32Ret,
        stChnPort.u32ChnId, stChnPort.u32PortId);
    return NULL;
}
}
```

```

sprintf(szFileName, "scl%d.es", stChnPort.u32ChnId);
printf("start to record %s\n", szFileName);
fd = open(szFileName, O_RDWR | O_CREAT | O_TRUNC, S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH);
if (fd < 0)
{
    ST_ERR("create %s fail\n", szFileName);
}

while (1)
{
    FD_ZERO(&read_fds);
    FD_SET(s32Fd, &read_fds);

    TimeoutVal.tv_sec = 1;
    TimeoutVal.tv_usec = 0;

    s32Ret = select(s32Fd + 1, &read_fds, NULL, NULL, &TimeoutVal);

    if(s32Ret < 0)
    {
        ST_ERR("select failed!\n");
        // usleep(10 * 1000);
        continue;
    }
    else if(s32Ret == 0)
    {
        ST_ERR("get scl frame time out\n");
        //usleep(10 * 1000);
        continue;
    }
    else
    {
        if(FD_ISSET(s32Fd, &read_fds))
        {
            s32Ret = MI_SYS_ChnOutputPortGetBuf(&stChnPort, &stBufInfo, &stBufHandle);

            if(MI_SUCCESS != s32Ret)
            {
                //ST_ERR("MI_SYS_ChnOutputPortGetBuf err, %x\n", s32Ret);
                continue;
            }

            // save one Frame YUV data
            if (fd > 0)
            {

```

```

        if(!_bWriteFile)
        {
            write(fd, stBufInfo.stFrameData.pVirAddr[0], stBufInfo.stFrameData.u16Height * stBufInfo.stFrameData.u32Stride[0] +
                stBufInfo.stFrameData.u16Height * stBufInfo.stFrameData.u32Stride[1] / 2);
        }
    }

    ++u32GetFramesCount;
    printf("channelId[%u] u32GetFramesCount[%u]\n", stChnPort.u32ChnId, u32GetFramesCount);
};

    MI_SYS_ChnOutputPortPutBuf(stBufHandle);
}
}
}

if (fd > 0)
{
    close(fd);
    fd = -1;
}

MI_SYS_SetChnOutputPortDepth(&stChnPort, 0, 3);
printf("exit record\n");
return NULL;

```

➤ Related topics

[MI_SYS_ChnOutputPortPutBuf](#) / [MI_SYS_ChnPortInjectBuf](#)

2.4.8 MI_SYS_ChnOutputPortPutBuf

➤ Function

Release channel output port corresponding to buf object.

➤ Syntax

```
MI_S32 MI_SYS_ChnOutputPortPutBuf (MI_SYS_BUF_HANDLE hBufHandle);
```

➤ Parameters

Parameter Name	Description	Input/Output
hBufHandle	Handle for buf to be submitted	Input

- Return
 - 0 Successful.
 - !0 Failed, please refer to the [Error Code](#).

- Dependency
 - Head file: mi_sys_datatype.h, mi_sys.h
 - Library files: libmi_sys.a / libmi_sys.so

※ Note
N/A

- Sample
Refer to the sample in [MI_SYS_ChnOutputPortGetBuf](#).

- Related topics
[MI_SYS_ChnOutputPortGetBuf](#)
[MI_SYS_ChnPortInjectBuf](#)

2.4.9 MI_SYS_ChnInputPortGetBufPa

- Function
The buf object corresponding to the allocation channel input port, which only returns MIU physical address.

- Syntax
MI_S32 MI_SYS_ChnInputPortGetBufPa ([MI_SYS_ChnPort_t](#) MI_SYS_ChnPort_t
*pstChnPort, [MI_SYS_BufConf_t](#) *pstBufConf, [MI_SYS_BufInfo_t](#) MI_SYS_BufInfo_t
*pstBufInfo, MI_SYS_BUF_HANDLE *phHandle , MI_S32 s32TimeOutMs);

- Parameters

Parameter Name	Description	Input/Output
pstChnPort	A pointer to the input port of the module channel	Input
pstBufConf	Configuration information of the memory to be allocated	Input
pstPortBuf	Return buf pointer	Output
phHandle	Get the handle of input Port Buf	Output
s32TimeOutMs	Milliseconds waiting for timeout, >=0 is the specific time; <0 is 20ms, which is the default value	Input

- Return
 - 0 Successful.
 - !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

The difference with MI_SYS_ChnlInputPortGetBuf: Only phyAddr can be used in pstBufInfo obtained by MI_SYS_ChnlInputPortGetBufPa, while phyAddr and pVirAddr in pstBufInfo obtained by MI_SYS_ChnlInputPortGetBuf can be used.

➤ Sample

MI_SYS_ChnlInputPortGetBufPa Call Sample

```
MI_SYS_ChnlPort_t stIspChnlInput;
MI_SYS_BUF_HANDLE hHandle = 0;
MI_SYS_BufConf_t stBufConf;
MI_SYS_BufInfo_t stBufInfo;
struct timeval stTv;
MI_U16 u16Width = 1920, u16Height = 1080;
FILE *fp = NULL;
void *pVirAddr = NULL;

MI_U16 u16SocId = ST_DEFAULT_SOC_ID;

memset(&stIspChnlInput, 0x0, sizeof(MI_SYS_ChnlPort_t));
memset(&stBufConf, 0x0, sizeof(MI_SYS_BufConf_t));
memset(&stBufInfo, 0x0, sizeof(MI_SYS_BufInfo_t));

stVpeChnlInput.eModId = E_MI_MODULE_ID_ISP;
stVpeChnlInput.u32DevId = 0;
stVpeChnlInput.u32ChnlId = 0;
stVpeChnlInput.u32PortId = 0;

fp = fopen("/mnt/ispport0_1920x1080_pixel0_737.raw", "rb");
if (fp == NULL)
{
    printf("file %s open fail\n", "/mnt/ispport0_1920x1080_pixel0_737.raw");
    return 0;
}

while (1)
{
    stBufConf.eBufType = E_MI_SYS_BUFDATA_FRAME;
    gettimeofday(&stTv, NULL);
    stBufConf.u64TargetPts = stTv.tv_sec * 1000000 + stTv.tv_usec;
    stBufConf.stFrameCfg.eFormat = E_MI_SYS_PIXEL_FRAME_YUV422_YUYV;
    stBufConf.stFrameCfg.eFrameScanMode = E_MI_SYS_FRAME_SCAN_MODE_PROGRESSIVE;
    stBufConf.stFrameCfg.u16Width = u16Width;
```

```

stBufConf.stFrameCfg.u16Height = u16Height;

if (MI_SUCCESS == MI_SYS_ChnInputPortGetBufPa(&stIspChnInput, &stBufConf, &stBufInfo, &hHandle, 0))
{
    pVirAddr = MI_SYS_Mmap(stBufInfo.stFrameData.phyAddr[0], stBufInfo.stFrameData.u32BufSize,
&pVirAddr, TRUE);
    assert(pVirAddr);
    if (fread(pVirAddr, u16Width * u16Height * 2, 1, fp) <= 0)
    {
        fseek(fp, 0, SEEK_SET);
    }
    MI_SYS_Munmap(pVirAddr, stBufInfo.stFrameData.u32BufSize);
    MI_SYS_ChnInputPortPutBufPa(hHandle, &stBufInfo, FALSE);
}
}

```

➤ Related topics

[MI_SYS_ChnInputPortPutBufPa](#)

[MI_SYS_ChnInputPortPutBuf](#)

[MI_SYS_ChnPortInjectBuf](#)

2.4.10 MI_SYS_ChnInputPortPutBufPa

➤ Function

Add the buf object corresponding to the channel input port to the pending queue, which needs to be used in pair with the function MI_SYS_ChnInputPortGetBufPa.

➤ Syntax

MI_S32 MI_SYS_ChnInputPortPutBufPa (MI_SYS_BUF_HANDLE hHandle
, [MI_SYS_BufInfo_t](#) *pstPortBuf, MI_BOOL bDropBuf);

➤ Parameters

Parameter Name	Description	Input/Output
hHandle	Current buf's Idr Handle	Input
pstPortBuf	Buf pointer to be submitted	Input
bDropBuf	Drop direct waiver of modifications to buf	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

The difference with MI_SYS_ChnInputPortPutBuf: Only phyAddr can be used in pstBufInfo submitted by MI_SYS_ChnInputPortPutBufPa, while phyAddr and pVirAddr in pstBufInfo submitted by MI_SYS_ChnInputPortPutBuf can be used.

➤ Sample

Please refer to the `sample` in [MI_SYS_ChnOutputPortGetBufPa](#)

➤ Related topics

[MI_SYS_ChnInputPortGetBufPa](#)

[MI_SYS_ChnInputPortGetBuf](#)

[MI_SYS_ChnInputPortPutBuf](#)

[MI_SYS_ChnPortInjectBuf](#)

2.4.11 MI_SYS_ChnOutputPortGetBufPa

➤ Function

The buf object corresponding to the allocation channel output port, which only returns MIU physical address.

➤ Syntax

```
MI_S32 MI_SYS_ChnOutputPortGetBufPa (MI\_SYS\_ChnPort\_t MI_SYS_ChnPort_t
*pstChnPort, MI\_SYS\_BufInfo\_t MI_SYS_BufInfo_t *pstBufInfo ,
MI_SYS_BUF_HANDLE *phHandle);
```

➤ Parameters

Parameter Name	Description	Input/Output
pstChnPort	A pointer to the input port of the module channel	Input
pstBufInfo	Return buf pointer	Output
phHandle	Get the handle of outputPort Buf	Output

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

The difference with MI_SYS_ChnOutputPortGetBuf: Only phyAddr can be used in pstBufInfo obtained by MI_SYS_ChnOutputPortGetBufPa, while phyAddr and pVirAddr in pstBufInfo obtained by MI_SYS_ChnOutputPortGetBuf can be used.

➤ Sample

MI_SYS_ChnOutputPortGetBufPa Sample

```
MI_SYS_ChnPort_t stChnPort;
MI_SYS_BufInfo_t stBufInfo;
MI_SYS_BUF_HANDLE stBufHandle;
MI_S32 s32Ret = MI_SUCCESS;
MI_S32 s32Fd = 0;
fd_set read_fds;
struct timeval TimeoutVal;
char szFileName[128];
int fd = 0;
MI_U32 u32GetFramesCount = 0;
MI_BOOL _bWriteFile = TRUE;
void *pVirAddr = NULL;

MI_U16 u16SocId = ST_DEFAULT_SOC_ID;

stChnPort.eModId = E_MI_MODULE_ID_SCL;
stChnPort.u32DevId = 0;
stChnPort.u32ChnId = SCL_CHN_FOR_VDF;
stChnPort.u32PortId = 0;

s32Ret = MI_SYS_GetFd(&stChnPort, &s32Fd);
if (MI_SUCCESS != s32Ret)
{
    ST_ERR("MI_SYS_GetFd 0, error, %X\n", s32Ret);
    return NULL;
}
s32Ret = MI_SYS_SetChnOutputPortDepth(u16SocId, &stChnPort, 2, 3);
if (MI_SUCCESS != s32Ret)
{
    ST_ERR("MI_SYS_SetChnOutputPortDepth err:%x, chn:%d,port:%d\n", s32Ret,
        stChnPort.u32ChnId, stChnPort.u32PortId);
    return NULL;
}

sprintf(szFileName, "scl%d.es", stChnPort.u32ChnId);
printf("start to record %s\n", szFileName);
fd = open(szFileName, O_RDWR | O_CREAT | O_TRUNC, S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH);
if (fd < 0)
```

```

{
    ST_ERR("create %s fail\n", szFileName);
}

while (1)
{
    FD_ZERO(&read_fds);
    FD_SET(s32Fd, &read_fds);

    TimeoutVal.tv_sec = 1;
    TimeoutVal.tv_usec = 0;

    s32Ret = select(s32Fd + 1, &read_fds, NULL, NULL, &TimeoutVal);

    if (s32Ret < 0)
    {
        ST_ERR("select failed!\n");
        // usleep(10 * 1000);
        continue;
    }
    else if (s32Ret == 0)
    {
        ST_ERR("get scl frame time out\n");
        //usleep(10 * 1000);
        continue;
    }
    else
    {
        if (FD_ISSET(s32Fd, &read_fds))
        {
            s32Ret = MI_SYS_ChOutputPortGetBuf(&stChnPort, &stBufInfo, &stBufHandle);

            if (MI_SUCCESS != s32Ret)
            {
                //ST_ERR("MI_SYS_ChOutputPortGetBuf err, %x\n", s32Ret);
                continue;
            }

            pVirAddr = MI_SYS_Mmap(stBufInfo.stFrameData.phyAddr[0],
stBufInfo.stFrameData.u32BufSize, &pVirAddr, TRUE);
            assert(pVirAddr);
            // save one Frame YUV data
            if (fd > 0)
            {
                if (_bWriteFile)
                {

```

```

        write(fd, pVirAddr, stBufInfo.stFrameData.u16Height * stBufInfo.stFrameData.u32Stride[0]
+ stBufInfo.stFrameData.u16Height * stBufInfo.stFrameData.u32Stride[1] / 2);
    }
}

++u32GetFramesCount;
printf("channelId[%u] u32GetFramesCount[%u]\n", stChnPort.u32ChnId, u32GetFramesCount);

MI_SYS_Munmap(pVirAddr, stBufInfo.stFrameData.u32BufSize);
MI_SYS_ChOutputPortPutBufPa(stBufHandle);
}
}
}

if (fd > 0)
{
    close(fd);
    fd = -1;
}

MI_SYS_SetChOutputPortDepth(u16SocId, &stChnPort, 0, 3);
printf("exit record\n");
return NULL;

```

➤ Related topics

[MI_SYS_ChOutputPortPutBufPa](#),

[MI_SYS_ChOutputPortPutBuf](#) ,

[MI_SYS_ChPortInjectBuf](#)

2.4.12 MI_SYS_ChOutputPortPutBufPa

➤ Function

Release channel output port corresponding to the buf object, which needs to be used in pair with the function MI_SYS_ChOutputPortGetBufPa.

➤ Syntax

MI_S32 MI_SYS_ChOutputPortPutBufPa (MI_SYS_BUF_HANDLE hBufHandle);

➤ Parameters

Parameter Name	Description	Input/Output
hBufHandle	Handle for buf to be submitted	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

The difference with MI_SYS_ChnOutputPortPutBuf: Only phyAddr can be used in the pstBufInfo corresponding to the handle submitted by MI_SYS_ChnOutputPortPutBufPa, and both phyAddr and pVirAddr in the pstBufInfo corresponding to the handle submitted by MI_SYS_ChnOutputPortPutBuf can be used.

➤ Sample

Please refer to the sample in [MI_SYS_ChnOutputPortGetBufPa](#)

➤ Related topics

[MI_SYS_ChnOutputPortGetBufPa](#)

[MI_SYS_ChnOutputPortGetBuf](#)

[MI_SYS_ChnPortInjectBuf](#)

2.4.13 MI_SYS_SetChnOutputPortDepth

➤ Function

Set the number of system bufs corresponding to the channel output port and the number of bufs that users can get.

➤ Syntax

```
MI_S32 MI_SYS_SetChnOutputPortDepth(MI_U16 u16SocId,
MI\_SYS\_ChnPort\_tMI_SYS_ChnPort_t *pstChnPort, MI_U32 u32UserFrameDepth,
MI_U32 u32BufQueueDepth);
```

➤ Parameters

Parameter Name	Description	Input/Output
u16SocId	Specify the target Soc for this operation	Input
pstChnPort	A pointer to the output port of the module channel	Input
u32UserFrameDepth	Set the maximum number of buf that the output user can get	Input
u32BufQueueDepth	Set the maximum number of buf for this output system	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

- u32UserFrameDepth defaults to 0, u32BufQueueDepth defaults to 4.
- After using OutputPortBuf (MI_SYS_ChnOutputPortGetBuf / MI_SYS_ChnOutputPortPutBuf no longer need to be called), it is recommended to set u32UserFrameDepth to 0, which can reduce the buffers of output buf and save memory.
- u32UserFrameDepth must be less than or equal to u32BufQueueDepth.
- When u32UserFrameDepth is equal to u32BufQueueDepth and the number of user fifo queue buffers is equal to u32BufQueueDepth, the current output port will stop working.
- When the output port is not bound to the later stage, to make the current output port work, u32UserFrameDepth must be set greater than 0.
- If the output port is bound by realtime mode or ring mode, the u32UserFrameDepth of the current output port must be set to 0.
- For other restrictions, please refer to vif/isp/scl/venc/jpd API.

➤ Sample

Please refer to the `sample` in [MI_SYS_ChnOutputPortGetBuf](#)

➤ Related topics

N/A

2.4.14 MI_SYS_ChnPortInjectBuf

➤ Function

Plug the acquired output Port buf into the specified input Port Buf Queue.

➤ Syntax

```
MI_S32 MI_SYS_ChnPortInjectBuf(MI_SYS_BUF_HANDLE hHandle , MI\_SYS\_ChnPort\_t
*pstChnPort);
```

➤ Parameters

Parameter Name	Description	Input/Output
hHandle	Obtained outputPort Buf handle	Input
pstChnPort	A pointer to the input port of the module channel	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

After calling MI_SYS_ChnPortInjectBuf, user can no longer display the release handle.

➤ Sample

Please refer to the `sample` in [MI_SYS_ChnOutputPortGetBuf](#)

➤ Related topics

[MI_SYS_ChnInputPortGetBuf](#)

[MI_SYS_ChnInputPortPutBuf](#)

[MI_SYS_ChnOutputPortGetBuf](#)

[MI_SYS_ChnOutputPortPutBuf](#)

2.4.15 MI_SYS_GetFd

➤ Function

Get the file Description Number for the current output Port waiting-up event.

➤ Syntax

```
MI_S32 MI_SYS_GetFd(MI\_SYS\_ChnPort\_t MI_SYS_ChnPort_t *pstChnPort , MI_S32
*ps32Fd);
```

➤ Parameters

Parameter Name	Description	Input/Output
pstChnPort	Port information structure pointer	Input
ps32Fd	File Description for waiting for event	Output

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

- Need to be used in pairs with [MI_SYS_CloseFd](#).
- It is recommended to use the fd + select to extract the data, so that the MI_SYS only enable thread when the corresponding port has data, the efficiency is more efficient than use while + sleep loop to take data.

➤ Sample

Please refer to the `sample` in [MI_SYS_ChOutputPortGetBuf](#)

➤ Related topics

[MI_SYS_CloseFd](#)

2.4.16 MI_SYS_CloseFd

➤ Function

File Description Number to close the current output Port.

➤ Syntax

```
MI_S32 MI_SYS_CloseFd(MI_S32 s32ChnPortFd);
```

➤ Parameters

Parameter Name	Description	Input/Output
s32ChnPortFd	File Description Members for waiting for event	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

N/A

➤ Sample

Please refer to the `sample` in [MI_SYS_ChOutputPortGetBuf](#)

➤ Related topics

[MI_SYS_GetFd](#)

2.4.17 MI_SYS_DupBuf

➤ Function

Duplicate buf object.

➤ Syntax

```
MI_S32 MI_SYS_DupBuf (MI_SYS_BUF_HANDLE srcBufHandle , MI_SYS_BUF_HANDLE
*pDupTargetBufHandle);
```

➤ Parameters

Parameter Name	Description	Input/Output
srcBufHandle	Source buf object	Input
pDupTargetBufHandle	Return buf object pointer	Output

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

N/A

➤ Sample

MI_SYS_DupBuf Sample

```
int test0()
{
    MI_SYS_ChnnPort_t stChnnPort;
    MI_SYS_BufInfo_t stBufInfo;
    MI_SYS_BUF_HANDLE bufHandle ,DupTargetBufHandle;
    MI_S32 s32Ret;
    stChnnPort.eModId = E_MI_MODULE_ID_ISP;
    stChnnPort.u32DevId = 0;
    stChnnPort.u32ChnId = 0;
    stChnnPort.u32PortId = 0;
    s32Ret = MI_SYS_ChnnOutputPortGetBuf (&stChnnPort,&stBufInfo,&bufHandle);
    if(s32Ret != MI_SUCCESS)
        return 0;
    s32Ret = MI_SYS_DupBuf(bufHandle, &DupTargetBufHandle);
    if(s32Ret != MI_SUCCESS)
    {
        MI_SYS_ChnnOutputPortPutBuf(bufHandle);
        return 0;
    }
}
```

```

}
stChnPort.eModId = E_MI_MODULE_ID_SCL;
stChnPort.u32DevId = 0;
stChnPort.u32ChnId = 0;
stChnPort.u32PortId = 0;
MI_SYS_ChnPortInjectBuf(DupTargetBufHandle , &stChnPort);

*****

    continue to use the stBufInfo
*****

MI_SYS_ChnOutputPortPutBuf(bufHandle);
return 1;
}

int test1()
{
    MI_SYS_ChnPort_t stChnPort;
    MI_SYS_BufInfo_t stBufInfo;
    MI_SYS_BufConf_t stBufConf;
    MI_SYS_BUF_HANDLE bufHandle ,DupTargetBufHandle;
    MI_S32 s32Ret;
    stChnPort.eModId = E_MI_MODULE_ID_SCL;
    stChnPort.u32DevId = 0;
    stChnPort.u32ChnId = 0;
    stChnPort.u32PortId = 0;
    memset(&stBufConf , 0 , sizeof(stBufConf));
    stBufConf.eBufType = E_MI_SYS_BUFDATA_FRAME;
    stBufConf.stFrameCfg.eFormat = E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_422;
    stBufConf.stFrameCfg.u16Height = 1080;
    stBufConf.stFrameCfg.u16Width = 1920;
    stBufConf.stFrameCfg.eFrameScanMode = E_MI_SYS_FRAME_SCAN_MODE_PROGRESSIVE;
    s32Ret = MI_SYS_ChnInputPortGetBuf (&stChnPort,&stBufConf,&stBufInfo,&bufHandle ,0);
    if(s32Ret != MI_SUCCESS)
        return 0;
    *****

    Fill stBufInfo
    *****

    s32Ret = MI_SYS_DupBuf(bufHandle, &DupTargetBufHandle);
    if(s32Ret != MI_SUCCESS)
    {
        MI_SYS_ChnInputPortPutBuf(bufHandle);
        return 0;
    }
    MI_SYS_ChnInputPortPutBuf(DupTargetBufHandle , & stChnPort , FALSE);
    *****

```

```

        continue to use the stBufInfo
        *****

    return 1;
}

```

➤ Related topics

[MI_SYS_ChOutputPortGetBuf](#)

[MI_SYS_ChOutputPortPutBuf](#)

[MI_SYS_ChInputPortGetBuf](#)

[MI_SYS_ChInputPortPutBuf](#)

[MI_SYS_ChPortInjectBuf](#)

2.4.18 MI_SYS_ChInputPortSetUserPicture

➤ Function

According to the set frame rate, the buf will be repeatedly input to the current input port and add to the waiting queue.

➤ Syntax

```
MI_SYS_ChInputPortSetUserPicture(MI_SYS_BUF_HANDLE BufHandle ,
MI_SYS_BufInfo_t *pstBufInfo, MI_SYS_UserPictureInfo_t *pstUserPictureInfo);
```

➤ Parameters

Parameter Name	Description	Input/Output
BufHandle	Current buf's Handle	Input
pstBufInfo	Buf pointer to be submitted	Input
pstUserPictureInfo	Pointer for setting frame rate information to the driver	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

The Input port for enable user picture cannot be bound and needs to be enabled.

➤ Sample

MI_SYS_ChnInputPortSetUserPicture Sample

```

int test0()
{
    MI_SYS_ChnPort_t stChnPort;
    MI_SYS_BufConf_t stBufConf;
    MI_SYS_BufInfo_t stBufInfo;
    MI_SYS_BUF_HANDLE bufHandle;
    MI_SYS_UserPictureInfo_t stUserPictureInfo;
    memset(&stChnPort, 0, sizeof(stChnPort));
    memset(&stBufConf, 0, sizeof(stBufConf));
    memset(&stBufInfo, 0, sizeof(stBufInfo));

    stChnPort.eModId = E_MI_MODULE_ID_ISP;
    stChnPort.u32DevId = 0;
    stChnPort.u32ChnId = 0;
    stChnPort.u32PortId = 0;
    stBufConf.eBufType = E_MI_SYS_BUFDATA_FRAME;
    stBufConf.stFrameCfg.u16Width = 1920;
    stBufConf.stFrameCfg.u16Height = 1080;
    stBufConf.stFrameCfg.eFormat = E_MI_SYS_PIXEL_FRAME_YUV422_YUYV;

    MI_SYS_ChnInputPortGetBuf(&stChnPort, &stBufConf, &stBufInfo, &bufHandle, 100);
    /*****
    Fill the stBufInfo with data
    *****/

    stUserPictureInfo.u32SrcFrc = 30; // 30 frames per second
    MI_SYS_ChnInputPortSetUserPicture(bufHandle, &stBufInfo, &stUserPictureInfo);
    MI_SYS_EnableUserPicture(bufHandle);
    .....
    .....
    .....
    MI_SYS_DisableUserPicture(bufHandle);
    return 0;
}

```

➤ Related topics

[MI_SYS_ChnInputPortGetBuf](#)[MI_SYS_ChnInputPortSetUserPicture](#)[MI_SYS_EnableUserPicture](#)[MI_SYS_DisableUserPicture](#)

2.4.19 MI_SYS_EnableUserPicture

➤ Function

Enable user picture.

➤ Syntax

MI_S32 MI_SYS_EnableUserPicture(MI_SYS_BUF_HANDLE BufHandle);

➤ Parameters

Parameter Name	Description	Input/Output
BufHandle	Handle of bufinfo set to the driver through MI_SYS_ChnInputPortSetUserPicture	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

N/A

➤ Sample

Please refer to the `sample` in MI_SYS_ChnInputPortSetUserPicture

➤ Related topics

[MI_SYS_ChnInputPortGetBuf](#)

[MI_SYS_ChnInputSetUserPicture](#)

[MI_SYS_EnableUserPicture](#)

[MI_SYS_DisableUserPicture](#)

2.4.20 MI_SYS_DisableUserPicture

➤ Function

turn off user picture function and release buf.

➤ Syntax

```
MI_S32 MI_SYS_DisableUserPicture(MI_SYS_BUF_HANDLE BufHandle);
```

➤ Parameters

Parameter Name	Description	Input/Output
BufHandle	Handle of bufinfo set to the driver through MI_SYS_ChnInputPortSetUserPicture	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

N/A

➤ Sample

Please refer to the `sample` in MI_SYS_ChnInputPortSetUserPicture

➤ Related topics

[MI_SYS_ChnInputPortGetBuf](#)

[MI_SYS_ChnInputPortSetUserPicture](#)

[MI_SYS_EnableUserPicture](#)

[MI_SYS_DisableUserPicture](#)

2.4.21 MI_SYS_SetChnOutputPortUserFrc

➤ Function

set output buf framerate.

➤ Syntax

MI_S32 MI_SYS_SetChnOutputPortUserFrc(([MI_SYS_ChnPort_t](#) *pstChnPort, MI_U32 u32Frmrate);

➤ Parameters

Parameter Name	Description	Input/Output
pstChnPort	Port information structure pointer	Input
u32Frmrate	Out port buf framerate	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

N/A

➤ Sample

Please refer to the `sample` in [MI_SYS_ChnOutputPortGetBuf](#)

➤ Related topics

[MI_SYS_ChnInputPortGetBuf](#)

[MI_SYS_ChnInputPortPutBuf](#)

[MI_SYS_ChnOutputPortGetBuf](#)

[MI_SYS_ChnOutputPortPutBuf](#)

2.5. Memory Management Class API

2.5.1 MI_SYS_SetChnMMAConf

➤ Function

Set the MMA pool name of the module device channel Output default allocation memory.

➤ Syntax

```
MI_S32 MI_SYS_SetChnMMAConf (MI_U16 u16SocId, MI\_ModuleId\_e eModId, MI_U32 u32DevId, MI_U32 u32ChnId, MI_U8 *pu8MMAHeapName);
```

➤ Parameters

Parameter Name	Description	Input/Output
u16SocId	Specify the target Soc for this operation	Input
eModId	Module ID to be configured	Input
u32DevId	Device ID to be configured	Input
u32ChnId	Channel number to be configured	Input
pu8MMAHeapName	Set MMA heap name	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

If MI_SYS_SetChnMMAConf setting is not called or the pu8MMAHeapName value is set to NULL, the default mma_heap_name0 is used.

➤ Sample

```
MI_ModuleId_e eVifModeId = E_MI_MODULE_ID_VIF;
MI_VIF_DEV vifDev = 0;
MI_VIF_CHN vifChn = 0;

MI_U16 u16SocId = ST_DEFAULT_SOC_ID;

MI_SYS_SetChnMMAConf(u16SocId, eVifModeId, vifDev, vifChn, "mma_heap_name0");
```

➤ Related topics

N/A

2.5.2 MI_SYS_GetChnMMAConf

➤ Function

Get the MMA pool name of the module device channel Output port's default allocated memory.

➤ Syntax

```
MI_S32 MI_SYS_GetChnMMAConf (u16SocId, MI_ModuleId_e eModId, MI_U32
u32DevId, MI_U32 u32ChnId, void *pu8MMAHeapName, MI_U32 u32Length);
```

➤ Parameters

Parameter Name	Description	Input/Output
u16SocId	Specify the target Soc for this operation	Input
eModId	Module ID to be configured	Input
u32DevId	Device ID to be configured	Input
u32ChnId	Channel number to be configured	Input
pu8MMAHeapName	MMA heap name to be obtained	Output
u32Length	Length of the memory pointed by pu8MMAHeapName, the max 64Byte	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

N/A

➤ Sample

N/A

➤ Related topics

[MI_SYS_SetChnMMAConf](#)

2.5.3 MI_SYS_MMA_Alloc

➤ Function

Request the allocation of memory directly to the MMA Memory Manager.

➤ Syntax

```
MI_S32 MI_SYS_MMA_Alloc(MI_U16 u16SocId, MI_U8 *pstMMAHeapName, MI_U32
u32BlkSize ,MI_PHY *phyAddr);
```

➤ Parameters

Parameter Name	Description	Input/Output
u16SocId	Specify the target Soc for this operation	Input
pstMMAHeapName	Target MMA heapname	Input
u32BlkSize	The size of the block byte to sedated	Input
phyAddr	Physical address of memory block returned	Output

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

alloc from the default heap, when pstMMAHeapName is NULL

➤ Sample

MI_SYS_MMA_Alloc Sample

```
MI_PHY phySrcBufAddr = 0;
void *pVirSrcBufAddr = NULL;
MI_U32 srcBuffSize = 1920 * 1980 * 3 / 2;
MI_U16 u16SocId = ST_DEFAULT_SOC_ID;

srcBuffSize = ALIGN_UP(srcBuffSize, 4096);

ret = MI_SYS_MMA_Alloc(u16SocId, NULL, srcBuffSize, &phySrcBufAddr);
if(ret != MI_SUCCESS)
{
    printf("alloc src buff failed\n");
}
```

```

    return -1;
}

ret = MI_SYS_Mmap(phySrcBufAddr, srcBuffSize, &pVirSrcBufAddr, TRUE);
if(ret != MI_SUCCESS)
{
    MI_SYS_MMA_Free(u16SocId, phySrcBufAddr);
    printf("mmap src buff failed\n");
    return -1;
}

memset(pVirSrcBufAddr, 0, srcBuffSize);
MI_SYS_FlushInvCache(pVirSrcBufAddr,srcBuffSize);
MI_SYS_Munmap(pVirSrcBufAddr, srcBuffSize);
MI_SYS_MMA_Free(u16SocId, phySrcBufAddr);

```

➤ Related topics

[MI_SYS_MMA_Free](#) / [MI_SYS_Mmap](#) / [MI_SYS_FlushCache](#) / [MI_SYS_Munmap](#)

2.5.4 MI_SYS_MMA_Free

➤ Function

Memory allocated before being released directly to mmA Memory Manager.

➤ Syntax

MI_S32 MI_SYS_MMA_Free(MI_U16 u16SocId, MI_U64 phyAddr);

➤ Parameters

Parameter Name	Description	Input/Output
u16SocId	Specify the target Soc for this operation	Input
phyAddr	Physical address of memory to be freed	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

N/A

➤ Sample

Please refer to the `sample` in [MI_SYS_MMA_Alloc](#)

➤ Related topics

[MI_SYS_MMA_Alloc](#) / [MI_SYS_Mmap](#) / [MI_SYS_FlushCache](#) / [MI_SYS_Munmap](#)

2.5.5 MI_SYS_Mmap

➤ Function

Mapping any physical memory to the CPU virtual address space for the current user-state process.

➤ Syntax

```
MI_S32 MI_SYS_Mmap(MI_U64 u64PhyAddr, MI_U32 u32Size, void **ppVirtualAddress,
MI_BOOL bCache);
```

➤ Parameters

Parameter Name	Description	Input/Output
u64PhyAddr	Physical address to be mapped	Input
u32Size	The length of the physical address to be mapped	Input
ppVirtualAddress	Pointer to store the CPU virtual address pointer	Output
bCache	Whether map into cache or un-cache	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

- The physical address is the SStar memory controller address.
- The physical address is recommended to be 4KByte aligned.
- The physical address length is recommended to be 4KByte aligned.
- Physical memory must fall completely outside the memory range managed by MMA or from the memory managed by Linux kernel

➤ Sample

Please refer to the `sample` in [MI_SYS_MMA_Alloc](#)

➤ Related topics

[MI_SYS_MMA_Alloc](#)

[MI_SYS_MMA_Free](#)

[MI_SYS_FlushInvCache](#)

[MI_SYS_Munmap](#)

2.5.6 MI_SYS_FlushInvCache

➤ Function

Flush cache.

➤ Syntax

```
MI_S32 MI_SYS_FlushInvCache(MI_VOID *pVirtualAddress, MI_U32 u32Size);
```

➤ Parameters

Parameter Name	Description	Input/Output
pVirtualAddress	Previously MI_SYS_Mmap returned CPU virtual address	Input
u32Size	The length of the cache to be flush	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

- The physical address to flush cache is recommended to be 4KByte aligned.
- The mapping length to flush cache is recommended to be 4KByte aligned.
- The map memory range to flush cache must be previously obtained through the MI_SYS_Mmap API.
- The mapping memory for flush cache should be MI_SYS_Mmap in the way of cache, nocache way without flush cache.

➤ Sample

Please refer to the `sample` in [MI_SYS_MMA_Alloc](#)

➤ Related topics

[MI_SYS_MMA_Alloc](#)

[MI_SYS_MMA_Free](#)

[MI_SYS_Mmap](#)

[MI_SYS_Munmap](#)

2.5.7 MI_SYS_Munmap

➤ Function

Cancel the mapping of physical memory to the virtual address..

➤ Syntax

```
MI_S32 MI_SYS_Munmap(MI_VOID *pVirtualAddress, MI_U32 u32Size);
```

➤ Parameters

Parameter Name	Description	Input/Output
pVirtualAddress	Previously MI_SYS_Mmap returned CPU virtual address	Input
u32Size	The length of the map to be unmapped	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

- The virtual address to be unmapped is recommended to be 4KByte aligned.
- The mapping length to be unmapped is recommended to be 4KByte aligned.
- The range of mapped memory to be canceled must be previously obtained through the MI_SYS_Mmap API.

➤ Sample

Please refer to the `sample` in [MI_SYS_MMA_Alloc](#)

➤ Related topics

[MI_SYS_MMA_Alloc](#) / [MI_SYS_MMA_Free](#) / [MI_SYS_Mmap](#) / [MI_SYS_FlushCache](#)

2.5.8 MI_SYS_MemsetPa

➤ Function

Fill the entire physical memory with the DMA hardware module.

➤ Syntax

```
MI_S32 MI_SYS_MemsetPa(MI_U16 u16SocId, MI_PHY phyPa, MI_U32 u32Val, MI_U32 u32Lenth);
```

➤ Parameters

Parameter Name	Description	Input/Output
----------------	-------------	--------------

u16SocId	Specify the target Soc for this operation	Input
phyPa	The physical address of the padding	Input
u32Val	Fill value of 4bytes size	Input
u32Lenth	Fill size, in byte	Input

- Return
 - 0 Successful.
 - !0 Failed, please refer to the [Error Code](#).

- Dependency
 - Head file: mi_sys_datatype.h, mi_sys.h
 - Library files: libmi_sys.a / libmi_sys.so

※ Note

N/A

➤ Sample

MI_SYS_MemsetPa Sample

```
MI_PHY phySrcBufAddr = 0;
MI_PHY phyDstBufAddr = 0;
void *pVirSrcBufAddr = NULL;
void *pVirDstBufAddr = NULL;
MI_U32 buffSize = 1920 * 1980 * 3 / 2;
MI_U16 u16SocId = ST_DEFAULT_SOC_ID;

buffSize = ALIGN_UP(buffSize, 4096);

ret = MI_SYS_MMA_Alloc(u16SocId, NULL, buffSize, &phySrcBufAddr);
if(ret != MI_SUCCESS)
{
    printf("alloc src buff failed\n");
    return -1;
}

ret = MI_SYS_MMA_Alloc(u16SocId, NULL, buffSize, &phyDstBufAddr);
if(ret != MI_SUCCESS)
{
    MI_SYS_MMA_Free(u16SocId, phySrcBufAddr);
    printf("alloc dts buff failed\n");
    return -1;
}

MI_SYS_MemsetPa(u16SocId, phySrcBufAddr, 0xffffffff, buffSize);
MI_SYS_MemsetPa(u16SocId, phyDstBufAddr, 0x00, buffSize);
MI_SYS_MemcpyPa(u16SocId, phyDstBufAddr, phySrcBufAddr, buffSize);
MI_SYS_MMA_Free(u16SocId, phySrcBufAddr);
MI_SYS_MMA_Free(u16SocId, phyDstBufAddr);
```

➤ Related topics

[MI_SYS_MemcpyPa](#)

2.5.9 MI_SYS_MemcpyPa

➤ Function

Copy the source memory data to the target memory via the DMA hardware module.

➤ Syntax

```
MI_S32 MI_SYS_MemcpyPa(MI_U6 u16SocId, MI_PHY phyDst, MI_PHY phySrc, MI_U32 u32Lenth);
```

➤ Parameters

Parameter Name	Description	Input/Output
u16SocId	Specify the target Soc for this operation	Input
phyDst	Destination physical address	Input
phySrc	Source physical address	Input
u32Lenth	Copy size, in byte	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

N/A

➤ Sample

Please refer to the `sample` in [MI_SYS_MemsetPa](#)

➤ Related topics

[MI_SYS_MemsetPa](#)

2.5.10 MI_SYS_BufFillPa

➤ Function

Fill some of the physical memory with the DMA hardware module.

➤ Syntax

```
MI_S32 MI_SYS_BufFillPa(MI_U16 u16SocId, MI_SYS_FrameData_t *pstBuf, MI_U32
u32Val, MI_SYS_WindowRect_t *pstRect);
```

➤ Parameters

Parameter Name	Description	Input/Output
u16SocId	Specify the target Soc for this operation	Input
pstBuf	The structure of the filled frame data Description	Input
u32Val	Fill value	Input
pstRect	The extent of the data populated	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

- The pstRect data range is based on the first address of the pstBuf Description as the first address (0, 0), and the width and height are partially filled with the ePixel Format in pstBuf to calculate the size of the memory data moved by each pixel.
- PstBuf in u16Width, u16 Height, phyAddr, u32Stride, ePixelFormat is required, the rest of the value is meaningless.

➤ Sample

MI_SYS_BufFillPa Sample

```
MI_S32 ret = 0;
MI_SYS_WindowRect_t rect;
MI_SYS_FrameData_t stSysFrame;
MI_U16 u16SocId = ST_DEFAULT_SOC_ID;
memcpy(&stSysFrame, &buf->stFrameData, sizeof(MI_SYS_FrameData_t));

if(pRect) {
    rect.u16X = pRect->left;
    rect.u16Y = pRect->top;
    rect.u16Height = pRect->bottom-pRect->top;
    rect.u16Width = pRect->right-pRect->left;
} else {
    rect.u16X = 0;
    rect.u16Y = 0;
    rect.u16Height = stSysFrame.u16Height;
    rect.u16Width = stSysFrame.u16Width;
```

```

}

DBG_INFO("rect %d %d %d %d \n", rect.u16X, rect.u16Y
, rect.u16Width, rect.u16Height);
ret = MI_SYS_BufFillPa(u16SocId, &stSysFrame, u32ColorVal, &rect);

return ret;

```

➤ Related topics

[MI_SYS_BufBlitPa](#)

2.5.11 MI_SYS_BufBlitPa

➤ Function

Copy parts of the source memory data to parts of the target memory through the DMA hardware module.

➤ Syntax

```

MI_S32 MI_SYS_BufBlitPa(MI_U16 u16SocId, MI_SYS_FrameData_t *pstDstBuf,
MI_SYS_WindowRect_t *pstDstRect, MI_SYS_FrameData_t *pstSrcBuf,
MI_SYS_WindowRect_t *pstSrcRect);

```

➤ Parameters

Parameter Name	Description	Input/Output
u16SocId	Specify the target Soc for this operation	Input
pstDstBuf	Target memory physical first address	Input
pstDstRect	The area of the target memory copy	Input
pstSrcBuf	Source memory physical first address	Input
pstSrcRect	Area of the source memory copy	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

- The pstDstRect/pstSrcRect data range is based on the first address of pstDstBuf/pstSrcBuf Buf Description as the first address (0,0), and the width and height are partially populated with ePixelFormat to calculate the size of memory data moved by each pixel.
- PstDdbuf/pstSrcBuf in u16Width, u16 Height, phyAddr, u32Stride, e32Cre, ePixelFormat is required, the rest of the value is meaningless.

- The area portion of the source memory or target memory exceeds its original range and will only copy the data on which it is not exceeded.

➤ Sample

MI_SYS_BufBlitPa Sample

```
MI_S32 ret = MI_SUCCESS;
vdisp_copyinfo_plane_t *plane;
MI_SYS_FrameData_t stSrcFrame, stDstFrame;
MI_SYS_WindowRect_t stSrcRect, stDstRect;
MI_U16 u16SocId = ST_DEFAULT_SOC_ID;

plane = &copyinfo->plane[0];
stSrcFrame.ePixelFormat = E_MI_SYS_PIXEL_FRAME_I8;
stSrcFrame.phyAddr[0] = plane->src_paddr;
stSrcFrame.u16Width = plane->width;
stSrcFrame.u16Height = plane->height;
stSrcFrame.u32Stride[0] = plane->src_stride;

stDstFrame.ePixelFormat = E_MI_SYS_PIXEL_FRAME_I8;
stDstFrame.phyAddr[0] = plane->dst_paddr;
stDstFrame.u16Width = plane->width;
stDstFrame.u16Height = plane->height;
stDstFrame.u32Stride[0] = plane->dst_stride;

stSrcRect.u16X = 0;
stSrcRect.u16Y = 0;
stSrcRect.u16Height = stSrcFrame.u16Height;
stSrcRect.u16Width = stSrcFrame.u16Width;

stDstRect.u16X = 0;
stDstRect.u16Y = 0;
stDstRect.u16Height = stDstFrame.u16Height;
stDstRect.u16Width = stDstFrame.u16Width;

ret = MI_SYS_BufBlitPa(u16SocId, &stDstFrame, &stDstRect, &stSrcFrame, &stSrcRect);

return ret;
```

➤ Related topics

[MI_SYS_BufFillPa](#)

2.5.12 MI_SYS_ConfigPrivateMMAPool

➤ Function

Configure private MMA Heap.

➤ Syntax

```
MI_S32 MI_SYS_ConfigPrivateMMAPool(MI_U16 u16SocId, MI\_SYS\_GlobalPrivPoolConfig\_t
*pstGlobalPrivPoolConf);
```

➤ Parameters

Parameter Name	Description	Input/Output
u16SocId	Specify the target Soc for this operation	Input
pstGlobalPrivPoolConf	Configure private MMA Heap for modules	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

- **Device private MMA Heap and channel private MMA Heap cannot coexist.**
- It is recommended to create a private MMA heap for each module after [MI_SYS_Init](#).
- When pstGlobalPrivPoolConf-bCreate is TRUE, create a private POOL, destroy the private POOL when it is FALSE
- Various eConfigType use scenarios are as follows:

eConfigType	Applicable Scenario
E_MI_SYS_VPE_TO_VENC_PRIVATE_RING_POOL	Set up a private ring heap pool for scl and venc ports bound to E_MI_SYS_BIND_TYPE_HW_RIN mode. Note: Mi_Sys2.0 indicates the settings between vpe and venc ports.
E_MI_SYS_PRE_CHN_PRIVATE_POOL	Set up a private heap pool for the module channel
E_MI_SYS_PRE_DEV_PRIVATE_POOL	Set up a private heap pool for module devices
E_MI_SYS_PER_CHN_PORT_OUTPUT_POOL	Set heap pool for module output port, after which the output buf priority of the port is assigned

➤ Sample

Suppose the scene is: Two streams under IPC, one main Stream H265 maximum resolution 1920 x 1080, and all the way Sub Stream H264 max resolution is 720 x 576.

VENC Creat MMA Heap:

```
//Main Sream:
//Create private MMA heap with size of 38745760 for Channel 1
MI_SYS_GlobalPrivPoolConfig_t stConfig;
MI_U16 u16SocId = ST_DEFAULT_SOC_ID;
memset(&stConfig, 0, sizeof(stConfig));
```

```

stConfig.eConfigType = E_MI_SYS_PER_CHN_PRIVATE_POOL;
stConfig.bCreate = TRUE;
stConfig.uConfig.stPreChnPrivPoolConfig.eModule = E_MI_MODULE_ID_VENC;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Devid = 0;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Channel = 1;
stConfig.uConfig.stPreChnPrivPoolConfig.u32PrivateHeapSize = 38745760;
MI_SYS_ConfigPrivateMMAPool(u16SocId, &stConfig);

//Sub Stream:
//Create private MMA heap with size of 805152 for Channel 2
stConfig.eConfigType = E_MI_SYS_PER_CHN_PRIVATE_POOL;
stConfig.bCreate = TRUE;
stConfig.uConfig.stPreChnPrivPoolConfig.eModule = E_MI_MODULE_ID_VENC;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Devid = 0;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Channel = 2;
stConfig.uConfig.stPreChnPrivPoolConfig.u32PrivateHeapSize = 805152;
MI_SYS_ConfigPrivateMMAPool(u16SocId, &stConfig);

```

VENC Destroyed MMA Heap:

```

//Main Stream:
//Destroyed private MMA heap of Channel 1
stConfig.eConfigType = E_MI_SYS_PER_CHN_PRIVATE_POOL;
stConfig.bCreate = FALSE;
stConfig.uConfig.stPreChnPrivPoolConfig.eModule = E_MI_MODULE_ID_VENC;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Devid = 0;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Channel = 1;
MI_SYS_ConfigPrivateMMAPool(&stConfig);

//Sub Stream:
//Destroyed private MMA heap of Channel 2
stConfig.eConfigType = E_MI_SYS_PER_CHN_PRIVATE_POOL;
stConfig.bCreate = FALSE;
stConfig.uConfig.stPreChnPrivPoolConfig.eModule = E_MI_MODULE_ID_VENC;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Devid = 0;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Channel = 2;
MI_SYS_ConfigPrivateMMAPool(u16SocId, &stConfig);

```

ISP Creat MMA Heap:

```

//Creat private MMA Heap with size of 0x4f9200 for Device 0
stConfig.eConfigType = E_MI_SYS_PER_DEV_PRIVATE_POOL;
stConfig.bCreate = TRUE;
stConfig.uConfig.stPreDevPrivPoolConfig.eModule = E_MI_MODULE_ID_ISP;
stConfig.uConfig.stPreDevPrivPoolConfig.u32Devid = 0;
MI_SYS_ConfigPrivateMMAPool(u16SocId, &stConfig);

```

ISP Destroyed MMA Heap:

```
//Destroyed private MMA Heap of Device 0
stConfig.eConfigType = E_MI_SYS_PER_DEV_PRIVATE_POOL;
stConfig.bCreate = FALSE;
stConfig.uConfig.stPreDevPrivPoolConfig.eModule = E_MI_MODULE_ID_ISP;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Devid = 0;
MI_SYS_ConfigPrivateMMAPool(u16SocId, &stConfig);
```

Creat SCL -> VENC ring MMA Heap:

```
stConfig.eConfigType = E_MI_SYS_VPE_TO_VENC_PRIVATE_RING_POOL;
stConfig.bCreate = TRUE;
stConfig.uConfig.stPreVpe2VencRingPrivPoolConfig.u32VencInputRingPoolStaticSize = 8*1024*1024;
MI_SYS_ConfigPrivateMMAPool(u16SocId, &stConfig);
```

Destroyed SCL -> VENC ring MMA Heap:

```
stConfig.eConfigType = E_MI_SYS_VPE_TO_VENC_PRIVATE_RING_POOL;
stConfig.bCreate = FALSE;
MI_SYS_ConfigPrivateMMAPool(u16SocId, &&stConfig);
```

Different resolutions or enable different function have different sizes to match, detail parameters should be calculated according to the scene and specifications.

➤ Related topics

[MI_SYS_PrivateDevChnHeapAlloc](#)

[MI_SYS_PrivateDevChnHeapFree](#)

2.5.13 MI_SYS_PrivateDevChnHeapAlloc

➤ Function

Request memory from module channel private MMA Pool.

➤ Syntax

```
MI_S32 MI_SYS_PrivateDevChnHeapAlloc(MI_U16 u16SocId, MI_ModuleId_e eModule,
MI_U32 u32Devid, MI_S32 s32ChnId, MI_U8 *pu8BufName, MI_U32 u32blkSize,
MI_PHY *pphyAddr, MI_BOOL bTailAlloc);
```

➤ Parameters

Parameter Name	Description	Input/Output
u16SocId	Specify the target Soc for this operation	Input
eModule	Module ID	Input
u32Devid	The device ID of the module	Input
s32ChnId	Channel ID of the module	Input

Parameter Name	Description	Input/Output
pu8BufName	When applying for the name of private memory, pass null, use the "app-privAlloc" as the default name	Input
u32blkSize	Request the size of private memory	Input
pphyAddr	Physical address of allocated private memory	Output
bTailAlloc	Whether to apply from the tail of the channel private pool	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

When using this interface, make sure that you have applied E_MI_SYS_PRE_CHN_PRIVATE_POOL type of private memory pool using MI_SYS_ConfigPrivateMMAPool first.

➤ Sample

MI_SYS_PrivateDevChnHeapAlloc Sample

```
MI_PHY *pphyAddr = NULL;
MI_SYS_GlobalPrivPoolConfig_t stConfig;
MI_U16 u16SocId = ST_DEFAULT_SOC_ID;

Memset(&stConfig, 0, sizeof(stConfig));
stConfig.eConfigType = E_MI_SYS_PER_CHN_PRIVATE_POOL;
stConfig.bCreate = TRUE;
stConfig.uConfig.stPreChnPrivPoolConfig.eModule = E_MI_MODULE_ID_VENC;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Devid = 0;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Channel = 1;
stConfig.uConfig.stPreChnPrivPoolConfig.u32PrivateHeapSize = 38745760;
MI_SYS_ConfigPrivateMMAPool(u16SocId, &stConfig);

ret = MI_SYS_PrivateDevChnHeapAlloc(u16SocId,
E_MI_MODULE_ID_VENC, 0, 1, NULL, 4096, pphyAddr, FALSE);
if(ret != MI_SUCCESS)
{
    MI_SYS_ConfigPrivateMMAPool(u16SocId, &stConfig);
    printf("alloc buff from chn private heap failed\n");
    return -1;
}
```

```
//do something...
```

```
MI_SYS_PrivateDevChnHeapFree(u16SocId, E_MI_MODULE_ID_VENC, 0, 1, *pphyAddr);  
MI_SYS_ConfigPrivateMMAPool(u16SocId, &stConfig);
```

➤ Related topics

[MI_SYS_ConfigPrivateMMAPool](#)

[MI_SYS_PrivateDevChnHeapFree](#)

2.5.14 MI_SYS_PrivateDevChnHeapFree

➤ Function

Free memory from module channel private MMA Pool.

➤ Syntax

```
MI_S32 MI_SYS_PrivateDevChnHeapFree(MI_U16 u16SocId, MI_ModuleId_e eModule,  
MI_U32 u32Devid, MI_S32 s32ChnId, MI_PHY phyAddr);
```

➤ Parameters

Parameter Name	Description	Input/Output
u16SocId	Specify the target Soc for this operation	Input
eModule	Module ID	Input
u32Devid	The device ID of the module	Input
s32ChnId	Channel ID of the module	Input
phyAddr	Physical address for memory that needs to be freed	Input

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

When using this interface, make sure that you have applied a private memory pool of E_MI_SYS_PRE_CHN_PRIVATE_POOL type using the MI_SYS_ConfigPrivateMMAPool first. Otherwise it will apply memory from the heap mma_heap_name0.

➤ Sample

Please refer to the `sample` in [MI_SYS_PrivateDevChnHeapAlloc](#)

➤ Related topics

[MI_SYS_ConfigPrivateMMAPool](#)

[MI_SYS_PrivateDevChnHeapAlloc](#)

2.5.15 MI_SYS_Va2Pa

➤ Function

Turn CPU virtual address pointer to Physical address.

➤ Syntax

MI_S32 MI_SYS_Va2Pa (void *pVirtualAddress, MI_PHY *pPhyAddr);

➤ Parameters

Parameter Name	Description	Input/Output
pVirtualAddress	CPU virtual address pointer	Input
pPhyAddr	Pointer to store the physical address of the memory block	Output

➤ Return

- 0 Successful.
- !0 Failed, please refer to the [Error Code](#).

➤ Dependency

- Head file: mi_sys_datatype.h, mi_sys.h
- Library files: libmi_sys.a / libmi_sys.so

※ Note

When using the interface, make sure that the input CPU virtual address is valid and points to the address space allocated in MI.

➤ Sample

MI_SYS_Va2Pa Sample

```
MI_PHY phySrcBufAddr = 0;
MI_PHY phyAddrVa2Pa = 0;
void *pVirSrcBufAddr = NULL;
MI_U32 srcBuffSize = 1920 * 1980 * 3 / 2;
MI_U16 u16SocId = ST_DEFAULT_SOC_ID;

srcBuffSize = ALIGN_UP(srcBuffSize, 4096);

ret = MI_SYS_MMA_Alloc(u16SocId, NULL, srcBuffSize, &phySrcBufAddr);
if(ret != MI_SUCCESS)
{
    printf("alloc src buff failed\n");
}
```

```

    return -1;
}

ret = MI_SYS_Mmap(phySrcBufAddr, srcBuffSize, &pVirSrcBufAddr, TRUE);
if(ret != MI_SUCCESS)
{
    MI_SYS_MMA_Free(u16SocId, phySrcBufAddr);
    printf("mmap src buff failed\n");
    return -1;
}

MI_SYS_Va2Pa(pVirSrcBufAddr, &phyAddrVa2Pa);
assert(phySrcBufAddr == phyAddrVa2Pa);
memset(pVirSrcBufAddr, 0, srcBuffSize);
MI_SYS_FlushInvCache(pVirSrcBufAddr, srcBuffSize);
MI_SYS_Munmap(pVirSrcBufAddr, srcBuffSize);
MI_SYS_MMA_Free(u16SocId, phySrcBufAddr);

```

➤ Related topics

[MI_SYS_Mmap](#)

3. Data type

3.1. Description of data structure Description Format

This manual uses information about the five reference domain Description data types, which act as information about the reference domain Description data types, as shown in Table 3-1.

Table 3-4 : Data Structure Description Format Description

Label	Function
Description	Briefly describe the main function of data type.
Define	List the definition statement of the data type
Members	List the data structure of the members and meanings.
Note	List things that should be in Note when using data types.
Related data types and interfaces	List the other data types and interfaces associated with this data type.

3.2. List of data structures

Relevant data types and data structures are defined as follows:

Table 3-5 : Summary of Data Structures

Data Structures	definition
MI_ModuleId_e	Define module ID enumeration type
MI_SYS_PixelFormat_e	Define pixel enumeration type
MI_SYS_CompressMode_e	Define compression enumeration type
MI_SYS_FrameTileMode_e	Define Tile format enumeration type
MI_SYS_FieldType_e	Define Field Enumeration Type

Data Structures	definition
<u>MI_SYS_BufDataType_e</u>	Define module ID enumeration type
<u>MI_SYS_FrameIspInfoType_e</u>	Define types of ISP info enumerations carried by frame data
<u>MI_SYS_ChnPort_t</u>	Define module device channel structure
<u>MI_SYS_MetaData_t</u>	Define the structure for MetaData
<u>MI_SYS_RawData_t</u>	Structure of Define Stream RawData
<u>MI_SYS_WindowRect_t</u>	Structure of Define Window coordinates
<u>MI_SYS_FrameData_t</u>	Structure of Define Stream FrameData
<u>MI_SYS_BufInfo_t</u>	Buf Information Structure
<u>MI_SYS_FrameBufExtraConfig_t</u>	Define Structure sits additionally configured by Stream Frame buffer
<u>MI_SYS_BufFrameConfig_t</u>	Frame buf configuration information structure
<u>MI_SYS_BufRawConfig_t</u>	Raw buf configuration information structure
<u>MI_SYS_MetaDataConfig_t</u>	Meta buf configuration information structure
<u>MI_SYS_BufConf_t</u>	Configure the Buf information structure
<u>MI_SYS_Version_t</u>	Sys version information structure
<u>MI_VB_PoolListConf_t</u>	Structure of DescribeVB Pool list information
<u>MI_SYS_BindType_e</u>	Enumeration type of Pre- and Post-Level Working Mode of Define
<u>MI_SYS_FrameData_PhySignalType</u>	Enumeration type of buffer type to which Describe frame data belongs
<u>MI_SYS_InsidePrivatePoolType_e</u>	Describe the created enumeration type of private MMA POOL type
<u>MI_SYS_PerChnPrivHeapConf_t</u>	Define the structure describing channel private MMA Pool
<u>MI_SYS_PerDevPrivHeapConf_t</u>	Define the structure describing the device's private MMA Pool
<u>MI_SYS_PerVpe2VencRingPoolConff_t</u>	Define the structure describing private Ring MMA Pool between SCL and VENC. Mi_Sys2.0 indicates the settings between vpe and venc ports.
<u>MI_SYS_PerChnPortOutputPool_t</u>	Define the private MMA Pool structure describing output port
<u>MI_SYS_GlobalPrivPoolConfig_t</u>	Define the structure describing private MMA Pool
<u>MI_SYS_FrameIspInfo_t</u>	Define the structure of ISP info in frame data
<u>MI_SYS_FbcData_t</u>	Define data structure for frame buffer compress
<u>MI_SYS_InitParam_t</u>	Define the initialized parameter of SYS device
<u>MI_SYS_BufFrameMultiPlaneConfig_t</u>	MultiPlane Frame buf configuration information structure
<u>MI_SYS_BufFrameMetaConfig_t</u>	Define structure of frame meta data
<u>MI_SYS_FrameDataSubPlane_t</u>	Structure of Define Stream SubPlane FrameData
<u>MI_SYS_FrameDataMultiPlane_t</u>	Structure of Define Stream MultiPlane FrameData
<u>MI_SYS_UserPictureInfo_t</u>	Define the structure describing user picture info

3.2.1 MI_ModuleId_e

➤ Description

Define Module ID enumeration type.

➤ Define

```
typedef enum
{
    E_MI_MODULE_ID_IVE    = 0,
    E_MI_MODULE_ID_VDF    = 1,
    E_MI_MODULE_ID_VENC   = 2,
    E_MI_MODULE_ID_RGN    = 3,
    E_MI_MODULE_ID_AI     = 4,
    E_MI_MODULE_ID_AO     = 5,
    E_MI_MODULE_ID_VIF    = 6,
    E_MI_MODULE_ID_VPE    = 7,
    E_MI_MODULE_ID_VDEC   = 8,
    E_MI_MODULE_ID_SYS    = 9,
    E_MI_MODULE_ID_FB     = 10,
    E_MI_MODULE_ID_HDMI   = 11,
    E_MI_MODULE_ID_DIVP   = 12,
    E_MI_MODULE_ID_GFX    = 13,
    E_MI_MODULE_ID_VDISP  = 14,
    E_MI_MODULE_ID_DISP   = 15,
    E_MI_MODULE_ID_OS     = 16,
    E_MI_MODULE_ID_IAE    = 17,
    E_MI_MODULE_ID_MD     = 18,
    E_MI_MODULE_ID_OD     = 19,
    E_MI_MODULE_ID_SHADOW = 20,
    E_MI_MODULE_ID_WARP   = 21,
    E_MI_MODULE_ID_UAC    = 22,
    E_MI_MODULE_ID_LDC    = 23,
    E_MI_MODULE_ID_SD     = 24,
    E_MI_MODULE_ID_PANEL  = 25,
    E_MI_MODULE_ID_CIPHER = 26,
    E_MI_MODULE_ID_SNR    = 27,
    E_MI_MODULE_ID_WLAN   = 28,
    E_MI_MODULE_ID_IPU    = 29,
    E_MI_MODULE_ID_MIPITX = 30,
    E_MI_MODULE_ID_GYRO   = 31,
    E_MI_MODULE_ID_JPD    = 32,
    E_MI_MODULE_ID_ISP    = 33,
    E_MI_MODULE_ID_SCL    = 34,
```

```

E_MI_MODULE_ID_WBC = 35,
E_MI_MODULE_ID_DSP = 36,
E_MI_MODULE_ID_PCIE = 37,
E_MI_MODULE_ID_DUMMY = 38,

//E_MI_MODULE_ID_SED = 29,
E_MI_MODULE_ID_MAX,
} MI_ModuleId_e;

```

➤ Members

Module ID	Module ID HEX Value	Members Name	Module
0	0x00	E_MI_MODULE_ID_IVE	Module ID for Image Intelligent Operator IVE
1	0x01	E_MI_MODULE_ID_VDF	Module ID of Video Intelligent Algorithm Framework Module VDF
2	0x02	E_MI_MODULE_ID_VENC	Module ID for video coding module VENC
3	0x03	E_MI_MODULE_ID_RGN	MODULE ID FOR OSD OVERLAY AND MASKING MODULE REG
4	0x04	E_MI_MODULE_ID_AI	Module ID for Audio Input Module AI
5	0x05	E_MI_MODULE_ID_AO	Module ID for audio Output Module AO
6	0x06	E_MI_MODULE_ID_VIF	Video Input VIF's Module ID
7	0x07	E_MI_MODULE_ID_VPE	Module ID of the video image processing module VPE
8	0x08	E_MI_MODULE_ID_VDEC	Module ID for video decoding VEDC
9	0x09	E_MI_MODULE_ID_SYS	Module ID of System Module SYS
10	0x0A	E_MI_MODULE_ID_FB	SStar UI shows the module ID of the FrameBuffer Device module
11	0x0B	E_MI_MODULE_ID_HDMI	HMDI Module ID
12	0x0C	E_MI_MODULE_ID_DIVP	Module ID for video DI and post-processing module DIVP
13	0x0D	E_MI_MODULE_ID_GFX	Module ID of 2D Graphics Processing Acceleration Module GFX
14	0x0E	E_MI_MODULE_ID_VDISP	Image Puzzle Module VDISP Module ID
15	0x0F	E_MI_MODULE_ID_DISP	Video display module ID of module DISP
16	0x10	E_MI_MODULE_ID_OS	RTOS System Module ID
17	0x11	E_MI_MODULE_ID_IAE	Module ID for Audio Smart Operator IAE
18	0x12	E_MI_MODULE_ID_MD	Module ID of the motion detection module MD
19	0x13	E_MI_MODULE_ID_OD	Block ingesting module ID for the OD of the detection module
20	0x14	E_MI_MODULE_ID_SHADOW	Module ID for virtual MI framework SHADOW

Module ID	Module ID HEX Value	Members Name	Module
21	0x15	E_MI_MODULE_ID_WARP	Module ID of the image malformation correction algorithm WARP
22	0x16	E_MI_MODULE_ID_UAC	Module ID for USB Aduio Class-specific ALSA devices
23	0x17	E_MI_MODULE_ID_LDC	Module ID of image malformation correction algorithm LDC
24	0x18	E_MI_MODULE_ID_SD	Module ID for image zoom function
25	0x19	E_MI_MODULE_ID_PANEL	The display of panel's module ID
26	0x1A	E_MI_MODULE_ID_CIPHER	Chip encrypts CIPHER's module ID
27	0x1B	E_MI_MODULE_ID_SNR	Sensor module ID
28	0x1C	E_MI_MODULE_ID_WLAN	Module ID for wireless communication network Wi-Fi
29	0x1D	E_MI_MODULE_ID_IPU	Module ID for image recognition processing IPU
30	0x1E	E_MI_MODULE_ID_MIPITX	Send data module IDs using THE MIPI protocol
31	0x1F	E_MI_MODULE_ID_GYRO	Module ID of the gyroscope
32	0x20	E_MI_MODULE_ID_JPD	Module ID processed by JPD
33	0x21	E_MI_MODULE_ID_ISP	Module ID for ISP
34	0x22	E_MI_MODULE_ID_SCL	Module ID for image scaling
35	0x23	E_MI_MODULE_ID_WBC	Module ID for video writeback
36	0X24	E_MI_MODULE_ID_DSP	Module ID for DSP
37	0X25	E_MI_MODULE_ID_PCIE	Module ID for PCIE
38	0X26	E_MI_MODULE_ID_DUMMY	Module ID for internal test function

※ Note

N/A

➤ Related data types and interfaces

N/A

3.2.2 MI_SYS_PixelFormat_e

➤ Description

Define Pixel enumeration type.

➤ Define

```
typedef enum
```

```

{
    E_MI_SYS_PIXEL_FRAME_YUV422_YUYV = 0,
    E_MI_SYS_PIXEL_FRAME_ARGB8888,
    E_MI_SYS_PIXEL_FRAME_ABGR8888,
    E_MI_SYS_PIXEL_FRAME_BGRA8888,

    E_MI_SYS_PIXEL_FRAME_RGB565,
    E_MI_SYS_PIXEL_FRAME_ARGB1555,
    E_MI_SYS_PIXEL_FRAME_ARGB4444,
    E_MI_SYS_PIXEL_FRAME_I2,
    E_MI_SYS_PIXEL_FRAME_I4,
    E_MI_SYS_PIXEL_FRAME_I8,

    E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_422,
    E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_420,
    E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_420_NV21,
    E_MI_SYS_PIXEL_FRAME_YUV_TILE_420,
    E_MI_SYS_PIXEL_FRAME_YUV422_UYVY,
    E_MI_SYS_PIXEL_FRAME_YUV422_YVYU,
    E_MI_SYS_PIXEL_FRAME_YUV422_VYUY,

    E_MI_SYS_PIXEL_FRAME_YUV422_PLANAR,
    E_MI_SYS_PIXEL_FRAME_YUV420_PLANAR,

    E_MI_SYS_PIXEL_FRAME_FBC_420,

    E_MI_SYS_PIXEL_FRAME_RGB_BAYER_BASE,
    E_MI_SYS_PIXEL_FRAME_RGB_BAYER_NUM = E_MI_SYS_PIXEL_FRAME_RGB_BAYER_BASE + E_MI_SYS_D
ATA_PRECISION_MAX * E_MI_SYS_PIXEL_BAYERID_MAX - 1,

    E_MI_SYS_PIXEL_FRAME_RGB888,
    E_MI_SYS_PIXEL_FRAME_BGR888,
    E_MI_SYS_PIXEL_FRAME_GRAY8,
    E_MI_SYS_PIXEL_FRAME_RGB101010,
    E_MI_SYS_PIXEL_FRAME_RGB888_PLANAR,
    E_MI_SYS_PIXEL_FRAME_FORMAT_MAX,
} MI_SYS_PixelFormat_e;

```

➤ Members

Members Name	Description
E_MI_SYS_PIXEL_FRAME_YUV422_YUYV	YUV422_YUYV format
E_MI_SYS_PIXEL_FRAME_ARGB8888	ARGB8888 format
E_MI_SYS_PIXEL_FRAME_ABGR8888	ABGR8888 format
E_MI_SYS_PIXEL_FRAME_BGRA8888	BGRA8888 format
E_MI_SYS_PIXEL_FRAME_RGB565	RGB565 format

Members Name	Description
E_MI_SYS_PIXEL_FRAME_ARGB1555	ARGB1555 format
E_MI_SYS_PIXEL_FRAME_ARGB4444	ARGB4444 format
E_MI_SYS_PIXEL_FRAME_I2	2 bpp format
E_MI_SYS_PIXEL_FRAME_I4	4 bpp format
E_MI_SYS_PIXEL_FRAME_I8	8 bpp format
E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_422	YUV422 semi-planar format
E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_420	YUV420sp nv12 format
E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_420_NV21	YUV420sp nv21 format
E_MI_SYS_PIXEL_FRAME_TILE_420	Internal custom TILE format
E_MI_SYS_PIXEL_FRAME_YUV422_UYVY	YUV422_UYVY format
E_MI_SYS_PIXEL_FRAME_YUV422_YVYU	YUV422_YVYU format
E_MI_SYS_PIXEL_FRAME_YUV422_VYUY	YUV422_VYUY format
E_MI_SYS_PIXEL_FRAME_YUV422_PLANAR	YUV422_PLANAR format
E_MI_SYS_PIXEL_FRAME_YUV420_PLANAR	YUV420_PLANAR format
E_MI_SYS_PIXEL_FRAME_FBC_420	Internal customer format
E_MI_SYS_PIXEL_FRAME_RGB_BAYER_BASE	RGB raw data combined format base num value
E_MI_SYS_PIXEL_FRAME_RGB_BAYER_NUM	RGB raw data combined format max num value
E_MI_SYS_PIXEL_FRAME_RGB888	RGB888 format
E_MI_SYS_PIXEL_FRAME_BGR888	BGR888 format
E_MI_SYS_PIXEL_FRAME_GRAY8	8bit gray format
E_MI_SYS_PIXEL_FRAME_RGB101010	RGB101010 format
E_MI_SYS_PIXEL_FRAME_RGB888_PLANAR	RGB888_PLANER format
E_MI_SYS_PIXEL_FRAME_FORMAT_MAX	Enum max value

※ Note

E_MI_SYS_PIXEL_FRAME_RGB_BAYER_BASE,
E_MI_SYS_PIXEL_FRAME_RGB_BAYER_NUM is available in the RGB raw data
combination format. In this format, the bit ePixPrecision and the orderee eBayerId
provided by the sensor generate an enum value in the RGB raw data combination format
from RGB_BAYER_PIXEL macro.

RGB_BAYER_PIXEL macro definition: #define RGB_BAYER_PIXEL(BitMode, PixelID)
(E_MI_SYS_PIXEL_FRAME_RGB_BAYER_BASE+
BitMode*E_MI_SYS_PIXEL_BAYERID_MAX+ PixelID), the range of values:
(E_MI_SYSAME_BASE_RGB_FRAME_FRAME_RGB_FRAME_PIXER_RGB_PIXER)

A small example of the use is provided below, and for more detailed information, please
refer to the API documentation for the corresponding module.

```
static MI_SYS_PixelFormat_e getRawType(STUB_SensorType_e sensorType)
{
```

```

MI_SYS_PixelFormat_e eRawType = E_MI_SYS_PIXEL_FRAME_RGB_BAYER_NUM;
MI_SNR_PlaneInfo_t stSnrPlane0Info;

memset(&stSnrPlane0Info, 0x0, sizeof(MI_SNR_PlaneInfo_t));
MI_SNR_GetPlaneInfo(E_MI_SNR_PAD_ID_0, 0, &stSnrPlane0Info);
eRawType = (MI_SYS_PixelFormat_e)RGB_BAYER_PIXEL(stSnrPlane0Info.ePixPrecision,
stSnrPlane0Info.eBayerId);

return eRawType;
}

```

- Related data types and interfaces

N/A

3.2.3 MI_SYS_CompressMode_e

- Description

Define Compression enumeration type.

- Define

```

typedef enum
{
    E_MI_SYS_COMPRESS_MODE_NONE, //no compress
    E_MI_SYS_COMPRESS_MODE_SEG, //compress unit is 256 bytes as a segment
    E_MI_SYS_COMPRESS_MODE_LINE, //compress unit is the whole line
    E_MI_SYS_COMPRESS_MODE_FRAME, //compress unit is the whole frame
    E_MI_SYS_COMPRESS_MODE_TO_8BIT,
    E_MI_SYS_COMPRESS_MODE_TO_6BIT,
    E_MI_SYS_COMPRESS_MODE_AFBC,
    E_MI_SYS_COMPRESS_MODE_SFBC0,
    E_MI_SYS_COMPRESS_MODE_SFBC1,
    E_MI_SYS_COMPRESS_MODE_SFBC2,
    E_MI_SYS_COMPRESS_MODE_BUTT, //number
}MI_SYS_CompressMode_e;

```

- Members

Members Name	Description
E_MI_SYS_COMPRESS_MODE_NONE	Uncompressed video formats
E_MI_SYS_COMPRESS_MODE_SEG	Segment compressed video format
E_MI_SYS_COMPRESS_MODE_LINE	Line-compressed video format, compressed by one segment of behavior
E_MI_SYS_COMPRESS_MODE_FRAME	Frame-compressed video format that compresses a frame of data
E_MI_SYS_COMPRESS_MODE_TO_8BIT	Compress pixel to 8-bit

E_MI_SYS_COMPRESS_MODE_TO_6BIT	Pixel compressed to 6bit
E_MI_SYS_COMPRESS_MODE_AFBC	Arm frame buffer compression
E_MI_SYS_COMPRESS_MODE_SFBC0	Sigmastar compression method 0
E_MI_SYS_COMPRESS_MODE_SFBC1	Sigmastar compression method 1
E_MI_SYS_COMPRESS_MODE_SFBC2	Sigmastar compression method 2
E_MI_SYS_COMPRESS_MODE_BUTT	Number of video compression methods

※ Note

N/A

➤ Related data types and interfaces

N/A

3.2.4 MI_SYS_FrameTileMode_e

➤ Description

Define Tile Format enumeration type.

➤ Define

```
typedef enum
{
    E_MI_SYS_FRAME_TILE_MODE_NONE = 0,
    E_MI_SYS_FRAME_TILE_MODE_16x16,    // tile mode 16x16
    E_MI_SYS_FRAME_TILE_MODE_16x32,    // tile mode 16x32
    E_MI_SYS_FRAME_TILE_MODE_32x16,    // tile mode 32x16
    E_MI_SYS_FRAME_TILE_MODE_32x32,    // tile mode 32x32
    E_MI_SYS_FRAME_TILE_MODE_128x32,   // tile mode 128x32
    E_MI_SYS_FRAME_TILE_MODE_128x64,   // tile mode 128x64
    E_MI_SYS_FRAME_TILE_MODE_64x4,     // tile mode 64x4
    E_MI_SYS_FRAME_TILE_MODE_MAX
} MI_SYS_FrameTileMode_e;
```

➤ Members

Members Name	Description
E_MI_SYS_FRAME_TILE_MODE_NONE	None
E_MI_SYS_FRAME_TILE_MODE_16x16	16x16 mode
E_MI_SYS_FRAME_TILE_MODE_16x32	16x32 mode
E_MI_SYS_FRAME_TILE_MODE_32x16	32x16 mode
E_MI_SYS_FRAME_TILE_MODE_32x32	32x32 mode
E_MI_SYS_FRAME_TILE_MODE_128x32	128x32 mode
E_MI_SYS_FRAME_TILE_MODE_128x64	128x64 mode
E_MI_SYS_FRAME_TILE_MODE_64x4	64x4 mode

※ Note

N/A

➤ Related data types and interfaces

N/A

3.2.5 MI_SYS_FieldType_e

➤ Description

Define Enumeration type.

➤ Define

```
typedef enum
{
    E_MI_SYS_FIELDTYPE_NONE,    //< no field.
    E_MI_SYS_FIELDTYPE_TOP,    //< Top field only.
    E_MI_SYS_FIELDTYPE_BOTTOM, //< Bottom field only.
    E_MI_SYS_FIELDTYPE_BOTH,   //< Both fields.
    E_MI_SYS_FIELDTYPE_NUM
} MI_SYS_FieldType_e;
```

➤ Members

Members Name	Description
E_MI_SYS_FIELDTYPE_NONE	None
E_MI_SYS_FIELDTYPE_TOP	Top field only
E_MI_SYS_FIELDTYPE_BOTTOM	Bottom field only
E_MI_SYS_FIELDTYPE_BOTH	Both fields

※ Note

N/A

➤ Related data types and interfaces

N/A

3.2.6 MI_SYS_BufDataType_e

➤ Description

Define Module ID enumeration type.

➤ Define

```
typedef enum
{
    E_MI_SYS_BUFDATA_RAW = 0,
```

```
E_MI_SYS_BUFDATA_FRAME,  
E_MI_SYS_BUFDATA_META,  
E_MI_SYS_BUFDATA_MULTIPLANE  
E_MI_SYS_BUFDATA_FBC,  
E_MI_SYS_BUFDATA_MAX  
} MI_SYS_BufDataType_e;
```

➤ Members

Members Name	Description
E_MI_SYS_BUFDATA_RAW	Raw data type
E_MI_SYS_BUFDATA_FRAME	Frame data type
E_MI_SYS_BUFDATA_META	Meta data type
E_MI_SYS_BUFDATA_MULTIPLANE	Multiplane data type
E_MI_SYS_BUFDATA_FBC	FBC data type
E_MI_SYS_BUFDATA_MAX	Number of data types

※ Note

N/A

➤ Related data types and interfaces

N/A

3.2.7 MI_SYS_FrameIsInfoType_e

➤ Description

Define types of ISP info enumerations carried by frame data.

➤ Define

```
{  
E_MI_SYS_FRAME_ISP_INFO_TYPE_NONE,  
E_MI_SYS_FRAME_ISP_INFO_TYPE_GLOBAL_GRADIENT  
} MI_SYS_FrameIsInfoType_e;
```

➤ Members

Members Name	Description
E_MI_SYS_FRAME_ISP_INFO_TYPE_NONE	NONE
E_MI_SYS_FRAME_ISP_INFO_TYPE_GLOBAL_GRADIENT	Global gradient data types for ISPs

※ Note

N/A

- Related data types and interfaces

N/A

3.2.8 MI_SYS_ChnPort_t

- Description

Define Module equipment channel structure.

- Define

```
typedef struct MI_SYS_ChnPort_s
{
    MI_ModuleId_e eModId;
    MI_U32 u32DevId;
    MI_U32 u32ChnId;
    MI_U32 u32PortId;
} MI_SYS_ChnPort_t;
```

- Members

Members Name	Description
eModId	Module No.
u32DevId	Device No.
u32ChnId	Channel No.
u32PortId	Port No.

- ※ Note

N/A

- Related data types and interfaces

N/A

3.2.9 MI_SYS_MetaData_t

- Description

Define Structure of the Code Stream MetaData.

- Define

```
typedef struct MI_SYS_MetaData_s
{
    void* pVirAddr;
    MI_PHY phyAddr; //notice that this is miu bus addr, not cpu bus addr.
}
```

```
MI_U32 u32Size;
MI_U32 u32ExtraData;  /*driver special flag*/
MI_ModuleId_e eDataFromModule;
} MI_SYS_MetaData_t;
```

➤ Members

Members Name	Description
pVirAddr	Data storage virtual address.
phyAddr	Data storage physical address
u32Size	Data size
u32ExtraData	driver special flag
eDataFromModule	Data from which module

※ Note

N/A

➤ Related data types and interfaces

N/A

3.2.10 MI_SYS_RawData_t

➤ Description

Define Structure of The Stream RawData.

➤ Define

```
typedef struct MI_SYS_RawData_s
{
    void* pVirAddr;
    MI_PHY phyAddr; /*notice that this is miu bus addr, not cpu bus addr.*/
    MI_U32 u32BufSize;

    MI_U32 u32ContentSize;
    MI_BOOL bEndOfFrame;
    MI_U64 u64SeqNum;
} MI_SYS_RawData_t;
```

➤ Members

Members Name	Description
pVirAddr	The virtual address of the stream package.
phyAddr	The physical address of the stream package
u32BufSize	Buf size

Members Name	Description
u32ContentSize	Data actually takes up buf size
bEndOfFrame	Whether the current frame ends. (Reserved parameter) currently only supports frame-by-frame transmission in frame mode.
u64SeqNum	Frame number of the current frame

※ Note

- Currently only support the transmission of data by frame, each time need to transfer a complete frame of data.
- When the stream frame data comes with PTS, the decoded Output data Output is the same PTS. When PTS is 1, do not refer to the system clock Output data frame.

➤ Related data types and interfaces

N/A

3.2.11 MI_SYS_WindowRect_t

➤ Description

Define window Structure of coordinates.

➤ Define

```
typedef struct MI_SYS_WindowRect_s
{
    MI_U16 u16X;
    MI_U16 u16Y;
    MI_U16 u16Width;
    MI_U16 u16Height;
}MI_SYS_WindowRect_t;
```

➤ Members

Members Name	Description
u16X	The value of the horizontal direction of the window start position.
u16Y	The value of the vertical direction of the window start position.
u16Width	Window width.
u16Height	Window height.

※ Note

N/A

➤ Related data types and interfaces

N/A

3.2.12 MI_SYS_FrameData_t

➤ Description

Define Structure of The Stream FrameData.

➤ Define

```
//N.B. in MI_SYS_FrameData_t should never support u32Size,
//for other values are enough,and not support u32Size is general standard method.
typedef struct MI_SYS_FrameData_s
{
    MI_SYS_FrameTileMode_e eTileMode;
    MI_SYS_PixelFormat_e ePixelFormat;
    MI_SYS_CompressMode_e eCompressMode;
    MI_SYS_FrameScanMode_e eFrameScanMode;
    MI_SYS_FieldType_e eFieldType;
    MI_SYS_FrameData_PhySignalType ePhylayoutType;

    MI_U16 u16Width;
    MI_U16 u16Height;

    //in case ePhylayoutType equal to REALTIME_FRAME_DATA, pVirAddr would be MI_SYS_REALTIME_MAGIC_PA
    //DDR and phyAddr would be MI_SYS_REALTIME_MAGIC_VADDR

    void* pVirAddr[3];
    MI_PHY phyAddr[3]; //notice that this is miu bus addr,not cpu bus addr.
    MI_U32 u32Stride[3];
    MI_U32 u32BufSize; //total size that allocated for this buffer,include consider alignment.

    MI_U16 u16RingBufStartLine; //Valid in case RINGBUF_FRAME_DATA, u16RingBufStartLine must be LGE tha
    //n 0 and less than u16Height
    MI_U16 u16RingBufRealTotalHeight; //Valid in case RINGBUF_FRAME_DATA, u16RingBufStartLine must be
    //LGE than u16Height

    MI_SYS_FrameIspInfo_t stFrameIspInfo; //isp info of each frame
    MI_SYS_WindowRect_t stContentCropWindow;
} MI_SYS_FrameData_t;
```

➤ Members

Members Name	Description
eTileMode	Tile mode
ePixelFormat	Pixel format
eCompressMode	Compressed format
eFrameScanMode	Frame scan module
eFieldType	File type
ePhylayoutType	The type of buffer that the frame belongs to
u16Width	Frame width
u16Height	Frame height

Members Name	Description
pVirAddr	Virtual address
phyAddr	Physical address
u32Stride	Number of bytes per row of pictures
u32BufSize	The actual buf size assigned by Sys to the current frame
u16RingBufStartLine	When ring mode, frame data starts with the number of rows of ring buffer
u16RingBufRealTotalHeight	The true height of frame data when ring mode
stFrameIspInfo	ISP info struct
stContentCropWindow	Crop info struct

➤ Note

N/A

➤ Related data types and interfaces

N/A

3.2.13 MI_SYS_BufInfo_t

➤ Description

Define Structure of code flow information.

➤ Define

```
typedef struct MI_SYS_BufInfo_s
{
    MI_U64 u64Pts;
    MI_U64 u64SidebandMsg;
    MI_SYS_BufDataType_e eBufType;
    MI_U32 u32SequenceNumber;
    MI_U32 bEndOfStream : 1;
    MI_U32 bUsrBuf : 1;
    MI_U32 bDrop : 1;
    MI_U32 bCrcCheck : 1;
    MI_U32 u32IrFlag : 2; // For Window Hello Usage: 0x00/off, 0x01/on, 0x02/invalid
    MI_U32 u32Reserved : 26;
    union
    {
        MI_SYS_FrameData_t stFrameData;
        MI_SYS_RawData_t stRawData;
        MI_SYS_MetaData_t stMetaData;
        MI_SYS_FrameDataMultiPlane_t stFrameDataMultiPlane;
        MI_SYS_FbcData_t stFbcData;
    };
} MI_SYS_BufInfo_t;
```

➤ Members

Members Name			Description
u64Pts			Timestamp
u64SidebandMsg			Side band message
eBufType			Buf type
u32SequenceNumber			Sequence number
Bit field	Bit 0	bEndOfStream	Whether all the information has been sent
	Bit 1	bUsrBuf	Whether user used buf
	Bit 2	bDrop	Whether to drop buf
	Bit 3	bCrcCheck	Whether to enable crc verification
	Bit 4-5	u32IrFlag	Ir flag bit
	Bit 6-31	u32Reserved	Reserved bit
Union	stFrameData		Frame data format structure
	stRawData		Raw data format structure
	stMetaData		Meta data format structure
	stFrameDataMultiPlane		Multi-Frame collection data format structure
	stFbcData		Frame buffer compressed data structure

※ Note

N/A

➤ Related data types and interfaces

N/A

3.2.14 MI_SYS_FrameBufExtraConfig_t

➤ Description

Define Structure sits additionally configured by Stream Frame buffer.

➤ Define

```
typedef struct MI_SYS_FrameBufExtraConfig_s
{
    //Buf alignment requirement in horizontal
    MI_U16 u16BufHAlignment;
    //Buf alignment requirement in vertical
    MI_U16 u16BufVAlignment;
    //Buf alignment requirement in chroma
    MI_U16 u16BufChromaAlignment;
    // Buf alignment requirement after compress
    MI_U16 u16BufCompressAlignment;
    // Buf Extra add after alignment
    MI_U16 u16BufExtraSize;
    //Clear padding flag
    MI_BOOL bClearPadding;
}
```

MI_SYS_FrameBufExtraConfig_t;

Members

Members Name	Description
u16BufHAlignment	Horizontal alignment values
u16BufVAlignment	Vertical alignment values
u16BufChromaAlignment	Chroma buffer size alignment value
u16BufCompressAlignmen t	Compress buffer size alignment value
u16BufExtraSize	Buffer size extra allocation value
bClearPadding	Whether to blacken the edge of the buffer

※ Note

N/A

Related data types and interfaces

N/A

3.2.15 MI_SYS_BufFrameConfig_t

Description

Define Structure of the Code Stream Frame buffer configuration.

Define

```
typedef struct MI_SYS_BufFrameConfig_s
{
    MI_U16 u16Width;
    MI_U16 u16Height;
    MI_SYS_FrameScanMode_e eFrameScanMode;
    MI_SYS_PixelFormat_e eFormat;
    MI_SYS_CompressMode_e eCompressMode;
}MI_SYS_BufFrameConfig_t;
```

Members

Members Name	Description
u16Width	Frame width
u16Height	Frame height
eFrameScanMode	Frame Scan Mode
eFormat	Frame pixel format
eCompressMode	Frame compress mode

※ Note

N/A

- Related data types and interfaces

N/A

3.2.16 MI_SYS_BufRawConfig_t

- Description

Define Structure of the code flow raw buffer configuration.

- Define

```
typedef struct MI_SYS_BufRawConfig_s
{
    MI_U32 u32Size;
}MI_SYS_BufRawConfig_t;
```

- Members

Members Name	Description
u32Size	Buf Size

- ※ Note

N/A

- Related data types and interfaces

N/A

3.2.17 MI_SYS_MetaDataConfig_t

- Description

Define Structure of the code flow meta buffer configuration.

- Define

```
typedef struct MI_SYS_MetaDataConfig_s
{
    MI_U32 u32Size;
}MI_SYS_MetaDataConfig_t;
```

- Members

Members Name	Description
u32Size	Buf Size

- ※ Note

N/A

- Related data types and interfaces

N/A

3.2.18 MI_SYS_BufConf_t

- Description

Define Structure of the Flow Port Buf configuration.

- Define

```
typedef struct MI_SYS_BufConf_s
{
    MI_SYS_BufDataType_e eBufType;
    MI_U32 u32Flags; // 0 or MI_SYS_MAP_VA
    MI_U64 u64TargetPts;
    MI_BOOL bDirectBuf; // Direct alloc buffer by Others.
    MI_BOOL bCrcCheck;
    union
    {
        MI_SYS_BufFrameConfig_t stFrameCfg;
        MI_SYS_BufRawConfig_t stRawCfg;
        MI_SYS_MetaDataConfig_t stMetaCfg;
        MI_SYS_BufFrameMultiPlaneConfig_t stMultiPlaneCfg;
        MI_SYS_BufFrameMetaConfig_t stFrameMetaCfg; // support frame buffer has allocated by Others.
        MI_SYS_FbcDataConfig_t stFbcCfg;
    };
}MI_SYS_BufConf_t;
```

- Members

Members Name		Description
eBufType		Buf type
u32Flags		Buf is map kernel space va or not
u64TargetPts		Set buf pts
bDirectBuf		Direct alloc buffer by Others
bCrcCheck		Buf enable crc check or not
Union	stFrameCfg	Set data format parameter of Frame
	stRawCfg	Set data format parameter of Raw
	stMetaCfg	Set data format parameter of Meta
	stMultiPlaneCfg	Set data format parameter of multi-Frame
	stFrameMetaCfg	Frame meta data config
	stFbcCfg	Fbc data config

- ※ Note

N/A

- Related data types and interfaces

N/A

3.2.19 MI_SYS_Version_t

- Description

Define Sys version structure information.

- Define

```
typedef struct MI_SYS_Version_s
{
    MI_U8 u8Version[128];
}MI_SYS_Version_t;
```

- Members

Members Name	Description
u8Version[128]	Describe sys version information string buf

- ※ Note

N/A

- Related data types and interfaces

N/A

3.2.20 MI_VB_PoolListConf_t

- Description

Define DescribeVB Pool Structure of linked information.

- Define

```
typedef struct MI_VB_PoolListConf_s
{
    MI_U32 u32PoolListCnt;
    MI_VB_PoolConf_t stPoolConf[MI_VB_POOL_LIST_MAX_CNT];
} MI_VB_PoolListConf_t;
```

- Members

Members Name	Description
u32PoolListCnt	VB pool list number of Members
stPoolConf	VB pool linked table Members configuration information

※ Note

N/A

➤ Related data types and interfaces

N/A

3.2.21 MI_SYS_BindType_e

➤ Description

Define Front and rear operating modes.

➤ Define

```
typedef enum
{
    E_MI_SYS_BIND_TYPE_FRAME_BASE = 0x00000001,
    E_MI_SYS_BIND_TYPE_SW_LOW_LATENCY = 0x00000002,
    E_MI_SYS_BIND_TYPE_REALTIME = 0x00000004,
    E_MI_SYS_BIND_TYPE_HW_AUTOSYNC = 0x00000008,
    E_MI_SYS_BIND_TYPE_HW_RING = 0x00000010
}MI_SYS_BindType_e;
```

➤ Members

Members Name	Description
E_MI_SYS_BIND_TYPE_FRAME_BASE	frame mode, default how to work
E_MI_SYS_BIND_TYPE_SW_LOW_LATENCY	Low latency working
E_MI_SYS_BIND_TYPE_REALTIME	How hardware is directly connected
E_MI_SYS_BIND_TYPE_HW_AUTOSYNC	Front and rear handshake level, buffer size consistent with image resolution
E_MI_SYS_BIND_TYPE_HW_RING	Front and rear handshake levels, ring buffer depth can be trated less than the image resolution is high

※ Note

- Older versions of MI SYS do not provide this function, if not found, do not have to set.
- The FRAME_BASE and HW_RING types use DRAM, and the REALTIME type uses SRAM.
- Due to underlying restrictions, input ports bound by HW_RING or REALTIME cannot use MI_SYS_ChnInputPortGetBuf/ MI_SYS_ChnInputPortPutBuf, and output ports cannot use MI_SYS_ChnOutputPortGetBuf/ MI_SYS_ChnOutputPortPutBuf.
- Comparison of memory size used by different bindtypes: FRAME_BASE type> HW_RING type> REALTIME type.

➤ Related data types and interfaces

N/A

3.2.22 MI_SYS_FrameData_PhySignalType

➤ Description

Describe frame data Enumerated types of buffer types that are affiliated.

➤ Define

```
typedef enum
{
    REALTIME_FRAME_DATA,
    RINGBUF_FRAME_DATA,
    NORMAL_FRAME_DATA,
}MI_SYS_FrameData_PhySignalType;
```

➤ Members

Members Name	Description
REALTIME_FRAME_DATA	In E_MI_SYS_BIND_TYPE_REALTIME mode
RINGBUF_FRAME_DATA	The resulting frame data
NORMAL_FRAME_DATA	In E_MI_SYS_BIND_TYPE_HW_RING mode

※ Note

Older versions of MI SYS do not provide this function, if not found, do not have to set.

➤ Related data types and interfaces

N/A

3.2.23 MI_SYS_InsidePrivatePoolType_e

➤ Description

Describe the created enumeration type of private MMA POOL type.

➤ Define

```
typedef enum
{
    E_MI_SYS_VPE_TO_VENC_PRIVATE_RING_POOL = 0,
    E_MI_SYS_PER_CHN_PRIVATE_POOL=1,
    E_MI_SYS_PER_DEV_PRIVATE_POOL=2,
    E_MI_SYS_PER_CHN_PORT_OUTPUT_POOL=3,
    E_MI_SYS_PER_DEV_PRIVATE_RING_POOL=4,
}MI_SYS_InsidePrivatePoolType_e;
```

➤ Members

Members Name	Description
E_MI_SYS_VPE_TO_VENC_PRIVATE_RING_POOL	Private Ring Pool Type for SCL and VENC. mi_sys2.0 indicates VPE and VENC
E_MI_SYS_PER_CHN_PRIVATE_POOL	Channel Private Pool Type
E_MI_SYS_PER_DEV_PRIVATE_POOL	Device Private Pool Type
E_MI_SYS_PER_CHN_PORT_OUTPUT_POOL	Output Port Private Pool Type
E_MI_SYS_PER_DEV_PRIVATE_RING_POOL	Device Private Ring Pool Type In Maruko and later chips, it need to apply the ring pool with this enum type before binding with HW_Ring mode.

※ Note

Older versions of MI SYS do not provide this function, if not found, do not have to set.

➤ Related data types and interfaces

N/A

3.2.24 MI_SYS_PerChnPrivHeapConf_t

➤ Description

Define the structure describing channel private MMA Pool.

➤ Define

```
typedef struct MI_PerChnPrivHeapConf_s
{
    MI_ModuleId_e eModule;
    MI_U32 u32Devid;
    MI_U32 u32Channel;
    MI_U8 u8MMAHeapName[MI_MAX_MMA_HEAP_LENGTH];
    MI_U32 u32PrivateHeapSize;
}MI_SYS_PerChnPrivHeapConf_t;
```

➤ Members

Members Name	Description
eModule	Module ID
u32Devid	Device ID
u32Channel	Channel ID
u8MMAHeapName	Mma heap name
u32PrivateHeapSize	Private pool size

※ Note

N/A

- Related data types and interfaces

N/A

3.2.25 MI_SYS_PerDevPrivHeapConf_t

- Description

Define the structure describing the device's private MMA Pool.

- Define

```
typedef struct MI_PerDevPrivHeapConf_s
{
    MI_ModuleId_e eModule;
    MI_U32 u32Devid;
    MI_U32 u32Reserve;
    MI_U8 u8MMAHeapName[MI_MAX_MMA_HEAP_LENGTH];
    MI_U32 u32PrivateHeapSize;
}MI_SYS_PerDevPrivHeapConf_t;
```

- Members

Members Name	Description
eModule	Module ID
u32Devid	Device ID
u32Reserve	Reserved
u8MMAHeapName	Mma heap name
u32PrivateHeapSize	Private pool size

- ※ Note

N/A

- Related data types and interfaces

N/A

3.2.26 MI_SYS_PerVpe2VencRingPoolConff_t

- Description

Define the structure describing private Ring MMA Pool between SCL and VENC.
mi_sys2.0 indicates between VPE and VENC.

- Define

```
typedef struct MI_SYS_PerVpe2VencRingPoolConff_s
{
```

```
MI_U32 u32VencInputRingPoolStaticSize;
MI_U8 u8MMAHeapName[MI_MAX_MMA_HEAP_LENGTH];
}MI_SYS_PerVpe2VencRingPoolConf_t;
```

➤ Members

Members Name	Description
u32VencInputRingPoolStaticSize	Private pool size
u8MMAHeapName	Mma heap name

※ Note

N/A

➤ Related data types and interfaces

N/A

3.2.27 MI_SYS_PerChnPortOutputPool_t

➤ Description

Define the private MMA Pool structure describing output port.

➤ Define

```
typedef struct MI_SYS_PerChnPortOutputPool_s
{
    MI_ModuleId_e eModule;
    MI_U32 u32Devid;
    MI_U32 u32Channel;
    MI_U32 u32Port;
    MI_U8 u8MMAHeapName[MI_MAX_MMA_HEAP_LENGTH];
    MI_U32 u32PrivateHeapSize;
}MI_SYS_PerChnPortOutputPool_t;
```

➤ Members

Members Name	Description
eModule	Module ID
u32Devid	Device ID
u32Channel	Channel ID
u32Port	Port ID
u8MMAHeapName	Mma heap name
u32PrivateHeapSize	Private pool size

※ Note

N/A

- Related data types and interfaces

N/A

3.2.28 MI_SYS_PerDevPrivRingPoolConf_t

- Description

Define the private MMA Pool structure describing output port.

- Define

```
typedef struct MI_SYS_PerDevPrivRingPoolConf_s
{
    MI_ModuleId_e eModule;
    MI_U32 u32Devid;
    MI_U16 u16MaxWidth;
    MI_U16 u16MaxHeight;
    MI_U16 u16RingLine;
    MI_U8 u8MMAHeapName[MI_MAX_MMA_HEAP_LENGTH];
} MI_SYS_PerDevPrivRingPoolConf_t;
```

- Members

Members Name	Description
eModule	Module ID
u32Devid	Device ID
u16MaxWidth	The max width of Ring Pool
u16MaxHeight	The max Height of Ring Pool
u16RingLine	Ring Pool total line
u8MMAHeapName	Mma heap name

- ※ Note

N/A

- Related data types and interfaces

N/A

3.2.29 MI_SYS_GlobalPrivPoolConfig_t

- Description

Define the structure describing private MMA Pool.

- Define

```
typedef struct MI_SYS_GlobalPrivPoolConfig_s
```

```

{
    MI_SYS_InsidePrivatePoolType_e eConfigType;
    MI_BOOL bCreate;
    union
    {
        MI_SYS_PerChnPrivHeapConf_t stPreChnPrivPoolConfig;
        MI_SYS_PerDevPrivHeapConf_t stPreDevPrivPoolConfig;
        MI_SYS_PerVpe2VencRingPoolConf_t stPreVpe2VencRingPrivPoolConfig;
        MI_SYS_PerChnPortOutputPool_t stPreChnPortOutputPrivPool;
        MI_SYS_PerDevPrivRingPoolConf_t stpreDevPrivRingPoolConfig;
    }uConfig;
}MI_SYS_GlobalPrivPoolConfig_t;

```

➤ Members

Members Name	Description
eConfigType	Private pool type
bCreate	Whether Create PrivatePool.true: Creat false: Destroyed
uConfig	Structures of different types of private pool

※ Note

N/A

➤ Related data types and interfaces

N/A

3.2.30 MI_SYS_FrameIspInfo_t

➤ Description

Define the structure of ISP info in frame data.

➤ Define

```

typedef struct MI_SYS_FrameIspInfo_s
{
    MI_SYS_FrameIspInfoType_e eType;
    union
    {
        MI_U32 u32GlobalGradient;
    }uIspInfo;
}MI_SYS_FrameIspInfo_t;

```

➤ Members

Members Name	Description
eType	ISP info type

uIspInfo	ISP info union
----------	----------------

※ Note

N/A

➤ Related data types and interfaces

N/A

3.2.31 MI_SYS_FbcData_t

➤ Description

Define data structure for frame buffer compress.

➤ Define

```
typedef struct MI_SYS_FbcData_s
{
    MI_SYS_FrameTileMode_e eTileMode;
    MI_SYS_PixelFormat_e ePixelFormat;
    MI_SYS_CompressMode_e eCompressMode;

    MI_SYS_FrameData_PhySignalType ePhylayoutType;

    MI_U16 u16Width;
    MI_U16 u16Height;

    void* pVirAddr[2];
    MI_PHY phyAddr[2]; // notice that this is miu bus addr,not cpu bus addr.
    MI_U32 u32Stride[2];
    MI_U32 u32BufSize; // total size that allocated for this buffer,include consider alignment.

    /*
    *
    * -----
    *          |          |          ^
    *          |          |          |
    * u16RingBufStartLine --> |          |----|
    *          |          | ^          |
    *          | Ring    | |          |
    *          |          | u16Height |
    *          | Heap    | |          |
    *          |          | | u16RingHeapTotalLines
    *          |          | v          |
    * u16RingBufEndLine --> |          |----|
    *          |          |          |
    *          |          |          v
    */
```

```

*
*/

MI_U16 u16RingBufStartLine;
MI_U16 u16RingBufEndLine;
MI_U16 u16RingHeapTotalLines;

MI_PHY phyFbcTableAddr[2];
MI_U32 u32FbcTableSize[2];
} MI_SYS_FbcData_t;

```

➤ Members

Members Name	Description
eTileMode	Tile format type
ePixelFormat	Pixel enumeration type
eCompressMode	Compression mode
ePhylayoutType	Frame data buffer type
u16Width	Frame data width
u16Height	Frame data height
pVirAddr[2]	Frame data buffer virtual address
phyAddr[2]	Frame data buffer physical address
u32Stride[2]	Bytes occupied by each row of Frame data
u32BufSize	Frame data buffer size
u16RingBufStartLine	Ring buffer start line
u16RingBufEndLine	Ring buffer end line
u16RingHeapTotalLines	Ring buffer total line
phyFbcTableAddr[2]	Convert Fbc data to nv12 table, currently only supports to nv12.
u32FbcTableSize[2]	Convert Fbc data to nv12 table size.

※ Note

N/A

➤ Related data types and interfaces

N/A

3.2.32 MI_SYS_BufFrameMultiPlaneConfig_t

➤ Description

Define Structure of the code flow multiplane buffer configuration.

➤ Define

```
typedef struct MI_SYS_BufFrameMultiPlaneConfig_s
```

```
{
    MI_U8 u8SubPlaneNum;
    MI_SYS_BufFrameConfig_t stFrameCfg;
}MI_SYS_BufFrameMultiPlaneConfig_t;
```

➤ Members

Members Name	Description
u8SubPlaneNum	Buf count
stFrameCfg	Define Structure of the code flow frame buffer configuration.

※ Note

N/A

➤ Related data types and interfaces

N/A

3.2.33 MI_SYS_FrameDataSubPlane_t

➤ Description

Define Structure of The Stream SubPlane FrameData.

➤ Define

```
typedef struct MI_SYS_FrameDataSubPlane_s
{
    MI_SYS_PixelFormat_e ePixelFormat;
    MI_SYS_CompressMode_e eCompressMode;
    MI_U64 u64FrameId;

    MI_U16 u16Width;
    MI_U16 u16Height;
    void* pVirAddr[2];
    MI_PHY phyAddr[2];
    MI_U16 u16Stride[2];
    MI_U32 u32BufSize;
} MI_SYS_FrameDataSubPlane_t;
```

➤ Members

Members Name	Description
ePixelFormat	Pixel format
eCompressMode	Compressed format
u64FrameId	Frame id
u16Width	Frame width
u16Height	Frame height

Members Name	Description
pVirAddr	Virtual address
phyAddr	Physical address
u16Stride	Number of bytes per row of pictures
u32BufSize	The actual buf size assigned by Sys to the current frame

※ Note

N/A

➤ Related data types and interfaces

N/A

3.2.34 MI_SYS_BufFrameMetaConfig_t

➤ Description

Define structure of frame meta data.

➤ Define

```
typedef struct MI_SYS_BufFrameMetaConfig_s
{
    MI_U16          u16Width;
    MI_U16          u16Height;
    MI_SYS_FrameScanMode_e eFrameScanMode; //
    MI_SYS_PixelFormat_e  eFormat;
    MI_SYS_CompressMode_e eCompressMode;

    void* pVirAddr[3];
    MI_PHY phyAddr[3];
    MI_U16 u32Stride[3];
    MI_U32 u32BufSize;
} MI_SYS_BufFrameMetaConfig_t;
```

➤ Members

Members Name	Description
u16Width	Data width
u16Height	Data height
eFrameScanMode	Frame scan mode
eFormat	Pixel format
eCompressMode	Compression mode
pVirAddr[3]	Virtual address
phyAddr[3]	Physical address
u32Stride[3]	Bytes occupied by each line of the picture
u32BufSize	The actual buf size allocated by Sys to the current frame

※ Note

N/A

➤ Related data types and interfaces

N/A

3.2.35 MI_SYS_FrameDataMultiPlane_t

➤ Description

Define Structure of the Stream MultiPlane FrameData.

➤ Define

```
typedef struct MI_SYS_FrameDataMultiPlane_s
{
    MI_U8 u8SubPlaneNum;
    MI_SYS_FrameDataSubPlane_t stSubPlanes[MI_SYS_MAX_SUB_PLANE_CNT];
} MI_SYS_FrameDataMultiPlane_t;
```

➤ Members

Members Name	Description
u8SubPlaneNum	Buf count
stSubPlanes	Define Structure of the code flow MultiPlane buffer configuration.

※ Note

N/A

➤ Related data types and interfaces

N/A

3.2.36 MI_SYS_UserPictureInfo_t

➤ Description

Define the structure describing user picture info.

➤ Define

```
typedef struct MI_SYS_UserPictureInfo_s
{
    MI_U32 u32SrcFrc;
} MI_SYS_UserPictureInfo_t;
```

➤ Members

Members Name	Description
U32SrcFrc	Set the frame rate of Input port buf

※ Note

N/A

➤ Related data types and interfaces

N/A

4. Error Code

4.1. The composition of the error code

Mi returns the error code for 4 bytes, a total of 32bits, consisting of five parts as shown in Table 4-1:

Table 4-6: The composition of the error code

Position	Bits	Description
BIT [0-11]	12bits	The error type that indicates the specific error meaning of the error code.
BIT [12-15]	4bits	Error level, fixed return 2.
BIT [16-23]	8bits	Module ID, which module the error code belongs to.
BIT [24-31]	8bits	Fixed to 0xA0.

Tips:

We can simply think of the error code as two double bytes (16bits) for quick interpretation, in the case of the error code 0xA0092001:

"A009": Indicates that the error occurred in a module with module ID 9. By looking at the definition of "MI_ModuleId_e", it can be known that it is the MI_SYS module.

"2001": The error type that indicates the error is 1, i.e. "Device ID is out of legal range".

4.2. The error code list for MI_SYS API

MI_SYS module all APIs return a value of 4 bytes, 0 indicates execution success, and other values indicate execution failure, the need for Note is that all! 0 Return should be listed in Table 4-2, otherwise it is illegal.

Table 4-7: MI_SYS Module error code

Error Code	Definition	Description
0xA0092001	MI_ERR_SYS_INVALID_DEVID	Device ID out of legal range
0xA0092002	MI_ERR_SYS_INVALID_CHNID	Channel group number error or invalid area handle
0xA0092003	MI_ERR_SYS_ILLEGAL_PARAM	Argument out of legal range
0xA0092004	MI_ERR_SYS_EXIST	Repeat a device, channel, or resource that already exists in Create
0xA0092005	MI_ERR_SYS_UNEXIST	Trying to use a setting that does not exist in Destroyed
0xA0092006	MI_ERR_SYS_NULL_PTR	access, channel, or resource
0xA0092007	MI_ERR_SYS_NOT_CONFIG	There are empty pointers in the function arguments
0xA0092008	MI_ERR_SYS_NOT_SUPPORT	The module is not configured

Error Code	Definition	Description
0xA0092009	MI_ERR_SYS_NOT_PERM	Unsupported parameters or Function
0xA009200C	MI_ERR_SYS_NOMEM	This operation is not allowed, such as an attempt to modify static configuration parameters
0xA009200D	MI_ERR_SYS_NOBUF	Failed to allocate memory, such as insufficient system memory
0xA009200E	MI_ERR_SYS_BUF_EMPTY	Allocation cache failed, such as the requested data buffer is too large
0xA009200F	MI_ERR_SYS_BUF_FULL	No data in the buffer
0xA0092010	MI_ERR_SYS_NOTREADY	Full data in buffer
0xA0092011	MI_ERR_SYS_BADADDR	The system did not initialize or load the module
0xA0092012	MI_ERR_SYS_BUSY	The address's illegal
0xA0092013	MI_ERR_SYS_CHN_NOT_STARTED	The system is busy
0xA0092014	MI_ERR_SYS_CHN_NOT_STOPED	The channel didn't start.
0xA0092015	MI_ERR_SYS_NOT_INIT	The channel has not stopped.
0xA0092016	MI_ERR_SYS_INITED	The module is not initialized
0xA0092017	MI_ERR_SYS_NOT_ENABLE	Module has been initialized
0xA0092018	MI_ERR_SYS_NOT_DISABLE	Channel or port does not have ENABLE
0xA0092019	MI_ERR_SYS_TIMEOUT	Channel or port does not have DISABLE
0xA009201A	MI_ERR_SYS_DEV_NOT_STARTED	Timeout
0xA009201B	MI_ERR_SYS_DEV_NOT_STOPED	The device didn't start
0xA009201C	MI_ERR_SYS_CHN_NO_CONTENT	The device is not stopped
0xA009201D	MI_ERR_SYS_NOVASAPCE	There's no information in the channel.
0xA009201E	MI_ERR_SYS_NOITEM	Failed to map virtual address
0xA009201F	MI_ERR_SYS_FAILED	There's no record in RingPool.

5. PROCFS INTRODUCTION

5.1. common

5.1.1 dump_mmap

- Debug Information
/config/dump_mmap

```
/ # /config/dump_mmap
u32TotalSize:0x40000000
u32Miu0Size:0x40000000
u32Miu1Size:0x0
u32MiuBoundary:0x80000000
u32MmapItemsNum:4
bIs4kAlign:1
bMiu1Enable:0
E_LX_MEM:GID=0,Addr=0x0,Size=0x3ffe0000,Layer=0,Align=0x1000,MemoryType=0x4,MiuNo=0,CMAID=0
E_MMAP_ID_DUMMY1:GID=1,Addr=0x0,Size=0x3f000000,Layer=1,Align=0x1000,MemoryType=0x4,MiuNo=0,CMAID=0
E_LX_LOGO_RESERVED_FB:GID=2,Addr=0x3f000000,Size=0x800000,Layer=1,Align=0x1000,MemoryType=0x4,MiuNo=0,CMAID=0
E_MMAP_ID_EMI:GID=3,Addr=0x3ffe0000,Size=0x20000,Layer=0,Align=0x1000,MemoryType=0x4,MiuNo=0,CMAID=0
```

- Debug Information Analysis
./config/dump_mmap display the related memory information according to the /config/mmap.ini file.
- Parameter Description

Parameter		Description
u32TotalSize		the total Dram size
u32Miu0Size		the MIU0 address size
u32Miu1Size		the MIU1 address size
u32MiuBoundary		MIU0 end address
u32MmapItemsNum		the Dram mmap Item num. Currently it is two Dram mmap Items, E_LX_MEM and E_MMAP_ID_EMI
bIs4kAlign		4k alignment is required for reading and writing Dram or not.
bMiu1Enable		MIU1 enable or not
E_LX_MEM	GID	Mmap
	Addr	Memory address
	Size	Memory size
	Layer	Memory layer
	Align	Memory align
	MemoryType	Memory type

Parameter		Description
	MiuNo	MIU NO.
	CMAID	CMA id
E_MMAP_ID_DUMMY1		the same as above
E_LX_LOGO_RESERVED_FB		the same as above
E_MMAP_ID_EMI		the same as above

5.1.2 dump_config

- Debug Information
/config/dump_config

```

panel cnt:59
PanelName:RM68200_720x1280 Intftype:11 timing:55
PanelName:SAT070AT50H18BH_1024x600 Intftype:0 timing:55
PanelName:BT1120OUTPUT_720p_60HZ Intftype:14 timing:55
PanelName:BT656OUTPUT_576p_50HZ Intftype:12 timing:55
PanelName:ST7789V Intftype:16 timing:55
PanelName:ILI9881C_800x1280 Intftype:11 timing:55
PanelName:ILI9163C_128x128 Intftype:15 timing:55
PanelName:DACOUT_576I Intftype:7 timing:4
PanelName:DACOUT_480I Intftype:7 timing:2
PanelName:DACOUT_1080P_24 Intftype:8 timing:10
PanelName:DACOUT_1080P_25 Intftype:8 timing:11
PanelName:DACOUT_1080P_30 Intftype:8 timing:12
PanelName:DACOUT_1080P_50 Intftype:8 timing:15
PanelName:DACOUT_720P_2997 Intftype:8 timing:6
PanelName:DACOUT_720P_50 Intftype:8 timing:7
PanelName:DACOUT_720P_5994 Intftype:8 timing:8
PanelName:DACOUT_720P_60 Intftype:8 timing:9
PanelName:DACOUT_1080P_2997 Intftype:8 timing:13
PanelName:DACOUT_1080I_50 Intftype:7 timing:14
PanelName:DACOUT_1080P_5994 Intftype:8 timing:16
PanelName:DACOUT_1080I_60 Intftype:7 timing:17
PanelName:DACOUT_1080P_60 Intftype:8 timing:18
PanelName:DACOUT_576P Intftype:8 timing:5
PanelName:DACOUT_480P Intftype:8 timing:3
PanelName:DACOUT_2K2KP_30 Intftype:8 timing:27
PanelName:DACOUT_2K2KP_60 Intftype:8 timing:28
PanelName:DACOUT_4K2KP_24 Intftype:8 timing:33

```

PanelName:DACOUT_4K2KP_25 Intftype:8 timing:34
PanelName:DACOUT_4K2KP_30 Intftype:8 timing:35
PanelName:DACOUT_4K2KP_50 Intftype:8 timing:36
PanelName:DACOUT_4K2KP_60 Intftype:8 timing:37
PanelName:DACOUT_1440P_50 Intftype:8 timing:19
PanelName:DACOUT_1440P_60 Intftype:8 timing:20
PanelName:VGAOUT_1024x768P_60 Intftype:8 timing:45
PanelName:VGAOUT_1280x1024P_60 Intftype:8 timing:48
PanelName:VGAOUT_1366x768P_60 Intftype:8 timing:46
PanelName:VGAOUT_1440x900P_60 Intftype:8 timing:49
PanelName:VGAOUT_1280x800P_60 Intftype:8 timing:47
PanelName:VGAOUT_1680x1050P_60 Intftype:8 timing:51
PanelName:VGAOUT_1600x1200P_60 Intftype:8 timing:50
PanelName:BT1120OUT_576P_50 Intftype:14 timing:5
PanelName:BT1120OUT_480P_60 Intftype:14 timing:3
PanelName:BT1120OUT_1080P_25 Intftype:14 timing:11
PanelName:BT1120OUT_1080P_30 Intftype:14 timing:12
PanelName:BT1120OUT_1080P_50 Intftype:14 timing:15
PanelName:BT1120OUT_1080P_2997 Intftype:14 timing:13
PanelName:BT1120OUT_1080P_5994 Intftype:14 timing:16
PanelName:BT1120OUT_1080P_60 Intftype:14 timing:18
PanelName:BT1120OUT_720P_50 Intftype:14 timing:7
PanelName:BT1120OUT_720P_5994 Intftype:14 timing:8
PanelName:BT1120OUT_720P_60 Intftype:14 timing:9
PanelName:BT656OUT_576P_50 Intftype:12 timing:5
PanelName:BT656OUT_480P_60 Intftype:12 timing:3
PanelName:BT656OUT_1080P_25 Intftype:12 timing:11
PanelName:BT656OUT_1080P_2997 Intftype:12 timing:13
PanelName:BT656OUT_1080P_30 Intftype:12 timing:12
PanelName:BT656OUT_720P_50 Intftype:12 timing:7
PanelName:BT656OUT_720P_60 Intftype:12 timing:9
PanelName:BT656OUT_720P_5994 Intftype:12 timing:8
pq enable:0
pq table size:25892
DBC Value=
 0,0,0
 0,0,0
 0,0,0
start dump [motion_table](0, 0)
end dump
start dump [motion_hdmi_dtv_table](0, 0)
end dump
start dump [motion_comp_pc_table](0, 0)
end dump

```
start dump [misc_param_table](0, 0)
end dump
start dump [misc_luma_table](0, 0)
end dump
start dump [noise_table](0, 0)
end dump
start dump [misc_table](0, 0)
end dump
```

➤ Debug Information Analysis

./config/dump_config gets the information of the files in /proc/mi_modules/common/, including panel size, DBC Value, motion_table, motion_hdmi_dtv_table, motion_comp_pc_table, misc_param_table, misc_luma_table, noise_table, misc_table, etc.

➤ Parameter Description

Parameter		Description
panel	PanelName	LCD Panel Name
	Intftype	LCD Panel Interface Type: E_MAPI_PNL_INTF_TTL = 0, ///< TTL type E_MAPI_PNL_INTF_LVDS, ///< LVDS type E_MAPI_PNL_INTF_RSDS, ///< RSDS type E_MAPI_PNL_INTF_MINILVDS, ///< TCON ///< Analog TCON E_MAPI_PNL_INTF_ANALOG_MINILVDS, ///< Digital TCON E_MAPI_PNL_INTF_DIGITAL_MINILVDS, E_MAPI_PNL_INTF_MFC, ///< Ursa (TTL output to Ursa) E_MAPI_PNL_INTF_DAC_I, ///< DAC output E_MAPI_PNL_INTF_DAC_P, ///< DAC output ///< For PDP(Vsync use Manually MODE) E_MAPI_PNL_INTF_PDPLVDS, E_MAPI_PNL_INTF_EXT, ///< EXT LPLL TYPE E_MAPI_PNL_INTF_MIPI_DSI, E_MAPI_PNL_INTF_BT656, E_MAPI_PNL_INTF_BT601, E_MAPI_PNL_INTF_BT1120, ///< BT1120 E_MAPI_PNL_INTF_MCU_TYPE, ///< MCU Type E_MAPI_PNL_INTF_SRGB, ///< sRGB E_MAPI_PNL_INTF_TTL_SPI_IF,
	timing	LCD Panel Display Output Timing: DISPLAYTIMING_MIN = 0, DISPLAYTIMING_DACOUT_DEFAULT = 0, DISPLAYTIMING_DACOUT_FULL_HD,

Parameter	Description
	DISPLAYTIMING_DACOUT_480I, DISPLAYTIMING_DACOUT_480P, DISPLAYTIMING_DACOUT_576I, DISPLAYTIMING_DACOUT_576P, // 5 DISPLAYTIMING_DACOUT_720P_2997, DISPLAYTIMING_DACOUT_720P_50, DISPLAYTIMING_DACOUT_720P_5994, DISPLAYTIMING_DACOUT_720P_60, DISPLAYTIMING_DACOUT_1080P_24, DISPLAYTIMING_DACOUT_1080P_25, DISPLAYTIMING_DACOUT_1080P_30, // 12 DISPLAYTIMING_DACOUT_1080P_2997, DISPLAYTIMING_DACOUT_1080I_50, DISPLAYTIMING_DACOUT_1080P_50, DISPLAYTIMING_DACOUT_1080P_5994, DISPLAYTIMING_DACOUT_1080I_60, DISPLAYTIMING_DACOUT_1080P_60, DISPLAYTIMING_DACOUT_1440P_50, DISPLAYTIMING_DACOUT_1440P_60, DISPLAYTIMING_DACOUT_1470P_24, DISPLAYTIMING_DACOUT_1470P_30, DISPLAYTIMING_DACOUT_1470P_60, DISPLAYTIMING_DACOUT_2205P_24, // For 2k2k DISPLAYTIMING_DACOUT_2K2KP_24, // 23 DISPLAYTIMING_DACOUT_2K2KP_25, DISPLAYTIMING_DACOUT_2K2KP_30, DISPLAYTIMING_DACOUT_2K2KP_60, // For 4k0.5k DISPLAYTIMING_DACOUT_4K540P_240, // For 4k1k DISPLAYTIMING_DACOUT_4K1KP_30, DISPLAYTIMING_DACOUT_4K1KP_60, DISPLAYTIMING_DACOUT_4K1KP_120, // For 4k2k DISPLAYTIMING_DACOUT_4K2KP_24, DISPLAYTIMING_DACOUT_4K2KP_25, DISPLAYTIMING_DACOUT_4K2KP_30, // 33 DISPLAYTIMING_DACOUT_4K2KP_50, DISPLAYTIMING_DACOUT_4K2KP_60, DISPLAYTIMING_DACOUT_4096P_24, DISPLAYTIMING_DACOUT_4096P_25,

Parameter		Description
		DISPLAYTIMING_DACOUT_4096P_30, DISPLAYTIMING_DACOUT_4096P_50, DISPLAYTIMING_DACOUT_4096P_60, // For VGA OUTPUT DISPLAYTIMING_VGAOUT_640x480P_60, DISPLAYTIMING_VGAOUT_1280x720P_60, DISPLAYTIMING_VGAOUT_1024x768P_60, DISPLAYTIMING_VGAOUT_1366x768P_60, DISPLAYTIMING_VGAOUT_1280x800P_60, DISPLAYTIMING_VGAOUT_1280x1024P_60, DISPLAYTIMING_VGAOUT_1440x900P_60, DISPLAYTIMING_VGAOUT_1600x1200P_60, DISPLAYTIMING_VGAOUT_1680x1050P_60, DISPLAYTIMING_VGAOUT_1920x1080P_60, // 43 // For TTL output DISPLAYTIMING_TTLOUT_480X272_60, DISPLAYTIMING_DACOUT_4K2KP_120, DISPLAYTIMING_USER, DISPLAYTIMING_MAX_NUM,
pq enable		pq enable or not
pq table size		pq table size value
DBC Value		No effect at present
motion_table		No effect at present
motion_hdmi_dtv_table		No effect at present
motion_comp_pc_table		No effect at present
misc_param_table		No effect at present
misc_luma_table		No effect at present
noise_table		No effect at present
misc_table		No effect at present

5.2. mi_log_info

5.2.1 cat

➤ Debug Information

```
# cat /proc/mi_modules/mi_log_info
```

```
----- Log Path -----
```

```
log path:
```

```
----- Store Path -----
```

```
store path: /mnt
```

----- Module Log Level -----

Log module	Level
------------	-------

mi_ive	2(WRN)
mi_vdf	2(WRN)
mi_venc	2(WRN)
mi_rgn	2(WRN)
mi_ai	2(WRN)
mi_ao	2(WRN)
mi_vif	2(WRN)
mi_vpe	2(WRN)
mi_vdec	2(WRN)
mi_sys	2(WRN)
mi_fb	2(WRN)
mi_hdmi	2(WRN)
mi_divp	2(WRN)
mi_gfx	2(WRN)
mi_vdisp	2(WRN)
mi_disp	2(WRN)
mi_os	2(WRN)
mi_iae	2(WRN)
mi_md	2(WRN)
mi_od	2(WRN)
mi_shadow	2(WRN)
mi_warp	2(WRN)
mi_uac	2(WRN)
mi_ldc	2(WRN)
mi_sd	2(WRN)
mi_panel	2(WRN)
mi_cipher	2(WRN)
mi_sensor	2(WRN)
mi_wlan	2(WRN)
mi_ipu	2(WRN)

```
mi_mipitx      2(WRN)
mi_gyro        2(WRN)
mi_jpd         2(WRN)
mi_isp         2(WRN)
mi_scl         2(WRN)
mi_wbc         2(WRN)
mi_dsp         2(WRN)
mi_pcie        2(WRN)
mi_dummy       2(WRN)
```

help example:

```
echo mi_sys=2 > /proc/mi_modules/mi_log_info
echo mi_vdisp=1 > /proc/mi_modules/mi_log_info
echo log=/mnt > /proc/mi_modules/mi_log_info
echo storepath=/mnt > /proc/mi_modules/mi_log_info
```

- Debug Information Analysis
Show default log path or store path, also show the value of each module's debug_level.
- Parameter Description

Parameter		Description
mi_log_info	log path	No effect at present
	store path	No effect at present
	Log module	The name of each module
	Level	Log level

5.2.2 echo

Function	Modify module debug_level					
Command	echo [ModID]=[Level] > /proc/mi_modules/mi_log_info					
	[ModID] mi_ive mi_vdf mi_venc mi_rgn mi_ai mi_ao mi_vif mi_vdec mi_sys mi_fb mi_hdmi mi_gfx mi_vdisp mi_shadow mi_disp mi_os mi_iae mi_md mi_od					
Parameter Description	[Level] 0 No Debug Information (MI_DBG_NONE) 1 only show error Information (MI_DBG_ERR) 2 show level<=2 Information (MI_DBG_WRN) 3 show level<=3 Information (MI_DBG_API)					

	4 show level<=4 Information (MI_DBG_KMSG) 5 show level<=5 Information (MI_DBG_INFO) 6 show level<=6 Information (MI_DBG_DEBUG) 7 show level<=7 Information (MI_DBG_TRACE) 8 show all Information (MI_DBG_ALL)
Example	echo mi_sys=2 > /proc/mi_modules/mi_log_info The debug_level of mi_sys is modified to 2

Function	Modify log path
Command	echo log=[Path] > /proc/mi_modules/mi_log_info
Parameter Description	[Path] path
Example	echo log=/mnt > /proc/mi_modules/mi_log_info

Function	Modify path to store log
Command	echo storepath=[Path] > /proc/mi_modules/mi_log_info
Parameter Description	[Path] path
Example	echo storepath=/mnt > /proc/mi_modules/mi_log_info

5.3. mi_global_info

5.3.1 cat

➤ Debug Information

```
# cat /proc/mi_modules/mi_global_info
```

miu_and_lx_info:

```
ARM_MIU0_BUS_BASE 0x20000000    ARM_MIU0_BASE_ADDR 0x0
ARM_MIU1_BUS_BASE 0x60000000    ARM_MIU1_BASE_ADDR 0x80000000
ARM_MIU2_BUS_BASE 0xffffffff    ARM_MIU2_BASE_ADDR 0xffffffff
lx_mem_addr 0x20c00000          lx_mem_size 0xe300000
lx_mem2_addr 0xffffffff         lx_mem2_size 0xffffffff
lx_mem3_addr 0xffffffff         lx_mem3_size 0xffffffff
```

KernelProtect IP white list:

clientId	name
0x70	CPU_W
0x0f	MCU51_RW
0x12	USB20_H_RW
0x11	USB20_RW

0x10	USB30_RW
0x17	EMAC_RW
0x18	EMAC1_RW
0x13	SD30_RW
0x14	SDIO30_RW
0x04	AESDMA_RW
0x07	URDMA_RW
0x02	BDMA_RW
0x03	MOVDMA0_RW
0x16	SATA_RW
0x00	NONE
0x00	NONE

MmuProtect IP white list:

clientId	name
0x05	CMDQ0_R
0x06	CMDQ1_R
0x70	CPU_W
0x73	CPU_R
0x20	VCODEC_R
0x21	VCODEC_W
0x00	NONE
0x00	NONE
0x00	NONE
0x00	NONE
0x00	NONE
0x00	NONE
0x00	NONE
0x00	NONE
0x00	NONE
0x00	NONE
0x00	NONE

Sigmastar Module mi_sys version: project_commit.81f869a sdk_commit.513c3cc
build_time.20211029104555

- Debug Information Analysis
The Debug Information provide some global information.

- Parameter Analysis

Parameter		Description
miu_and_lx_info (Take only one MIU as an example)	ARM_MIU0_BUS_BASE	Miu0 bus base
	ARM_MIU0_BASE_ADDR	Miu0 base addr
	lx_mem_addr	The starting address of the memory occupied by the Linux image (Belong to cpu bus address)
	lx_mem_size	The memory size occupied by the Linux image
kernelProtect IP white list	clientId	Miu protect IP id in IP whitelist (The global id from the perspective of undivided group)
	name	The actual name of the IP corresponding to the clientId
	PAGE_OFFSET	the start virtual address of the kernel image
	TASK_SIZE	the maximum size of a user space task
	VMALLOC_START	the start address of vmalloc memory
	VMALLOC_END	the end address of vmalloc memory
Sigmastar Module mi_sys version	project_commit	Commit corresponding to the project
	sdk_commit	Commit corresponding to sdk
	build_time	Sdk build time

5.4. mi_bufqueue_status

5.4.1 cat

- Debug Information

cat /proc/mi_modules/mi_bufqueue_status

```
/ # cat /proc/mi_modules/mi_bufqueue_status
dump Queues in input port only for enabled port:
ModId  DevId  ChnId  PassId  InPortId  UsrInjectQ_cnt  BindInQ_cnt  InputPendingQueueSize
33      0      0      0      0          0              2            0
34      0      0      0      0          0              4            0
2       0      0      0      0          0              0            0
dump Queues in output port only for enabled port:
ModId  DevId  ChnId  PassId  OutPortId  DrvBkRefFifoQ_cnt  DrvBkRefFifoQ_size  UsrGetFifoQ_cnt  UsrGetFifoQ_size
6       0      0      0      0          0              0              0              0
33      0      0      0      0          0              0              0              0
34      0      0      0      0          0              0              0              0
```

- Debug Information Analysis
Dump current each modules's related Queue information of enable input/output port.

➤ Parameter Description

Parameter		Description
dump Queues in input port only for enabled port	ModId	The module id of the input port.
	DevId	The device id of the input port.
	ChnId	The channel id of the input port.
	PassId	The pass id of the input port.
	InPortId	The input port id.
	UsrInjectQ_cnt	The number of buffers in the input port's UsrInjectBufQueue.
	BindInQ_cnt	The number of buffers in the input port's UsrPipeInBufQueue.
	InputPendingQueueSize	The number of buffers in the input port's cur_working_input_queue.

Parameter		Description
dump Queues in output port only for enabled port	ModId	The module id of the output port.
	DevId	The device id of the output port.
	ChnId	The channel id of the output port.
	PassId	The pass id of the output port.
	OutPortId	The output port id.
	DrvBkRefFifoQ_cnt	The number of buffers in the input port's DrvBkRefFifoQueue.
	DrvBkRefFifoQ_size	The total size of buffers in the input port's DrvBkRefFifoQueue.
	UsrGetFifoQ_cnt	The number of buffers in the input port's UsrGetFifoBufQueue.
	UsrGetFifoQ_size	The total size of buffers in the input port's UsrGetFifoBufQueue.

5.5. mi_dump_buffer_delay_worker

5.5.1 cat

➤ Debug Information

cat /proc/mi_modules/mi_dump_buffer_delay_worker

```
delay_worker_id module_name force_stop dev_id chn_id port_type port_id Queue_name
0 mi_disp 0 0 0 inport 0 BindInput
stored_dir dump_method dump_method_value
/mnt bunfnun 2
```

➤ Debug Information Analysis

The file of /proc/mi_modules/mi_dump_buffer_delay_worker connects with the mi_modulename_{dev}id. The buffer in the Queue of the dump device is implemented by the delay worker. So cat /proc/mi_modules/mi_dump_buffer_delay_worker can see which delay workers are in progress, and echo

force_stop_dump delay_worker_id can force the end of a specified delay worker process.

➤ Parameter Description

Parameter	Description
delay_worker_id	The delay worker's id. If the delay worker completes, the id will be recycled and used as an id for other delay workers.
module_name	The module id corresponding to the delay worker.
force_stop	Whether the delay worker is currently in the compulsory end stage, 1 means that the compulsory end has been set, and 0 means that it has not been set.
dev_id	The device id corresponding to the delay worker.
chn_id	The channel id corresponding to the delay worker.
port_type	The port type corresponding to the delay worker, input port or output port.
port_id	The port id corresponding to the delay worker.
Queue_name	The Queue name corresponding to the delay worker.
stored_dir	Absolute path to store data dumped in the delay worker. (The file name of the final storage is not included)
dump_method	bufnum or time or start.
dump_method_value	The value corresponding to the dump method.

5.5.2 echo

Function	Force to stop delay worker
Command	echo force_stop_dump [WorkID] >/proc/mi_modules/mi_dump_buffer_delay_worker
Parameter Description	[WorkID] : delay_work 的 ID
Example	echo force_stop_dump 0 > /proc/mi_modules/mi_dump_buffer_delay_worker //Force to stop delay_work 0

5.6. mi_infer_graph.py

5.6.1 cat

➤ Debug Information

cat /proc/mi_modules/mi_infer_graph.py

```

/ # cat /proc/mi_modules/mi_infer_graph.py
"""
Sigmaster mi_sys infergraph debug utility.
please run it on linux with python installed.
Please run 'sudo apt-get install python-pydot' to make sure python-pydot been i
nstalled at first.
"""
try:
    # Try to load pydotplus
    import pydotplus as pydot
except ImportError:
    import pydot

def main():
    file_name = __file__
    base_name = file_name[file_name.find('.')]
    output_filename = base_name + "mi_sys_infer_DAG.png"
    pydot_graph = pydot.Dot("infer_graph", graph_type='digraph', rankdir='LR')
    pydot_nodes = {}
    for node in pydot_nodes.values():
        pydot_graph.add_node(node)

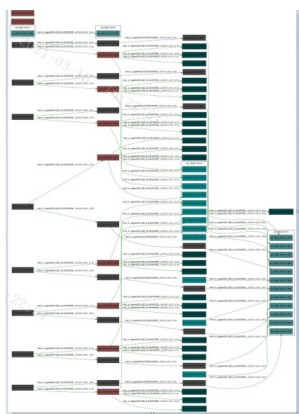
    ext = output_filename[output_filename.rfind('.')+1:]

    with open(output_filename, 'wb') as fid:
        fid.write(pydot_graph.create(format=ext))

```

➤ Debug Information Analysis

Sys provides a python script for proc fs to generate runtime information, which can be cat saved to the Linux server, then copy it to the text and run it on the Linux server using pyhton2, and two pictures will be output. The default image format is the simplified version, and one is "infer_DAG ", the other is "perf_static ". One of the output pictures is as follows:



The rectangle represents channel pass name;

The label on the line indicates from which input port to which output port, as well as bind type, source frame rate and dest frame rate.

5.7. mi_graph_ctrl

5.7.1 cat

➤ Debug Information

```
# cat /proc/mi_modules/mi_graph_ctrl
```

```

/ # cat /proc/mi_modules/mi_graph_ctrl
dump bind off

```

➤ Debug Information Analysis

The "dump bind" field of the above command indicates mi_infer_graph.py output style, "off" indicates output simplified binding graph, "on" indicates output detailed binding graph.

5.7.2 echo

Function	Control the image style output by Python scripts.
Command	echo [command] >/proc/mi_modules/mi_graph_ctrl
Parameter	[command] Command:
Description	dump bind on: control output detailed binding graph dump bind off: control output simplified binding graph
Example	output detailed binding graph : # echo dump bind on > /proc/mi_modules/mi_graph_ctrl # cat /proc/mi_modules/mi_infer_graph.py

5.8. debug_level

5.8.1 cat/echo

➤ Debug Information

```
# cat /proc/mi_modules/mi_sys/debug_level
```

2

➤ Debug Information Analysis

Each module (including the sys module) has its own debug level to control the printing level. The respective printing level is controlled in /proc/mi_modules/mi_modulename/debug_level respectively, where the modulename is shaped like disp, dipp, rgn and so on. The above is just taking /proc/mi_modules/mi_sys/debug_level as an example.

Function	Print debug warning level.
Command	cat /proc/mi_modules/[ModuleName]/debug_level
Parameter	[ModuleName]
Description	mi_disp mi_gfx mi_rgn mi_vdec mi_ai mi_shadow mi_vdisp mi_ao mi_hdmi mi_sys mi_venc mi_bar mi_vif
Example	cat /proc/mi_modules/mi_sys/debug_level 2 The debug warning level of mi_sys is 2 (show warning and error's information)

Function	Modify warning level.
Command	echo [Level] > /proc/mi_modules/[ModuleName]/debug_level
Parameter	[Level] 0 No debug information (MI_DBG_NONE)
Description	1 Only show error information (MI_DBG_ERR)
n	2 show level<=2 information (MI_DBG_WRN)
	3 show level<=3 information (MI_DBG_API)
	4 show level<=4 information (MI_DBG_KMSG)
	5 show level<=5 information (MI_DBG_INFO)

	6 show level<=6 information (MI_DBG_DEBUG) 7 show level<=7 information (MI_DBG_TRACE) 8 show all information (MI_DBG_ALL)
	[ModuleName] mi_disp mi_gfx mi_rgn mi_vdec mi_ai mi_vdisp mi_ao mi_hdmi mi_sys mi_shadow mi_venc mi_bar mi_vif
Example	echo 1 > /proc/mi_modules/mi_vdec/debug_level Modify the warning level of the mi_vdec module to only show error information.

5.9. debug_file

5.9.1 echo

- Debug Information
echo mi_sys_impl.c > /proc/mi_modules/mi_sys/debug_file
- Debug Information Analysis
When used for debugging, all levels of DBG log Information in the mi_sys_impl.c file can be printed on the serial port.
- Parameter Description

Parameter	Description
Command	echo [fileName] ... > /proc/mi_modules/mi_xxx/debug_file Indicates that the DBG log information of all levels in the function fileName can be printed at runtime. Multiple functions can be enabled at the same time, separated by spaces. echo > /proc/mi_modules/mi_xxx/debug_file Indicates to close the printing of all levels of the previously opened function fileName. At this time, the DBG log in the function fileName can only be printed at the level set by /proc/mi_modules/mi_xxx/debug_level. PS: fileName must belong to the corresponding mi_xxx module.
Parameter Description	[mi_xxx] Indicates the name of the MI module, which can be: mi_disp mi_gfx mi_rgn mi_vdec mi_ai mi_vdisp mi_ao mi_hdmi mi_sys mi_shadow mi_venc mi_bar mi_vif

5.10. debug_func

5.10.1 echo

- Debug Information
echo MI_SYS_IMPL_Init > /proc/mi_modules/mi_sys/debug_func

➤ Debug Information Analysis

When used for debugging, all levels of DBG log Information in the MI_SYS_IMPL_Init() function can be printed on the serial port.

➤ Parameter Description

Parameter	Description
Command	echo [funcName] ... > /proc/mi_modules/mi_xxx/debug_func Indicates that the DBG log information of all levels in the function funcName can be printed at runtime. Multiple functions can be enabled at the same time, separated by spaces. echo > /proc/mi_modules/mi_xxx/debug_func Indicates to close the printing of all levels of the previously opened function fileName. At this time, the DBG log in the function funcName can only be printed at the level set by /proc/mi_modules/mi_xxx/debug_level. PS: funcName must belong to the corresponding mi_xxx module.
Parameter Description	[mi_xxx] Indicates the name of the MI module, which can be: mi_disp mi_gfx mi_rgn mi_vdec mi_ai mi_vdisp mi_ao mi_hdmi mi_sys mi_shadow mi_venc mi_bar mi_vif

5.11. module_version_file

5.11.1 cat

➤ Debug Information

```
# cat /proc/mi_modules/mi_ai/module_version_file
```

```
MStar Module version: project_commit.sdk_commit.build_time c1799df.fd2d52b.20171225180033
```

```
# cat /proc/mi_modules/mi_global_info
```

```
.....
```

```
MStar Module version: project_commit.sdk_commit.build_time cb68bfd.44aca45.20171226100257
```

```
.....
```

➤ Debug Information Analysis

xxxx_version_file provides a version Information, and /proc/mi_modules/mi_global_info also provides a version Information. But it essentially refers to the version Information of the mi_sys module. The above are just examples of mi_ai and mi_global_info. Each module has its own version file.

➤ Parameter Description

Parameter	Description
Version Info	project_commi
	When the module compiles ko, the project commit information

Parameter		Description
	t	of the whole package. If the commit based on the module changes when ko is replaced separately, then the commit obtained in the version of ko will also change accordingly.
	sdk_commit	When the module compiles ko, the sdk commit information of the whole package. If the commit based on the module changes when ko is replaced separately, then the commit obtained in the version of ko will also change accordingly.
	build_time	Time refers to the actual build time, accurate to seconds, even if make clean; When you build image as a whole, the time of each ko will also be different.

5.12. show threads

5.12.1 echo

➤ Debug Information

echo show_threads > /proc/mi_modules/mi_sys/mi_sys0

```
[<c028099f>] (__schedule) from [<c0280a8f>] (schedule+0x57/0x64)
[<c0280a8f>] (schedule) from [<c001e39d>] (do_wait+0xf1/0x128)
[<c001e39d>] (do_wait) from [<c001e643>] (SyS_wait4+0x47/0x88)
[<c001e643>] (SyS_wait4) from [<c000d261>] (ret_fast_syscall+0x1/0x4c)
[<c028099f>] (__schedule) from [<c0280a8f>] (schedule+0x57/0x64)
[<c0280a8f>] (schedule) from [<c002c647>] (kthreadd+0x65/0xfa)
[<c002c647>] (kthreadd) from [<c000d331>] (ret_from_fork+0x11/0x20)
[<c028099f>] (__schedule) from [<c0280a8f>] (schedule+0x57/0x64)
[<c0280a8f>] (schedule) from [<c002e161>] (smpboot_thread_fn+0xd9/0x144)
[<c002e161>] (smpboot_thread_fn) from [<c002bf77>] (kthread+0xc3/0xd4)
[<c002bf77>] (kthread) from [<c000d331>] (ret_from_fork+0x11/0x20)
[<c028099f>] (__schedule) from [<c0280a8f>] (schedule+0x57/0x64)
[<c0280a8f>] (schedule) from [<c0029487>] (worker_thread+0x23f/0x298)
[<c0029487>] (worker_thread) from [<c002bf77>] (kthread+0xc3/0xd4)
[<c002bf77>] (kthread) from [<c000d331>] (ret_from_fork+0x11/0x20)
[<c028099f>] (__schedule) from [<c0280a8f>] (schedule+0x57/0x64)
[<c0280a8f>] (schedule) from [<c0029487>] (worker_thread+0x23f/0x298)
[<c0029487>] (worker_thread) from [<c002bf77>] (kthread+0xc3/0xd4)
[<c002bf77>] (kthread) from [<c000d331>] (ret_from_fork+0x11/0x20)
[<c028099f>] (__schedule) from [<c0280a8f>] (schedule+0x57/0x64)
[<c0280a8f>] (schedule) from [<c02830e9>] (schedule_timeout+0x137/0x16a)
[<c02830e9>] (schedule_timeout) from [<c0049ad3>] (rcu_gp_kthread+0x4af/0x548)
[<c0049ad3>] (rcu_gp_kthread) from [<c002bf77>] (kthread+0xc3/0xd4)
```

➤ Debug Information Analysis

View the call stack status of all current threads.

- This command can help Query & locate deadlock problems.
- This Command prints a lot, so you'd better to print it in kmsg to avoid contamination by other information.

5.13. debug_frc

5.13.1 echo

➤ Debug Information

echo debug_frc on 12 0 0 0 0 "out" > /proc/mi_modules/mi_sys/mi_sys0

- Debug Information Analysis
printk in _MI_SYS_IMPL_Common_WriteProc,
Switch enable debug FRC, Modid:12, Dev:0, Chn:0, Pass:0, Port:0, BufType:1. Enable frame rate control debug switch.

➤ Parameter Description

Parameter	Description
Command	Switch enable debug FRC: echo debug_frc on [Modid] [Devid] [Chnid] [Passid] [Portid] [in/out] > /proc/mi_modules/mi_sys/mi_sys0 Open the frame rate control debug switch. Switch disable debug FRC: echo debug_frc off > /proc/mi_modules/mi_sys/mi_sys0 Close the frame rate control debug switch.
Parameter Description	[Modid] : module id. [Devid] : device id. [Chnid] : channel id. [Passid] : pass id. [Portid] : port id. [in/out] : input/output direction.

5.14. set_outputport_depth

5.14.1 echo

- Debug Information
echo set_outputport_depth 6 0 0 0 0 2 4 > /proc/mi_modules/mi_sys/mi_sys0

➤ Parameter Description

Parameter	Description
Command	Set output port depth:echo set_outputport_depth [Modid] [Devid] [Chnid] [Passid] [Portid] [u32userFrameDepth] [u32BufQueueDepth] > /proc/mi_modules/mi_sys/mi_sys0
Parameter Description	[Modid] : module id. [Devid] : device id. [Chnid] : channel id. [Passid] : pass id. [Portid] : port id. [u32userFrameDepth]: Set the maximum number of bufs that the output user can get. [u32BufQueueDepth]: Set the maximum number of bufs for this

Parameter	Description
	output system

5.15. set_thread_priority

5.15.1 echo

➤ Debug Information

```
# echo set_thread_priority 4 0 99 > /proc/mi_modules/mi_sys/mi_sys0
# echo set_thread_priority 8 849 99 > /proc/mi_modules/mi_sys/mi_sys0
```

➤ Parameter Description

Parameter	Description
Command	Set work_thread_priority by module id: echo set_thread_priority [Modid] [Devid] [u32Priority] > /proc/mi_modules/mi_sys/mi_sys0 set_thread_priority by pid: echo set_thread_priority 8 [pid] [u32Priority] > /proc/mi_modules/mi_sys/mi_sys0
Parameter Description	[Modid]: module id. [Devid]: device id. [u32Priority]: Thread level to be set. [pid]: Thread pid.

5.16. mmu debug

5.16.1 echo

➤ Debug Information

```
# echo debug_mmu debug_log 1 1 1 33,34 > /proc/mi_modules/mi_sys/mi_sys0
```

➤ Parameter Description

Parameter	Description
Command	Enable/disable alloc/free buf log: echo debug_mmu debug_log [enable/disable] [enable/disable free] [enable/disable alloc] [module id list] > /proc/mi_modules/mi_sys/mi_sys0
Parameter Description	[enable/disable] : enable or disable [module id list]: module list, Separated by commas

➤ Example

- 1.echo debug_mmu debug_log 1 --- enable alloc/free buf log
- 2.echo debug_mmu debug_log 1 0 --- enable alloc buf log
- 3.echo debug_mmu debug_log 1 1 0 --- enable free buf log
- 4.echo debug_mmu debug_log 1 1 1 4,6 --- enable ai & vif alloc/free buf log
- 5.echo debug_mmu debug_log 1 0 1 4,6 --- enable ai & vif alloc buf log
- 6.echo debug_mmu debug_log 1 1 0 4,6 --- enable ai & vif free buf log

5.16.2 insmod

- Debug Information
insmod /config/modules/4.9.227/mi_sys.ko bDebugMmu=1

- Parameter Description

Parameter	Description
Command	For debug memory access out of bounds: insmod /config/modules/4.9.227/mi_sys.ko bDebugMmu=1/bDebugMmu=0 # Depends on enable mmu
Parameter Description	bDebugMmu=1 :enable bDebugMmu=0 :disable default disable

5.17. set_inputport_pattern

5.17.1 echo

- Debug Information
echo set_inputPort_pattern 33 0 0 0 0 1 > /proc/mi_modules/mi_sys/mi_sys0

- Debug Information Analysis
Set to enable ModId33 DevId0 ChnId0 PassId0 InputPortId0, at this time, ModId33 module in pipeline only keeps one buf data to pass data to the post-module (The bindtype of ModId33 and post-module need to be set frame mode) ,which is used for debugging to locate which ModId's buf data is faulty.

- Parameter Description

Parameter	Description
Command	echo set_inputport_pattern [Modid] [Devid] [ChnId] [PassId] [InputPortId] [bEnable] > /proc/mi_modules/mi_sys/mi_sys0 Set a certain inputPort of ModId to retain only a piece of buf data to transfer data to the subsequent ModId (the bindType of the ModId and the subsequent ModId need to be set to frame mode), which is used for debugging to locate which ModId's buf data is faulty.
Parameter Description	[Modid] : module id. [Devid] : device id. [ChnId] : channel id. [PassId] : pass id. [inputPortId] : inputPort Id. [bEnable] : Whether to enable, 0/1.

5.18. mi_sys0

5.18.1 cat

➤ Debug Information

```
# cat /proc/mi_modules/mi_sys/mi_sys0
```

Chnid	Passid	Portid	Enable	bind_module_id	bind_module_name	bind_devId	bind_Chnid	bind_Portid	bind_Type	bind_Param	enable							
-----module: mi_vif devId:0-----																		
chn	pre	suc	busy	drop	Enq	run	done	retry	drop	bar	checkin	run	done	retry	drop	checkout	run	done
0	472	8ea	0	0	46d	0	0	0	0	0	0	0	0	0	0	0	0	0
-----module: mi_isc devId:0-----																		
chn	pre	suc	busy	drop	Enq	run	done	retry	drop	bar	checkin	run	done	retry	drop	checkout	run	done
0	46a	0	0	0	46a	0	0	0	0	469	0	0	0	0	0	0	0	0
-----module: mi_scl devId:0-----																		
chn	pre	suc	busy	drop	Enq	run	done	retry	drop	bar	checkin	run	done	retry	drop	checkout	run	done
0	46a	0	0	0	46a	0	0	0	0	469	0	0	0	0	0	0	0	0
-----module: mi_venc devId:0-----																		
chn	pre	suc	busy	drop	Enq	run	done	retry	drop	bar	checkin	run	done	retry	drop	checkout	run	done
0	45f	0	0	0	45c	3	0	0	0	3	0	0	0	0	0	0	0	0
-----module: mi_venc devId:8-----																		
chn	pre	suc	busy	drop	Enq	run	done	retry	drop	bar	checkin	run	done	retry	drop	checkout	run	done
31	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

➤ Debug Information Analysis

This command shows the relevant information processed by the data stream buf of each modId module when the current device is in the pipeline.

➤ Parameter Description

Parameter		Description
(mod-dev-chn info)	module	module Id, which can be: mi_disp mi_gfx mi_rgn mi_vdec mi_ai mi_shadow mi_vdisp mi_ao mi_hdmi mi_sys mi_venc mi_bar mi_vif
	devId	device Id
	chn	channel Id
pre (Consult whether the current modId can handle the handleinfo of buf)	suc	The number of times that the current modId has successfully preprocessed buf.
	busy	The number of times that the current modId has successfully preprocessed buf.
	drop	The number of drop operations performed by the current modId on the preprocessed buf.
Enq (Add buf to handleinfo in the queue of the current modId)	run	The current modId enqueue operation returns the number of times in the running state.
	done	The current modId has been processed to complete the number of bufs in the queue.
	retry	The current number of modId retrying to process buf in the queue.
	drop	The number of times the current modId drops the pending buf in the queue.
	bar	Cmdq related.
checkin	run	The number of times that the current

Parameter		Description
(inputport buf handleinfo)		modId processed the inputport buf operation to return to the running state.
	done	The current modId has processed the number of bufs in the inputport.
	retry	The current modId retry to process the coefficient of inputport buf.
	drop	The number of times the current modId drops the pending input port buf.
checkout (outputport buf handleinfo)	run	The current modId processing outputport buf operation returns the number of times in the running state.
	done	The number of times that the current modId has been processed to complete the buf in the outputport.
	retry	The current number of modId retries to process the outputport buf.
	drop	The number of times the current modId drops the pending outputport buf.

5.19. miu_protect

5.19.1 cat

➤ Debug Information

```
# cat /proc/mi_modules/mi_sys_mma/miu_protect
```

```
===== start miu_protect_info =====
```

```
kernel protect enabled
```

```
LX :
```

```
cpu_start_addr:0x20c00000 size:0xe300000
```

```
miu_index miuBlockIndex start_cpu_bus_pa length
```

```
0x0 0x00 0x20c00000 0x460000
```

```
KernelProtect IP white list:
```

clientId	name
43	MIU_CLIENT_MIPS_RW
50	MIU_CLIENT_NAND_RW
82	MIU_CLIENT_USB_UHC0_RW
83	MIU_CLIENT_USB_UHC1_RW
84	MIU_CLIENT_USB_UHC2_RW
18	MIU_CLIENT_G3D_RW
140	MIU_CLIENT_USB3_RW
129	MIU_CLIENT_SDIO_RW
165	MIU_CLIENT_SATA_RW

```
133      MIU_CLIENT_USB_UHC3_RW
225      MIU_CLIENT_USB30_1_RW
226      MIU_CLIENT_USB30_2_RW
5        MIU_CLIENT_BDMA_RW
14       MIU_CLIENT_EMAC_RW
```

- Debug Information Analysis
This command shows the related Information of miu protect.

- Parameter Analysis

Parameter		Description
LX(Take only one LX for example, LX represents memory corresponding to linux image)	cpu_start_addr	The starting CPU addr corresponding to this LX.
	size	The size corresponding to this LX.
Information about the range of a certain kernel protect	miu_index	Index number.
	miuBlockIndex	Total 4 miu range's index information.
	start_cpu_bus_pa	The starting cpu bus addr of this LX.
	length	The length of the range.
KernelProtect IP white list	clientId	Miu protect IP id in IP whitelist (The global id from the perspective of undivided group).
	name	The actual name of the IP corresponding to the clientId.

5.20. mma_heap_name0

5.20.1 cat

- Debug Information

```
# cat /proc/mi_modules/mi_sys_mma/mma_heap_name0
```

```
heap_info:
```

```
heap_name  pa_start  length  avail
mma_heap_name0  40000000  1f000000  1b238000
```

```
heap_param:
```

```
mmuEnable  addsize_before  addsize_after  delayUnmap  freeEntryNum
1          0      0      0      361
```

```
main_chunk_mgr:
```

```
vpa_start  length  avail  used  HighPeak  LowPeak
```

60000000 20000000 1c238000 3dc8000 40c5000 d40000

offset	length	used_flag	task_name	pid	Module
0	300000	1	CMDMEM	-1	mi_sys
300000	1000	1	sys-logConfig	-1	mi_sys
301000	40000	1	sys-logBuffer	-1	mi_sys
341000	1fb000	1	vif0-out0-0	867	mi_vif
53c000	1fb000	1	vif0-out0-0	867	mi_vif
737000	1fb000	1	vif0-out0-0	867	mi_vif
932000	3fa000	1	mma_heap_codec	-1	mi_sys
d2c000	21000	1	ven_mv_0	867	mi_venc
d4d000	23000	1	ven_fcty_0	867	mi_venc
d70000	12000	1	ven_fctc_0	867	mi_venc
d82000	41000	1	ven_rdo_0	867	mi_venc
dc3000	1000	1	ven_inst_0	867	mi_venc
dc4000	1000	1	venc-algo-m	867	mi_venc
dc5000	60000	0	NA	-1	mi_sys
e25000	a3000	1	ISP_STATIS	867	mi_isp
ec8000	1000	1	ISP_WDR	867	mi_isp
ec9000	22000	1	DNR_INFO1	867	mi_isp
eeb000	32a000	1	DNR_INFO0	867	mi_isp
1215000	6000	1	ISP_IDX_CMDQ	867	mi_isp
121b000	10000	1	ISP_CMDQ	867	mi_isp
122b000	4000	1	ISP_MLOAD	867	mi_isp
122f000	152000	1	RingHeapAlloc	-1	mi_sys
1381000	1a6000	0	NA	-1	mi_sys
1527000	4000	1	ISP_MLOAD	867	mi_isp
152b000	3f5000	1	venc-ring-mem0	867	mi_venc
1920000	3b1000	1	ven_rec_0_0	867	mi_venc
1cd1000	2fd000	0	NA	-1	mi_sys
1fce000	2000000	1	venc-ring-mem1	867	mi_venc
3fce000	2fd000	1	scl0-out0-0	867	mi_scl
42cb000	1bd35000	0	NA	-1	mi_sys

(mma_heap_codec)sub_chunk_mgr:

vpa_start	length	avail	used	HighPeak	LowPeak
60932000	3fa000	2df000	11b000	11b000	0

offset	length	used_flag	task_name	pid	Module
0	ea000	1	vcodec_dev	-1	mi_venc
ea000	31000	1	ven_work_ven_ta	-1	mi_venc
11b000	2df000	0	NA	-1	mi_sys

(CMDMEM)sub_chunk_mgr:

vpa_start	length	avail
60000000	300000	195000

offset	length	used_flag	cmdq_id	pid	ModDev
0	20000	0	-1	-1	NA
20000	16b000	1	1	0	mi_isp0
20000	16b000	1	1	0	mi_scl0
20000	16b000	1	1	0	mi_venc8
18b000	175000	0	-1	-1	NA

- Debug Information Analysis
The cat information corresponds to the basic information and current status of the mma heap.
- Parameter Description

Parameter		Description
heap_info	heap_name	The name of mma heap.
	pa_start	The start PA of the heap.
	length	The whole length of mma heap.
	avail	The total amount of unused memory remaining in the heap..
heap_param	mmuEnable	Enable mmu

Parameter		Description
	addrsz_before	Set the size of the reserved application before the application buf.
	addrsz_after	Set the size of the reserved application after the application buf.
	delayUnmap	Whether to enable the delayed unmap currently applied for buf when buf free, when it is applied again next time, first unmap and then map. 1: Enable to delay unmap 0: Disable to delay unmap
	freeEntryNum	Free entry number
main_chunk_mgr	vpa_start	Main chunk mgr start address. It name "vpa_start" when mmu is enable, or it will name "pa_start".
	length	The entire mma vpa length managed by chunk mgr. Since the entire mma heap is used as a chunk mgr, when mmu is turned off, the value is equal to mma heap length; When mmu is turned on, the value is greater than mma heap length.
	avail	The total amount of unused memory remaining in chunk mgr (heap).
	used	The total amount of used memory remaining in chunk mgr (heap).
	HighPeak	The peak memory counted over a period of time.
	LowPeak	The valley memory counted over a period of time.
	offset	The offset of this chunk in chunk mgr.
	length	The length of this chunk.

Parameter		Description
	used_flag	Whether the chunk is allocated. If yes, the value is 1; otherwise, the value is 0.
	task_name	If the used flag is 1, then task_name stores which task uses it; otherwise, it is an invalid value NA.
	pid	Invalid at present.
	Module	Which Module used the chunk.
(mma_heap_codec)sub_chunk_mgr	(mma_heap_codec)sub_chunk_mgr	The sub chunk mgr of main chunk mgr. It is the chunk name "mma_heap_codec" in main chunk mgr. About the parameter of this chunk mgr, please refer to the parameter of main chunk mgr. It doesn't show information of sub chunk mgr by default. After set one command, it perhaps show more than one sub chunk mgr, or show nothing for sub chunk mgr. The command is : <pre>echo show_privMma 1 > /proc/mi_modules/mi_sys/mi_sys0</pre>
(CMDMEM)sub_chunk_mgr	(CMDMEM)sub_chunk_mgr	The sub chunk mgr of main chunk mgr. It is the chunk name "CMDMEM" in main chunk mgr. This sub chunk mgr is show by default, it show the cmdq mma buf usage.
	vpa_start	(CMDMEM)sub chunk mgr start address. It name "vpa_start" when mmu is enable, or it will name "pa_start".
	length	The length of mma buf cmdq used.

Parameter		Description
	avail	The total amount of unused memory remaining in CMDMEM chunk mgr.
	offset	The offset of this chunk in chunk mgr.
	length	The length of this chunk.
	used_flag	Whether the chunk is allocated. If yes, the value is 1; otherwise, the value is 0.
	cmdq_id	The cmdq id witch use the chunk.
	pid	Invalid at present.
	ModDev	The module name and device id witch use the cmdq buf.

5.20.2 echo

Function	Dump the data of the specified offset and length to the specified path.
Command	echo [Path] [Offset] [Length] > /proc/mi_modules/mi_sys_mma/mma_heap_name[Num]
Parameter Description	[Path] The path to save the file, only need to provide the path, do not provide the specific file name, the system will automatically generate the corresponding file name according to the parameter.
	[Offset] Dump data from the offset of the mma heap, which must be 4KB aligned.
	[Length] The total amount of data dumped in the mma heap, must be 4KB aligned.
	[Num] The number corresponding to mma head, mma_heap_name0 can be viewed through cat /proc/cmdline at present.
Example	echo /mnt/ 0 4096 > /proc/mi_modules/mi_sys_mma/mma_heap_name0 Dump mma__mma_heap_name0__0__4096.bin file under /mnt.

5.21. imi_mma_heap

5.21.1 cat

➤ Debug Information

```
# cat /proc/mi_modules/mi_sys_mma/imi_mma_heap
```

```
mma heap nameheap_base_cpu_bus_addr      length  chunk_mgr_avail
mmuEnable  addsize_before  addsize_after  bMmuDelayUnmap  free_entry_num
imi_mma_heap      20000000      43800      26800      0
0      0      0      0
```

chunk_mgr info:

offset	length	avail	used	HighPeak	LowPeak
0	43800	26800	1d000	1d000	0

each chunk info:

offset	length	used_flag	task_name	pid	Module
0	1d000	1	SRAM	-1	mi_venc
1d000	26800	0	NA	-1	mi_sys

➤ Debug Information Analysis

The cat information corresponds to the basic information and current status of the mma heap.

➤ Parameter Analysis

Parameter		Description
heap basic info	mma heap name	mma heap name
	heap_base_cpu_bus_addr	The unused amount of allocations in allocator.
	length	The length of heap.
	chunk_mgr_avail	The total amount of unused memory remaining in heap.
	mmuEnable	Whether Mma heap support HW_MMU. 1 means enable, 0 means disable.
	addrsz_before	Set the reserved memory space size before the requested buf.
	addrsz_after	Set the reserved memory space size after the requested buf.
	bMmuDelayUnmap	Whether to delay unmap the current allocated buf when buf free is applied. When alloc buf next time, firstly unmap before mapping. 1: enable delaying unmap 0: disable delaying unmap
chunk_mgr info	free_entry_num	the free entry num
	offset	The offset of chunk mgr. Since the entire mma heap is used as a chunk mgr, this value is always 0.
	length	The length of chunk mgr. Since the entire mma heap is used as a chunk mgr, this value is always mma heap length.
	avail	The total free memory length in chunk mgr.

Parameter		Description
	used	The total used memory length in chunk mgr.
	HighPeak	The statistical memory peak value voer a period of time.
	LowPeak	The statistical memory valley value voer a period of time.
each chunk info (Information and usage of each chunk in chunk mgr)	offset	The offset of this chunk in chunk mgr.
	length	The length of this chunk.
	used_flag	Whether the chunk is allocated. If yes, the value is 1; otherwise, the value is 0.
	task_name	If the used flag is 1, then task_name stores which task uses it; otherwise, it is an invalid value NA.
	pid	current of the chunk ->tgid
	Module	The module of the chunk

5.22. meta

5.22.1 cat

➤ Debug Information

```
# cat /proc/mi_modules/mi_sys_mma/meta
```

```
===== start meta_info =====
```

```
basic info:
```

```
total page count:16      each meta data size:256  total_count_with_metadatasize 256
```

```
total_free_count_with_metadatasize:254
```

```
meta info :
```

```
ref_cnt=3
```

```
each allocation info:
```

offset_in_pool	length	phy_addr	va_in_kern	real_used_flag
0x0	0x100	0x2336000	c1736000	0
0x100	0x100	0x2336100	c1736100	0
0x200	0x100	0x2336200	c1736200	0
0x300	0x100	0x2336300	c1736300	0
0x400	0x100	0x2336400	c1736400	0
0x500	0x100	0x2336500	c1736500	0
0x600	0x100	0x2336600	c1736600	0
0x700	0x100	0x2336700	c1736700	0
0x800	0x100	0x2336800	c1736800	0
0x900	0x100	0x2336900	c1736900	0
0xa00	0x100	0x2336a00	c1736a00	0

```
.....
```

```
===== end meta_info =====
```

➤ Debug Information Analysis

Once meta allocator has been used (There is only one meta allocator),
/proc/mi_modules/mi_sys_mma/meta file will be generated, and the corresponding memory will allocate to be used.

5.22.2 echo

Function	Dump the data of the specified offset and length to the specified path.
Command	echo [Path] [Offset] [Length] > /proc/mi_modules/mi_sys_mma/vb_pool_global
Parameter Description	[Path] The path to save the file, only need to provide the path, do not provide the specific file name, the system will automatically generate the corresponding file name according to the parameter.
	[Offset] From which offset of the allocator to start dump data, it is the logical offset of the allocator.
	[Length] The size of the total amount of data dumped in the allocator.
Example	echo /mt 0 1024 > /proc/mi_modules/mi_sys_mma/meta Dump meta_0_1024.bin under /mnt.

5.23. mi_modulenameid

5.23.1 cat

➤ Debug Information

```
cat /proc/mi_modules/mi_disp/mi_disp0
```

```
/ # cat /proc/mi_modules/mi_scl/mi_scl0

-----Common info for mi_scl-----
ChnNum  EnChnNum  PassNum  InPortNum  OutPortNum  CollectSize
    34         1         1         1         6         0

-----CMDQ kickoff counter-----
PassId   CmdqId   TotalKickoff  KickoffFence  Idle
    0         1       2594       1732         1
Devid   current_buf_size  Peak_buf_size
    0         0         0

each dev buf info:
      offset          length      used_flag      task_name

-----Common info formi_scl only dump enabled chn-----
ChnId  PassNum  EnInPNum  EnOutPNum  MMAHeapName
    0         0         1         1      (null)
ChnId   current_buf_size  Peak_buf_size  user_pid
    0      8f7000      bf4000      849

each chn buf info:
      offset          length      used_flag      task_name
    9c9000      2fd000         1      scl0-out0-0
    1ee9000      2fd000         1      scl0-out0-0
    21e6000      2fd000         1      scl0-out0-0

-----chn0 pass0 pipeline delay in us-----
          Flow          Min          Max          Avg          Diff
OnPreProcessInputTask      92      44167      33103      33674
  EnqueueInputTask      225      44319      33302      33833
  BarrierInputTask      261      196698      33591      34024
  CheckInputTaskStatus      31720      246445      33634      33393
  DequeueInputTask      31723      246449      33637      33396
OnPollingAsyncOutputTaskConfig      -1         0         0         0
  EnqueueAsyncOutputTask      -1         0         0         0
  BarrierAsyncOutputTask      -1         0         0         0
  CheckOutputTaskStatus      31714      246461      33624      33383
  DequeueOutputTask      31726      246473      33637      33396
  FinDMADispatch      33419      42380      33553      33540
```

➤ Debug Information Analysis

The result of the cat operation is divided into two parts. The first part is the general Debug Information provided by MI_SYS for the device, which is provided from four perspectives: device, channel, input port, and output port; the second part is the private information of the device. The content of each module will explain the private Information elsewhere, here only the general Information will be explained from the SYS perspective.

Take cat /proc/mi_modules/mi_vdisp/mi_vdisp0 as an example to get the above result.

➤ Parameter Description

Parameter		Description
Common info for device	ChnNum	The channel number
	EnChnNum	The enabled channel number
	PassNum	The pass number
	InPortNum	The input port number
	OutPortNum	The output port number
	CollectSize	The total size of allocator contained in AllocatorCollection
CMDQ kickoff counter	PassId	The pass Id using cmdq in the device.
	CmdqId	The cmdq Id
	TotalKickoff	The current kickoff times of the cmdq.
	kickoffFence	The current kickoff fence value of the cmdq.
	Idle	The current Idle times of kickoff in the cmdq.
	DevId	The device Id.
	current_buf_size	The currnet used buf size of the device
	Peak_buf_size	The used buf peak value of the device.
each dev buf info	offset	The offset from buf.
	length	The length of buf.
	used_flag	The flag of recording whether buf is used.
	task_name	The name of task using the buf.
Common info for channel(only dump enabled channel)	ChnId	The channel Id.
	PassNum	The pass Id
	EnInPNum	The enabled input port number
	EnOutPNum	The enabled output port number.
	MMAHeapName	If SetChnMMAConf, the value is the corresponding mma heap name; otherwise, the value is NULL.
	current_buf_size	The current used buf size.
	Peak_buf_size	The current used buf peak value of the channel.

Parameter		Description
	user_pid	The task pid of the channel.
each chn buf info	offset	The offset from buf used by the channel.
	length	The buf length
	used_flag	The flag of recording whether buf is used.
	task_name	The name of task using the buf.
chn pass pipeline delay in us	Flow	OnPreProcessInputTask: MI PASS determines whether the current task needs to be processed, and if it can be configured for the output buffer; EnqueueInputTask: Hand over the input task to MI PASS to start processing; BarrierInputTask: Insert CMDQ synchronization instructions (such as waitevent, polling engine done); CheckInputTaskStatus: Check whether the input task is completed and the completion status (success, failure, etc); DequeueInputTask: Notify MI PASS to release the resources associated with the input task; OnPollingAsyncOutputTaskConfig: For MI Pass with asynchronous input and output, or MI Pass with only output without input, MI SYS queries whether there is an independent output buf request requirement; EnqueueAsyncOutputTask: Hand over the asynchronous output task to MI PASS for processing; BarrierAsyncOutputTask: Insert cmdq synchronization instructions for asynchronous output tasks; CheckOutputTaskStatus: Check whether the output task is completed and the

Parameter		Description
		completion status (success, failure, etc); DequeueOutputTask: Notify MI PASS to release the resources associated with the output task; FinDMADispatch: MI SYS transfers the buffer in the output task of MI PASS to the input queue of the bound post-level MI PASS.
	Min	The minimum delay between the buffer generation and the flow corresponding to the MI PASS.
	Max	The maximum delay from the buffer generation to the flow corresponding to the MI PASS.
	Avg	The average delay between the buffer generation and the flow corresponding to the MI PASS.
	Diff	The current delay elapsed from Buffer generation to the flow corresponding to the MI PASS.
Input port common info(only dump enabled Input port)	ChnId	Input port channel id
	PassId	
	PortId	Input port id
	SrcFrmrate	Src frame rate
	DstFrmrate	Dst frame rate
	user_buf_quota	The quota of buff count
	UsrInjectQ_cnt	The buff count of UsrInjectBufQueue
	UsrInjectQ_size	The buff size of UsrInjectBufQueue
	BindInQ_cnt	The buff count of BindInputBufQueue
	BindInQ_size	The buff size of BindInputBufQueue
	WorkingQ_cnt	The buff count of WorkingQueue
	WorkingQ_size	The buff count of WorkingQueue
	usrLockedInjectCnt	The buff count currently taken by user
Input port bind info(only dump enabled Input port)	ChnId	Channel id
	PortId	Input port id
	bind_module_id	The id of the module where the output port is bound to the input port
	bind_module_name	The name of the module where the output port is bound to the input port
	bind_ChnId	The id of the channel where the output

Parameter		Description
		port is bound to the input port.
	bind_PortId	The id of the port where the output port is bound to the input port.
Output port common info (only dump enabled Output port)	ChnId	Output port channel id
	PortId	Output port id
	BufCntQuota	The quota of buff count
	usrLockedCnt	The buff count actually taken by user in UsrGetFifoBufQueue
	totalOutPortInUsed	totalOutputPortInUsed buff count
	DrvBkRefFifoQ_cnt	DrvBkRefFifoQueue buff count
	DrvBkRefFifoQ_size	DrvBkRefFifoQueue buff count
	UsrGetFifoQ_cnt	UsrGetFifoBufQueue buff count
	UsrGetFifoQ_size	UsrGetFifoBufQueue buff size
	WorkingQ_cnt	WorkingQueue buff count
	WorkingQ_size	WorkingQueue buff size
Output port BindPeerInputPortList info(only dump enabled Output port)	ChnId	Output port channel id
	PortId	Output port id
	bind_module_id	The id of the module where one of the input ports binded to the output port (here referred to as the binded input port)
	bind_module_name	The name of the module where the binded input port is located
	bind_ChnId	The channel id where the binded input port is located
	bind_PortId	The portid where the binded input port is located

5.23.2 echo

Function	Get help information of the module.
Command	echo help > /proc/mi_modules/[ModName]/[ModID]
Parameter	[ModName]
Description	mi_disp mi_gfx mi_rgn mi_vdec mi_ai mi_shadow mi_vdisp mi_ao mi_hdmi mi_sys mi_venc mi_bar mi_vif [ModID] The corresponding file in the module is generally the module name + number Such as mi_disp0, mi_gfx0, mi_rgn0
Example	echo help > /proc/mi_modules/mi_disp/mi_disp0 Get help information of mi_disp0.

Function	Dump port information.
Command	echo dump_buffer [ChnID] [PortType] [PortID] [QueueName][Path] [EndMethod] > /proc/mi_modules/[ModName]/[ModID]
Parameter	[ChnID] Channel ID
Description	[PortType] iport or oport each represents input port or output port [PortID] Port ID [QueueName] If it is an input port, the value can only be "UsrInject" or "BindInput". If it is an output port, the value can only be "UsrGetFifo". [Path] The path of the file where the data is to be saved. Please note that it is an absolute path, do not provide the name of the file where the data is to be saved, the system will automatically generate the file name according to the Parameter, and different buffers are saved in different files. [EndMethod] The end method of dump buffer, currently only supports three types: (a) "bufnum=xxx": Dump the specified number of buffers (b) "time=xxx"(here unit is ms): The time corresponding to the lasting time of the dump process (c) "start/end" pair: Use start to start the dump, end to end the corresponding dump, and the other 5 parameters are exactly the same when these 2 Commands are executed. [ModName] mi_disp mi_gfx mi_rgn mi_vdec mi_ai mi_shadow mi_vdisp mi_ao mi_hdmi mi_sys mi_venc mi_bar mi_vif [ModID] The corresponding file in the module is generally the module name + number
Example	echo dump_buffer 0 iport 0 BindInput /mnt bufnum=1 > /proc/mi_modules/mi_disp/mi_disp0 Dump inport data of disp.

5.24. mem_stat

5.24.1 cat

➤ Debug Information

```
# cat /proc/mi_modules/mi_sys_mma/mem_stat
mi_sys
```

```

      CMDMEM      c0000
sys-logBuffer    40000
sys-logConfig     1000
      TotalSize    101000
      AllTotalSize 101000
```

➤ Debug Information Analysis

Print the memory status of mi sys.

➤ Parameter Description

Parameter		Description
Mem_stat	mi_sys	Owned mi_sys module
	CMDMEM	Cmd memory size
	sys-logBuffer	The maximum buf size applied to sys log
	sys-logConfig	The buf size allocated (applied) to sys log
	TotalSize	The size used by ModuleUsdTotalSize
	AllTotalSize	Total size